

【多易教育】SparkCore 教程



JUST DO IT
多易教育

1 Spark 概念简介

Spark 是一种快速、通用、可扩展的大数据引擎（运算引擎，类同 mapreduce）；

1.1 spark 发展历史

- 2009 年诞生于加州大学伯克利分校 AMPLab
- 2010 年开源
- 2013 年 6 月成为 Apache 孵化项目
- 2014 年 2 月成为 Apache 的顶级项目
- 2014 年 5 月发布 spark1.0
- 2016 年 7 月发布 spark2.0
- 2020 年 6 月 18 日发布 spark3.0.0

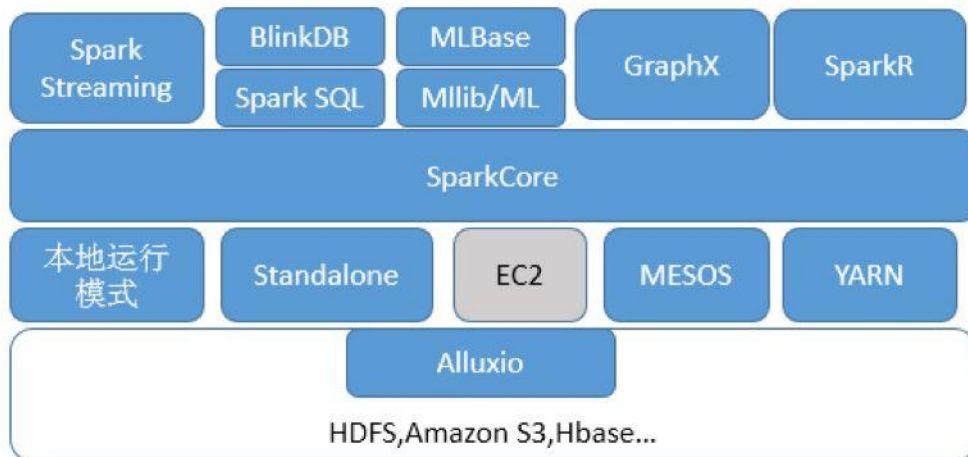
1.2 spark 特性

- 上层编程 api 丰富、强大
- 支持种类丰富的数据源
- 底层架构高效、先进
- 充分利用内存计算，提升运算速度

框架	优点	缺点
MapReduce	历史悠久、稳定	编程 API 不灵活、速度慢、只能做离线计算
Spark	通用、编程 API 简洁、快	有的时候没有 MR 程序稳定

1.3 spark 技术生态

- spark 自身拥有丰富的从 low-level 到 high-level 的库和组件
不同层次开发人员，不同场景需求，都有方便、适用的开发接口



- 大量基于 spark 开发的第三方应用
如，hive on spark，kylin 等

1.4 spark 使用方式

- 使用自带的 spark-shell 程序进行交互式数据分析（测试用）
- 用 idea 利用 api 开发自己的运算程序

spark 程序的运行模式：

- local 运行（可 linux，可 windows，可 idea）
- 分布式运行（先将程序开发好，然后提交到集群上分布式运行）
 - standalone 集群（spark 自带的一个资源调度集群）运行
 - yarn 集群运行
 - mesos 集群运行
 - k8s 容器化运行

2 Spark Core 编程体验

2.1 spark 开发工程搭建

开发语言选择

spark 提供了多种语言的开发 API，可以使用 Java，Scala，Python 等语言；

由于 spark 自身是用 scala 语言开发，而且 scala 语法（尤其是集合 api）与 spark 提供的数据计算 api 极为类似，因此，开发 spark 数据运算程序往往优先使用 scala 语言；

依赖管理工具

目前主流的依赖管理工具有 Maven，Sbt 等；

其中最为流行和通用的，还属 Maven；

因此，我们使用 Maven 进行依赖管理和编译、打包等处理；

```
<dependencies>
  <!-- scala 语言开发库 -->
  <dependency>
    <groupId>org.scala-lang</groupId>
    <artifactId>scala-library</artifactId>
    <version>2.12.12</version>
  </dependency>

  <!-- spark 依赖库 -->
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql_2.12</artifactId>
    <version>3.1.0</version>
  </dependency>
</dependencies>

<repositories>
  <!-- 阿里云 maven 镜像服务（依赖 jar 包） -->
  <repository>
    <id>nexus-aliyun</id>
    <name>Nexus aliyun</name>
    <layout>default</layout>
  </repository>
</repositories>
```

```
<url>http://maven.aliyun.com/nexus/content/groups/public</url>
<snapshots>
  <enabled>false</enabled>
  <updatePolicy>never</updatePolicy>
</snapshots>
<releases>
  <enabled>true</enabled>
  <updatePolicy>never</updatePolicy>
</releases>
</repository>
</repositories>

<pluginRepositories>
  <!-- 阿里云 maven 镜像服务 （依赖插件 jar 包） -->
  <pluginRepository>
    <id>ali-plugin</id>
    <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
    <snapshots>
      <enabled>false</enabled>
      <updatePolicy>never</updatePolicy>
    </snapshots>
    <releases>
      <enabled>true</enabled>
      <updatePolicy>never</updatePolicy>
    </releases>
  </pluginRepository>
</pluginRepositories>

<build>
  <plugins>
    <!-- scala 编译插件 -->
    <plugin>
      <groupId>net.alchim31.maven</groupId>
      <artifactId>scala-maven-plugin</artifactId>
      <version>3.2.2</version>
      <executions>
        <execution>
          <goals>
            <goal>compile</goal>
            <goal>testCompile</goal>
          </goals>
          <configuration>
            <args>
```

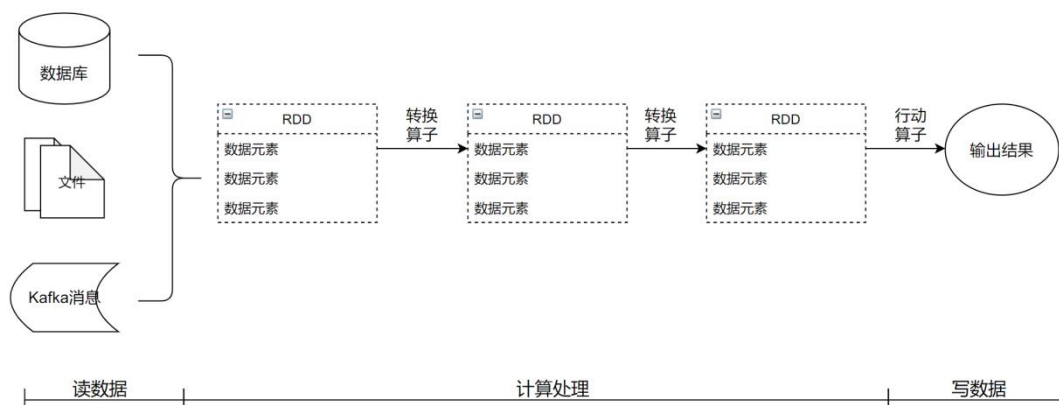
```
        <arg>-dependencyfile</arg>
        <arg>${project.build.directory}/.scala_dependencies</arg>
    </args>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>
```



2.2 spark 编程 api 套路

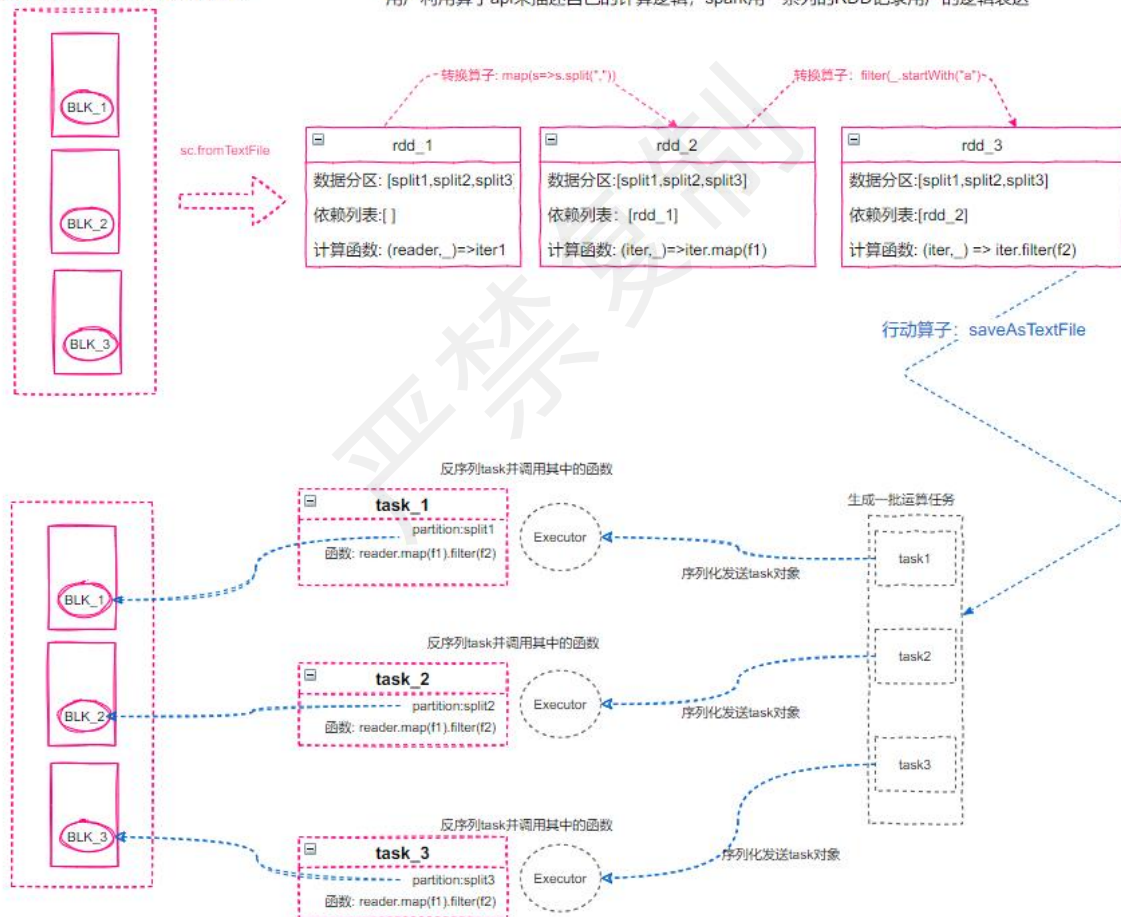
(1) 将待处理数据加载、映射成 RDD 分布式集合

- (2) 在 RDD 上根据需求, 调用一系列 (transformation) 算子, 对数据进行转换, 得到结果 RDD
- (3) 在最后的的结果 RDD 上调用 (action) 算子, 触发计算执行



hdfs://doitedu01:8020/data/a001.txt

用户利用算子api来描述自己的计算逻辑, spark用一系列的RDD记录用户的逻辑表达



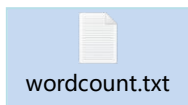
2.3 编程体验: wordcount

从一个本地磁盘的文件中 (或一个本地内存 `List[String]` 集合中), 统计每个单词出现的次数;

数据样例：

Broadcast variables allow the programmer to keep a read-only variable cached on each machine rather than shipping a copy of it with tasks. They can be used, for example, to give every node a copy of a large input dataset in an efficient manner. Spark also attempts to distribute broadcast variables using efficient broadcast algorithms to reduce communication cost.

测试数据文件：



代码开发

```
object WordCount {  
  def main(args: Array[String]): Unit = {  
    val conf = new SparkConf  
    conf.setMaster("local")  
    conf.setAppName("wordCount")  
    // 构造一个用于运行 spark 程序的上下文对象  
    val sc = new SparkContext(conf)  
  
    // 加载文件获得 RDD 数据集  
    val rdd1: RDD[String] = sc.textFile("d:/wc.txt", 2)  
  
    // 对“集合[String]”中的每条数据应用函数：按空格切分成数组，并压平，得到一个新的“集合[String]”  
    val rdd2: RDD[String] = rdd1.flatMap(s => s.split("\\s+"))  
  
    // 对“集合[String]”中的每条数据应用函数：字符串=>元组，得到一个新的“集合[(String,Int)]”  
    val rdd3: RDD[(String, Int)] = rdd2.map(s => (s, 1))  
  
    // 对“集合[(String,Int)]”进行分组聚合运算，分组 key:单词，聚合函数：将 1 累加  
    val rdd4: RDD[(String, Int)] = rdd3.reduceByKey((c1, c2) => c1 + c2)  
  
    // 将统计结果打印  
    rdd4.foreach(println)  
  
    sc.stop()  
  }  
}
```

```
}  
}
```

当然，更符合 `scala` 风格的写法应该如下：

```
object WordCount {  
  def main(args: Array[String]): Unit = {  
    val conf = new SparkConf()  
      .setMaster("local")  
      .setAppName("wordCount")  
  
    // 构造一个用于运行 spark 程序的上下文对象  
    val sc = new SparkContext(conf)  
  
    sc.textFile("d:/wc.txt", 2)  
      .flatMap(_.split("\\s+"))  
      .map((_,1))  
      .reduceByKey(_+_)  
      .foreach(println)  
  
    sc.stop()  
  }  
}
```

3 RDD 创建方式详解

3.1 从文件系统加载文件

spark 可以自动解析多种格式的文件，如 Text File, Json File, Parquet File, OrcFile 等；

此处以加载 Text 文件和 Sequence 文件为例进行示范；

注：其他的文件格式的加载解析，在 spark 的上层库（Spark Sql）有更方便的 API

```
// 从本地文件系统加载（只适用于开发测试）  
val rdd1 = sc.textFile("d:/doitedu.txt",2)  
rdd1.foreach(println)  
  
// 从 HDFS 文件系统加载（对应绝大多数生产应用场景）  
val rdd2 = sc.textFile("hdfs://doitedu01:8020/data/input/",2)
```

```
rdd2.foreach(println)

// 从文件系统加载 Sequence 文件 [Sequence 测试文件见下方]
val rdd4 = sc.sequenceFile("d:/seq", classOf[Int], classOf[String])
rdd4.foreach(println)
```

测试数据: sequece 文件

part-00000

测试数据: txt 文件

data.txt

3.2 从 MySQL 加载数据表

```
val getConn = ()=> DriverManager.getConnection(
    "jdbc:mysql://hdp01:3306/db","root","ABC123abc.123")
val resMapping = (rs:ResultSet)=>{
    val id = rs.getInt(1)
    val name = rs.getString(2)
    val role = rs.getString(3)
    val energy = rs.getInt(4)
    (id,name,role,energy)
}
val rdd: JdbcRDD[(Int, String, String, Int)] = new JdbcRDD[(Int, String, String, Int)](sc,
    getConn,
    "select id,name,role,energy from data where ? <= id and id <= ?",
    1,100,2,
    resMapping)
rdd.foreach(println)
```

测试数据:



3.3 从内存集合映射 RDD

```
// 存在一个本地内存集合
val lst = List(
    (1, "hello spark hello"),
    (2, "hello doitedu spark flink"),
    (3, "spark doitedu spark flink"),
    (4, "hello spark spark flink"),
    (5, "doitedu spark doitedu spark")
)
```

```
// 从本地集合创建 RDD
val rdd3: RDD[(Int, String)] = sc.makeRDD(lst, 1)
```

4 Spark Core 常用 API

4.1 重要特性

spark 中的算子，分为两大类型：

- transformation 算子，如：map / flatMap / filter 等
- action 算子，如：collect / take / saveAsTextFile 等

在 Spark 中几乎所有的 transformation 操作都是**懒执行的(lazy)**，也就是说 transformation 操作并不会立即计算他们的结果，而是记住了这个转换逻辑；

只有当通过一个 action 来获取结果返回给驱动程序的时候这些转换操作才开始计算。这种设计可以使 Spark 运行起来更加的高效。默认情况下，你每次在一个 RDD 上运行一个 action 的时候，前面的每个 transformed RDD 都会被重新计算；

但是我们可以通过 persist (or cache) 方法来持久化一个 RDD 在内存中，也可以持久化到磁盘上，来加快访问速度；

4.2 常用转换算子

Transformation	Meaning
map(func)	对每个元素应用 func 函数
filter(func)	对每个元素应用 func 函数，且只留下 func 返回 true 的元素
flatMap(func)	对每个元素应用 func 函数，并将该 func 返回的 iterable 压平
mapPartitions(func)	对每个 rdd 的分区应用 func 函数
mapPartitionsWithIndex(func)	类似 mapPartitions，不同之处在于 func 可以接收到每个元素所属分区号
sample(withReplacement, fraction, seed)	返回一个随机采样的样本 rdd
union(otherDataset)	求并集
intersection(otherDataset)	求交集
distinct([numPartitions])	去重
groupByKey([numPartitions])	按 key 分组，将 (K, V) RDD 转成: (K, Iterable<V>)RDD Note: 如果分组后是为了做聚合 (如: sum、average)，则使用 reduceByKey 或 aggregateByKey 将更加高效；

<code>reduceByKey(func, [numPartitions])</code>	按 key 进行分组聚合；聚合结果必须与输入数据相同 $v, v \Rightarrow v$
<code>aggregateByKey(zeroValue)(seqOp, combOp, [numPartitions])</code>	按 key 进行分组聚合；聚合结果可以与输入数据类型不同。 $u, v \Rightarrow u$
<code>sortByKey([ascending], [numPartitions])</code>	根据 key 排序
<code>join(otherDataset, [numPartitions])</code>	两个数据集进行 inner join；外连接可以使用 leftOuterJoin, rightOuterJoin, 和 fullOuterJoin
<code>cogroup(otherDataset, [numPartitions])</code>	When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called groupWith.
<code>cartesian(otherDataset)</code>	返回笛卡尔积
<code>pipe(command, [envVars])</code>	Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process's stdin and lines output to its stdout are returned as an RDD of strings.
<code>coalesce(numPartitions)</code>	降低 rdd 的分区数（常用于在一个大的 rdd 被执行 filter 后）
<code>repartition(numPartitions)</code>	以随机的方式来重新 shuffle 数据，以获得更大或更小的分区数
<code>repartitionAndSortWithinPartitions(partitioner)</code>	使用给定的 partitioner 执行 repartition 并进行分区内排序，效率比 repartition 更高，而且 sorting 操作会被下推到 shuffle 的内在过程中

- map 算子，功能：根据传入的函数 f，对每个元素做映射

```
val rdd1: RDD[Int] = sc.parallelize(List(5,6,4,7,3,8,2,9,1,10)).map(_*2)
```

- flatMap 算子，先 map 再压平，spark 中没有 flatten 方法

```
val rdd2 = sc.parallelize(Array("a b c", "d e f", "h i j"))
rdd2.flatMap(_.split(' ')).collect
val rdd3 = sc.parallelize(List(List("a b c", "a b b"), List("e f g", "a f g")))
rdd3.flatMap(_.flatMap(_.split(" "))).collect
```

- filter 算子,功能：根据传入的函数 f，对每个元素做判断以过滤数据

```
val rdd4 = sc.parallelize(List(5,6,4,7,3,8,2,9,1,10)).filter(_ % 2 == 0)
```

- mapPartitions，类似于 map 算子，但执行逻辑不同：mapPartitions 算子，是将一个分区的数据对应的迭代器传入用户的函数（相当于一次性传入一个分区的数据让用户的函数来处理）。

```
val rdd1 = sc.parallelize(List(1, 2, 3, 4, 5), 2)
var r1: RDD[Int] = rdd1.mapPartitions(it => it.map(x => x * 10))
```

- mapPartitionsWithIndex，类似于 mapPartitions，不过函数要输入两个参数，第一个参数为分区的索引，第二个是对应分区的迭代器。函数的返回的是一个经过该函数转换的迭代器。

```
val rdd1 = sc.parallelize(List(1,2,3,4,5,6,7,8,9), 2)
rdd1.mapPartitionsWithIndex((index, it) => {
  it.map(e => s"partition: $index, val: $e")
}).collect
```

- sortBy 算子，排序

```
sc.parallelize(List(5,11,22,13,2,1,10)).sortBy(x=>x,true)
sc.parallelize(List(5,11,22,13,2,1,10)).sortBy(x=>x+"",true)
sc.parallelize(List(5,11,22,13,2,1,10)).sortBy(x=>x.toString,true)
```

- sortByKey 算子，排序

```
val rdd1 = sc.parallelize(List(("hello", 9), ("tom", 8), ("kitty", 7), ("tom", 2)))
val rdd2 = rdd1.sortByKey()
```

- groupBy 算子，安装 key 分组

```
val rdd1 = sc.parallelize(List(("hello", 9), ("tom", 8), ("kitty", 7), ("tom", 2)))
val rdd2 = rdd1.groupBy(_._1)
```

- groupByKey 按照 key 进行分组

```
val rdd2 = sc.parallelize(List(("jerry", 9), ("tom", 8), ("shuke", 7), ("tom", 2)))
val rdd2: RDD[(String, Iterable[Int])] = rdd1.groupByKey()
```

- reduceByKey

```
val rdd2 = sc.parallelize(List(("jerry", 9), ("tom", 8), ("shuke", 7), ("tom", 2)))
val rdd3 = rdd1.reduceByKey(_+_)
```

- distinct 算子，去重

```
sc.parallelize(List(5,5,6,6,7,8,8,8)).distinct.collect
```

- union 算子，并集，两个 RDD 类型要一样

```
val rdd6 = sc.parallelize(List(5,6,4,7), 2)
val rdd7 = sc.parallelize(List(1,2,3,4), 3)
val rdd8 = rdd6.union(rdd7)
```

- cogroup 算子, 协分组, 有点跟 fullOuterJoin 类似, 但是没有关联上的返回 CompactBuffer()

```
def cogroup[W](other: RDD[(K, W)], partitioner: Partitioner)
  : RDD[(K, (Iterable[V], Iterable[W]))] = self.withScope {
  if (partitioner.isInstanceOf[HashPartitioner] && keyClass.isArray) {
    throw new SparkException("HashPartitioner cannot partition array keys.")
  }
  val cg = new CoGroupedRDD[K](Seq(self, other), partitioner)
  cg.mapValues { case Array(vs, w1s) =>
    (vs.asInstanceOf[Iterable[V]], w1s.asInstanceOf[Iterable[W]])
  }
}
```

```
val rdd1 = sc.parallelize(List(("tom", 1), ("tom", 2), ("jerry", 3), ("kitty", 2)))
val rdd2 = sc.parallelize(List(("jerry", 2), ("tom", 1), ("shuke", 2)))
val rdd3 = rdd1.cogroup(rdd2)
val rdd4 = rdd3.map(t=>(t._1, t._2._1.sum + t._2._2.sum))
```

- intersection 算子, 交集

```
val rdd6 = sc.parallelize(List(5,6,4,7))
val rdd7 = sc.parallelize(List(1,2,3,4))
val rdd9 = rdd6.intersection(rdd7)
```

- subtract 算子, 差集

```
val rdd6 = sc.parallelize(List(5,6,4,7))
val rdd7 = sc.parallelize(List(1,2,3,4))
val rdd9 = rdd6.subtract(rdd7)
```

- join, 相当于 SQL 中的内关联 join

```
val rdd1 = sc.parallelize(List(("tom", 1), ("jerry", 2), ("kitty", 3)))
val rdd2 = sc.parallelize(List(("jerry", 9), ("tom", 8), ("shuke", 7), ("tom", 2)))
val rdd3 = rdd1.join(rdd2)
```

- leftOuterJoin, 相当于 SQL 中的左外关联,

```
val rdd1 = sc.parallelize(List(("tom", 1), ("jerry", 2), ("kitty", 3)))
val rdd2 = sc.parallelize(List(("jerry", 9), ("tom", 8), ("shuke", 7), ("tom", 2)))
val rdd3 = rdd1.leftOuterJoin(rdd2)
```

- **rightOuterJoin**, 相当于 SQL 中的右外关联

```
val rdd1 = sc.parallelize(List(("tom", 1), ("jerry", 2), ("kitty", 3)))
val rdd2 = sc.parallelize(List(("jerry", 9), ("tom", 8), ("shuke", 7), ("tom", 2)))
val rdd3 = rdd1.rightOuterJoin(rdd2)
```

- **fullOuterJoin**, 相当于 SQL 中的全关联

```
val rdd1 = sc.parallelize(List(("tom", 1), ("jerry", 2), ("kitty", 3)))
val rdd2 = sc.parallelize(List(("jerry", 9), ("tom", 8), ("shuke", 7), ("tom", 2)))
val rdd3 = rdd1.fullOuterJoin(rdd2)
```

- **cartesian** 算子, 笛卡尔积

```
val rdd1 = sc.parallelize(List("tom", "jerry"))
val rdd2 = sc.parallelize(List("tom", "kitty", "shuke"))
val rdd3 = rdd1.cartesian(rdd2)
```

- **aggregateByKey**, 按照 key 进行聚合, 跟 **reduceByKey** 类似, 可以输入两个函数, 第一个函数局部聚合, 第二个函数全局聚合。初始值只在局部聚合时使用, 全局聚合不使用

```
def aggregateByKey[U: ClassTag](zeroValue: U)(seqOp: (U, V) => U,
  combOp: (U, U) => U): RDD[(K, U)] = self.withScope {
  aggregateByKey(zeroValue, defaultPartitioner(self))(seqOp, combOp)
}
```

```
val pairRDD = sc.parallelize(List(("cat", 2), ("cat", 5), ("mouse", 4),
  ("cat", 12), ("dog", 12), ("mouse", 2)), 2)
pairRDD.aggregateByKey(0)(math.max(_, _), _ + _).collect
pairRDD.aggregateByKey(100)(math.max(_, _), _ + _).collect
```

- **combineByKey**, 需要输入 3 个参数, 第 1 个参数为分组后 value 的第一个元素, 第 2 个函数为局部聚合函数, 第 3 个函数为全局聚合函数

reduceByKey、**foldByKey**、**aggregateByKey**、**combineByKey** 底层调用的都是 **combineByKeyWithClassTag**

```
def combineByKey[C](
  createCombiner: V => C,
  mergeValue: (C, V) => C,
  mergeCombiners: (C, C) => C): RDD[(K, C)] = self.withScope {
  combineByKeyWithClassTag(createCombiner, mergeValue, mergeCombiners)(ct = null)
}
```

```
val pairRDD = sc.parallelize(List(("cat",2), ("cat", 5), ("mouse", 4),
  ("cat", 12), ("dog", 12), ("mouse", 2)), 2)
pairRDD.combineByKey(x => x, (a: Int, b: Int) => a + b, (m: Int, n: Int) => m + n)

val a = sc.parallelize(List("dog", "cat", "gnu", "salmon",
  "rabbit", "turkey", "wolf", "bear", "bee"), 3)
val b = sc.parallelize(List(1,1,2,2,2,1,2,2,2), 3)
val c = b.zip(a)
val d = c.combineByKey(List(_), (x:List[String], y:String) => y :: x,
  (x:List[String], y:List[String]) => x ::: y)
d.collect
```

4.3 常用行动算子

Action	Meaning
<code>reduce(func)</code>	将 rdd 中的所有数据聚合成一个值; func 函数需要满足交换律和结合律以便能并行计算
<code>collect()</code>	从 rdd 中收集数据到 driver 本地
<code>count()</code>	求 rdd 的元素个数
<code>first()</code>	取 rdd 中的第一条数据
<code>take(n)</code>	取 rdd 中的 n 条数据
<code>takeSample(withReplacement, num, [seed])</code>	随机抽样, withReplacement: 相同元素是否可以被重复抽取
<code>takeOrdered(n, [ordering])</code>	按排序规则取前 n 条
<code>saveAsTextFile(path)</code>	把每个元素 toString 后写入文本文件
<code>saveAsSequenceFile(path)</code> (Java and Scala)	将 [KV] 数据写入 sequence 文件
<code>saveAsObjectFile(path)</code> (Java and Scala)	将元素用对象序列化后的字节, 封装到 BytesWritable 中, 写入 sequence 文件
<code>countByKey()</code>	计算每个 key 的元素个数, 返回 hashmap[k, count]
<code>foreach(func)</code>	Run a function func on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with

external storage systems.

Note: modifying variables other than Accumulators outside of the `foreach()` may result in undefined behavior. See [Understanding closures](#) for more details.

4.4 并行度调整算子

4.4.1 coalesce

```
val conf: SparkConf = new SparkConf()
    .setMaster("local[*]")
    .setAppName("分区测试")
    .set("spark.default.parallelism", "500")
val sc = new SparkContext(conf)

val rdd1 = sc.makeRDD((1 to 10000).zip(10000 to 20000), 100)
println(rdd1.partitions.size)

// coalesce
// 修改并行度：可以改大，也可以改小； 取决于算子中的参数： shuffle
// 如果 shuffle==true, 则该算子会生成一个 ShuffledRDD, 产生了 shuffle (分区数可改大, 可改小)
// 如果 shuffle==false, 则概算自不会产生 shuffle, 但是分区数只能该小; (改大不生效)
val rdd4 = rdd1.coalesce(20, true) // 20
val rdd5 = rdd1.coalesce(200, false) // 100
val rdd6 = rdd1.coalesce(200, true) // 200
println(rdd4.partitions.size)
println(rdd5.partitions.size)
println(rdd6.partitions.size)
```

4.4.2 repartition

```
// repartition(200)
// 调用的就是 coalesce(shuffle=true, 200)
// 所以, repartition 可以把并行度改大也可以改小, 而且一定会产生 shuffle
val rdd2: RDD[(Int, Int)] = rdd1.repartition(10) // 10
val rdd3: RDD[(Int, Int)] = rdd1.repartition(200) // 200
```

```
println(rdd2.partitions.size)
println(rdd3.partitions.size)
```

4.4.3 partitionBy

用于根据自己给定的规则对 RDD 进行重新分区

```
// partitionBy
// 允许开发人员传入一个 Partitioner 来对上游数据进行重新分区
// 自己传入的 Partitioner 就可以控制根据 Key 中的哪个成分来分区
val rdd_o = sc.makeRDD(Seq(
  (Order(1, "柳坤 1", 200, "团购订单"),1),
  (Order(2, "柳坤 2", 300, "普通订单"),2),
  (Order(3, "柳坤 3", 400, "团购订单"),3),
  (Order(4, "柳坤 4", 500, "普通订单"),4),
  (Order(5, "柳坤 5", 600, "普通订单"),5)
), 4)

// 根据订单的订单类型来分区
val rdd_v: RDD[(Order, Int)] = rdd_o.partitionBy(new OrderPartitioner(2))
println(rdd_v.partitioner)
println(rdd_v.partitions.size)
```

4.5 闭包引用

4.5.1 闭包引用的原理

如果在 rdd 算子的函数中，引用了外部的对象（称为闭包引用），则这个对象会随着 task 而被序列

化，且被发送到各个 `executor` 上被反序列化；

反序列化出来的 `task` 中的闭包对象（“副本”），已经是完全独立的了；修改 `task` 中的对象，不会影响到 `driver` 端的原对象；

```
var count: Int = 0 // 此处定义的 count 对象，在 Driver 端的 jvm 中

val rdd1 = rdd.map( tp => {
    count += 1 // 此处的 count 是在 executor 端反序列化出的 Task 中，与 driver 端已无联系
    (tp._1.toUpperCase, tp._2 * 10)
} ).collect()

println(count) // 此处打印的是 driver 端的 count，依然是 0
```

如下 `hashMap` 对象也如此：算子中对 `hashmap` 对象的修改，不会影响到 `driver` 端的 `hashmap` 对象

```
val hashMap = new mutable.HashMap[String, Int]() // 此处定义的 hashmap 对象，在 Driver 端的 jvm 中

rdd.map( tp=>{
    hashMap += tp // 此处的 hashMap 是在 executor 端反序列化出的 Task 中，与 driver 端已无联系
    val id: Int = TaskContext.getPartitionId()
    println( id + ": " + hashMap)
} ).count()
println(hashMap)
```

4.5.2 闭包引用的应用场景

如果需要让每个 `task` 都用有一份“只读”的“小量”数据，比如一个字典，则可以利用闭包引用；

如果共享的这分只读数据比较大，则应该使用“广播变量”效率更高！

4.5.3 闭包引用的注意事项

序列化检查

闭包引用的对象，必须实现序列化接口，否则会导致 `task` 序列化失败，从而快速报错

```
class Per extends Serializable {
    val id: Int = 0
}
```

```
val p = new Per()
val rd = sc.makeRDD(1 to 10, 2)
// 闭包引用的对象，必须实现序列化接口
rd.map( (_,p) ).foreach(println)
```

“副本”数量

如果闭包引用的是普通对象，则每个 task 中都有一份“copy”

如果闭包引用的是一个 object 对象（单例对象），则其实在整个 executor 中只有一份，如下

```
object Per extends Serializable {
    val id: Int = 0
}
val p = Per
val rd = sc.makeRDD(1 to 10, 2)
rd.map( (_,p) ).foreach(println)
```

如果使用闭包引用 object 对象，有可能产生线程安全问题（因为有多 task 线程共享这个对象）；

看似闭包，实非闭包

下面的代码，其实并不是“闭包引用”

当然，里面用到了自定义类型，也还是要注意序列化问题

```
class Phone(var brand: String, var price: Double)
val resRdd = rdd.map(tp => {
    new Phone(tp._1, tp._2) // 这里并没有引用外部的对象，所以不存在 f 序列化检查失败的问题 })
//.map(phone => (phone.brand, phone.price))
//.reduceByKey(_ + _) // 这里有 shuffle，但 shuffle 写出的是 2 元组，它能序列化，所以不会报错
.groupBy(p=>p.brand) // 这里有 shuffle，而且 shuffle 写出 phone 对象，它不能序列化，所以报错
.mapValues(_.size)
```

4.6 广播变量

4.6.1 简介

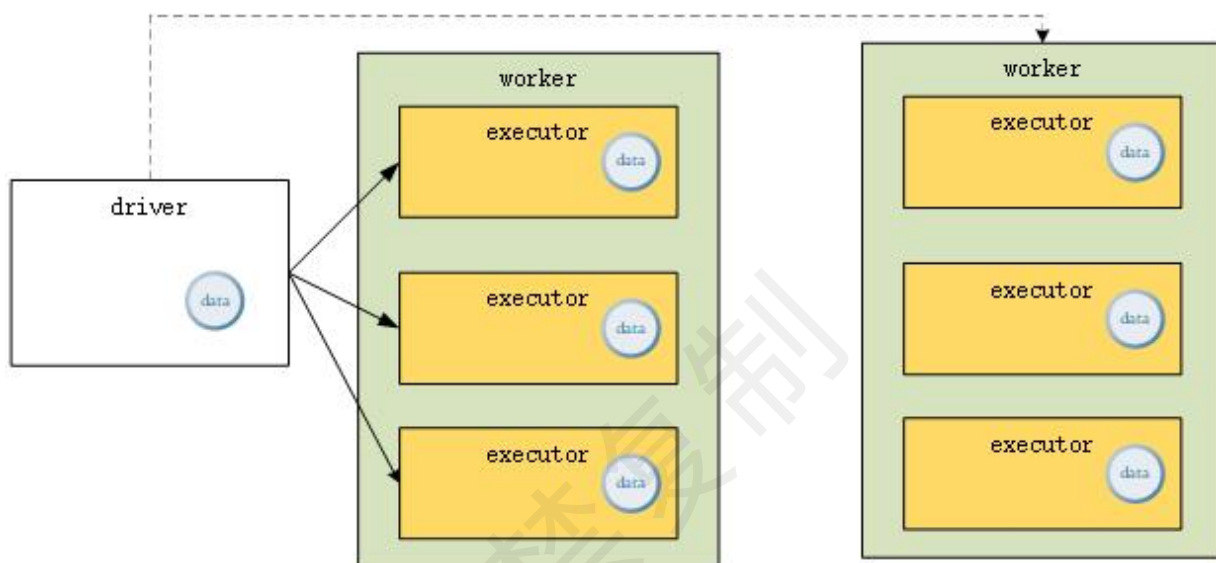
广播变量可以让我们在每个 executor 上保留一个只读变量，而不是为每个任务复制一份副本。就是说，在以下两种情况下显式的创建广播变量才有用：

- 1) 当任务跨多个 stage 并且需要同样的数据时;
- 2) 当需要以反序列化的形式来缓存数据时。

如果你的应用需要向所有节点发送一个较大的只读查询表, 广播变量非常适合。

4.6.2 特点

- 1) 广播变量会在每个 executor 中保留一份副本, 而不是为每个 Task 保留一份副本。



- 2) Spark 使用了 BitTorrent 协议进行广播变量的分发, 这样可以减少通信成本, 该分发算法在后面章节有介绍。

4.6.3 广播变量 API

不适用广播变量的问题

```
def main(args: Array[String]): Unit = {  
    val sc: SparkContext = SparkUtil.getSc  
    val rdd1: RDD[(String, Int)] = sc.parallelize(List(("a", 1), ("b", 2), ("c", 3)))  
    val rdd2: RDD[(String, Int)] = sc.parallelize(List(("a", 11), ("b", 22), ("c", 33)))  
    // 数据在进行 join 的时候会出现 shuffle , 我们在编程的时候尽量避免 shuffle 的产生  
    val resRDD1: RDD[(String, (Int, Int))] = rdd1.join(rdd2)  
  
    // 使用这样的方式来实现 join 逻辑, 这里调用的是 map 函数是没有 shuffle 产生的  
    // 1 将数据收集到 driver 端
```

```
val mp: Map[String, Int] = rdd2.collect().toMap
// map 函数的逻辑分布式执行在不同的 Task 中，那么我们的 mp 数据在每个 Task 中都存在
val resRDD2: RDD[(String, (Int, Int))] = rdd1.map {
  case (k, v) => {
    val i: Int = mp.getOrElse(k, -1)
    (k, (v, i))
  }
}
resRDD2.foreach(println)
sc.stop()
```

使用广播变量

```
def main(args: Array[String]): Unit = {
  val sc: SparkContext = SparkUtil.getSc
  val rdd1: RDD[(String, Int)] = sc.parallelize(List(("a", 1), ("b", 2), ("c", 3)))
  val rdd2: RDD[(String, Int)] = sc.parallelize(List(("a", 11), ("b", 22), ("c", 33)))
  val mp: Map[String, Int] = rdd2.collect().toMap
  // 广播数据
  val bc: Broadcast[Map[String, Int]] = sc.broadcast(mp)
  val resRDD: RDD[(String, (Int, Int))] = rdd1.map {
    case (k, v) => {
      // 获取广播数据
      val i: Int = bc.value.getOrElse(k, -1)
      (k, (v, i))
    }
  }
  resRDD.foreach(println)
  sc.stop()
}
```

销毁广播变量

```
// 销毁广播变量
bc.unpersist()
```

从 driver 端发送一个 RemoveBroadcast 消息。在 Executor 上的 BlockManager 服务接收该消息，就会把广播变量从 BlockManager 中删除。若 removeFromDriver 设置成 True，还会从 Driver 删除该变量的数据。

4.6.4 注意事项

1) 能不能将一个 RDD 使用广播变量广播出去？

不能，因为 RDD 是不存储数据的。可以将 RDD 的计算结果广播出去。

2) 广播变量只能在 Driver 端定义，不能在 Executor 端定义。

3) 在 Driver 端可以修改广播变量的值，在 Executor 端无法修改广播变量的值。

4.6.5 应用：map 端 join

订单和用户 join

join 算子可以实现 join 关联，但是一般情况下都会产生 shuffle，效率不高

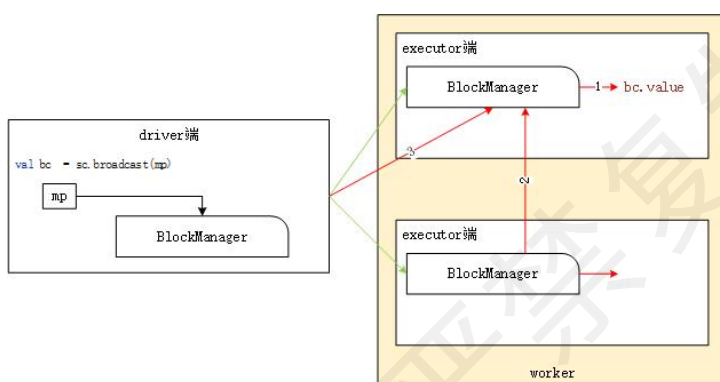
```
object _02Join {  
  def main(args: Array[String]): Unit = {  
    val sc: SparkContext = SparkUtils.getSparkContext()  
    // 读取用户数据  
    // u001,benge,18,male,angelababy  
    val userDataRDD: RDD[String] = sc.textFile("data/order/user.txt")  
    val userTupleRDD = userDataRDD.map(line=>{  
      val arr: Array[String] = line.split(",")  
      (arr(0),(arr(1),arr(2),arr(3),arr(4)))  
    })  
    // 读取订单数据  
    //order033 u005  
    val orderDataRDD: RDD[String] = sc.textFile("data/order/order.txt")  
    val orderTupleRDD = orderDataRDD.map(line=>{  
      val arr: Array[String] = line.split("\\s+")  
      (arr(1),arr(0))  
    })  
    // 方式2 广播变量 封装 List 不合适  
    val mp: Map[String, (String, String, String, String)] = userTupleRDD.collect().toMap  
    val bc: Broadcast[Map[String, (String, String, String, String)]] = sc.broadcast(mp)  
    // val ls: List[(String, String, String, String, String)] = userTupleRDD.collect().toList  
    // val bc: Broadcast[List[(String, String, String, String, String)]] = sc.broadcast(ls)  
    val res: RDD[String] = orderTupleRDD.map(order => {  
      // 从广播变量中获取值  
      val mp: Map[String, (String, String, String, String)] = bc.value  
      val uid: String = order._1
```

```
val oid: String = order._2
// 根据 uid 从集合中获取用户信息 拼接
val tuple: (String, String, String, String) = mp.getOrElse(uid, ("null", "null", "null", "null"))
oid + "," + uid + "," + tuple._1 + "," + tuple._2 + "," + tuple._3 + "," + tuple._4
})
res.foreach(println)

sc.stop()
}
}
```

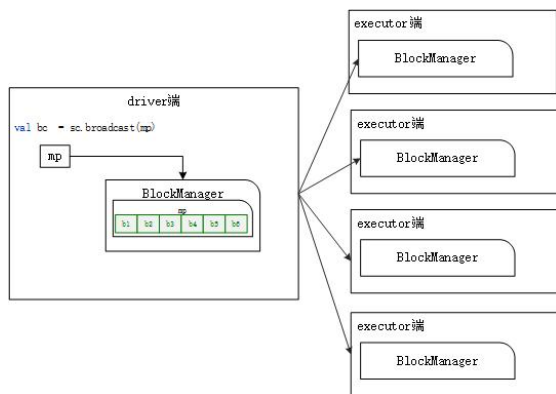
4.6.6 广播变量分发读取机制

4.6.6.1 创建原理



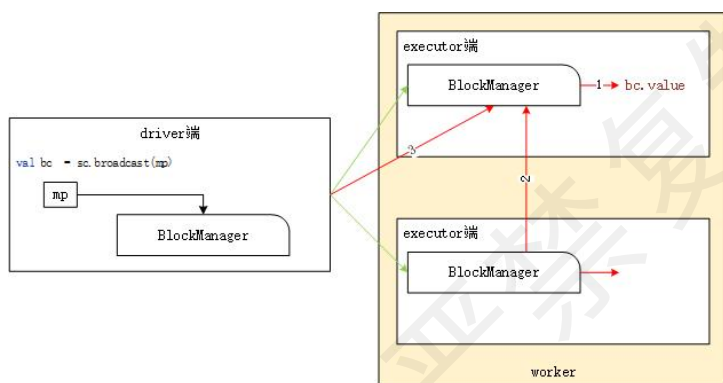
广播变量的创建发生在 Driver 端，如图所示，当调用 `SparkContext#broadcast` 来创建广播变量时，会把该变量的数据切分成多个数据块，保存到 driver 端的 `BlockManger` 中，使用的存储级别是：`MEMORY_AND_DISK_SER`。

所以，广播变量的读取也是懒加载的，只有在 Executor 端需要获取广播变量时才会去获取。此时广播变量的数据只在 Driver 端存在。



广播变量被保存在本地，并会把广播变量的值切分成多个数据块进行保存。广播变量数据块的默认大小是 4M，数据块太大或太小都不利于数据的传输。

4.6.6.2 读取原理



- 1) 第 1 步（红色线 1）：**首先从本 Executor 的 BlockManager 中（或本机的其他 executor）读取广播变量的数据，若存在就直接获取，并返回。若不存在，则执行 2 或 3。
- 2) 第 2 步：**从远端获取数据。先从同一个机架(rack)的主机的 Executor 端获取。若不能从其他 Executor 中获取广播变量，则会直接从 Driver 端获取。
- 3) 第 3 步（红色线 3）：**从 Driver 端获取广播变量的状态和位置信息（由于所有的 BlockManager slave 端都会向 Master 端汇报数据块状态）。

从以上获取流程可以看出，在执行 spark 应用时，只要有一个 worker 节点的 Executor 从 Driver 端获取到了广播变量的数据，则其他的 Executor 就不需要从 Driver 端获取了。

BT 协议

当某个 Executor 上的某个数据块被删除，可以从其他 Executor 直接获取该数据块，然后把数据块

保存到自己的 Executor 的 BlockManager 中。

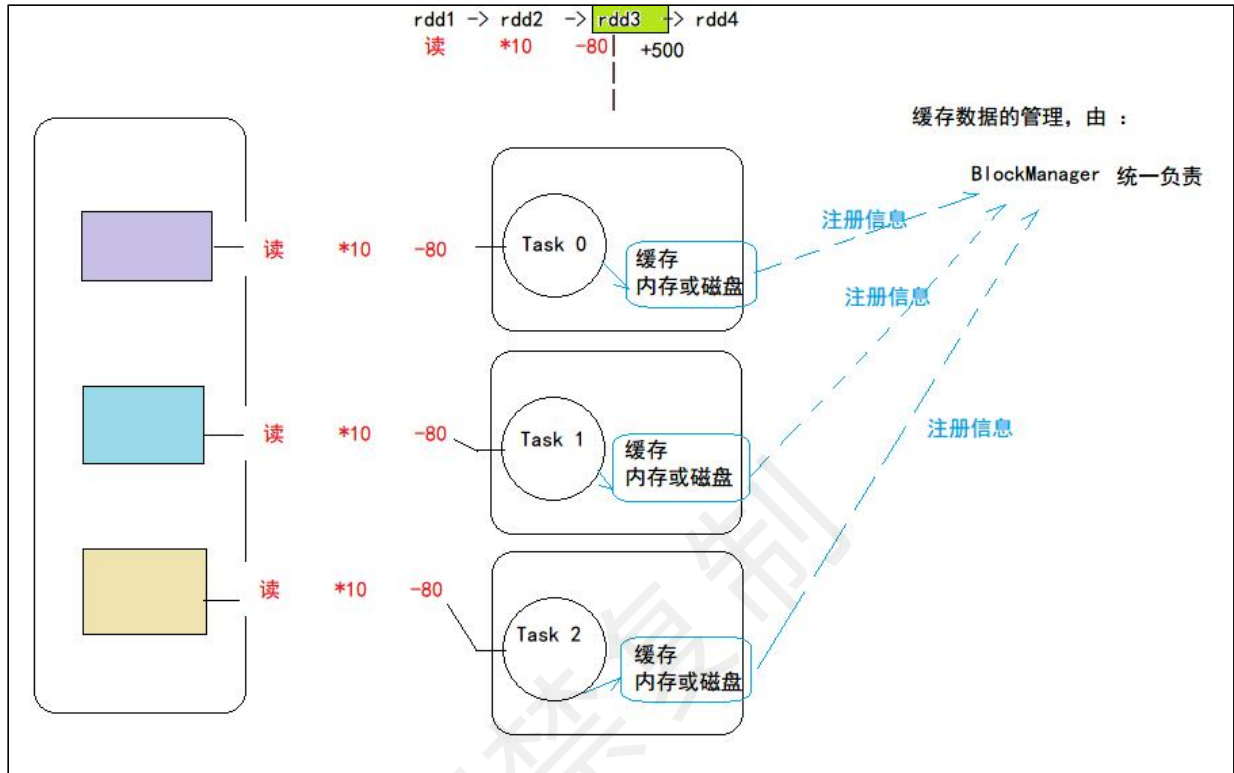


Executor4 中的任务需要使用广播变量，但它只有该变量的 b4 数据块。此时，它首先从同主机（worker2 节点）的中获取数据，获取到数据块；然后分别从不同主机的 Executor1 和 Executor2 中读取数据块。此时，Executor4 就获取到变量 b 的全部数据块了，然后把这些数据块在自己的 BlockManager 中保存一份。此时，其他 Executor 就可以从 Executor4 中读取数据了。

当完成这些操作后，各个 Executor 端的 BlockManager(slave 端)会向 Driver 端的 BlockManager(master 端)汇报数据块的状态。

4.7 操作 rdd 缓存

4.7.1 RDD 缓存的概念理解



RDD 本身是没有数据的，那么缓存 RDD，其实是缓存迭代器链条计算到本 RDD 时的结果数据。所以，缓存 RDD，就是有 RDD 的运算 task 得到计算结果后进行缓存，以便于其他的 job 可以反复重用该 RDD 的“数据”而不需要反复计算该 RDD。

4.7.2 RDD 缓存 API

4.7.2.1 persist

```
rdd2.persist(StorageLevel.MEMORY_AND_DISK) // 可以自己控制存储级别

// NONE 相当于没有存储
// DISK_ONLY 缓存到磁盘
// DISK_ONLY_2 缓存到磁盘，2 个副本
// MEMORY_ONLY 缓存到内存
// MEMORY_ONLY_2 缓存到内存，2 个副本
// MEMORY_ONLY_SER 缓存到内存，以序列化格式
// MEMORY_ONLY_SER_2 缓存到内存，以序列化格式，2 个副本
// MEMORY_AND_DISK 缓存到内存和磁盘
// MEMORY_AND_DISK_2 缓存到内存和磁盘，2 个副本
// MEMORY_AND_DISK_SER 缓存到内存和磁盘，以序列化格式
// MEMORY_AND_DISK_SER_2 缓存到内存和磁盘，以序列化格式，2 个副本
// OFF_HEAP 缓存到堆外内存
```

4.7.2.2 unpersist

如果需要清除已经缓存的 RDD 数据，可以调用 `rdd.unpersist()` 方法：

4.7.2.3 cache

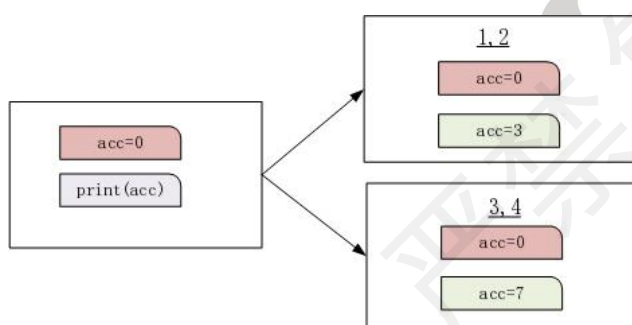
`cache()` 就是调用的 `persist(StorageLevel.MEMORY_ONLY)`

4.8 累加器

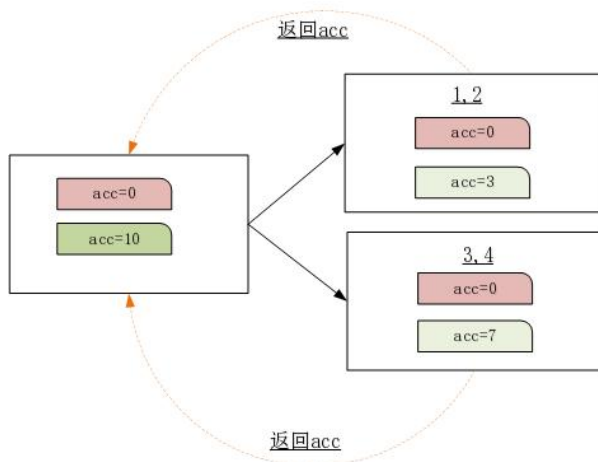
累加器用来把 **Executor** 端变量信息聚合到 **Driver** 端。在 **Driver** 程序中定义的变量，在 **Executor** 端的每个 **Task** 都会得到这个变量的一份新的副本，每个 **task** 更新这些副本的值后，传回 **Driver** 端进行 **merge**。

观察一个问题：原因是数据在 **executor** 端执行完毕以后并没有将 **acc** 结果数据返回

```
def main(args: Array[String]): Unit = {  
    val sc: SparkContext = SparkUtil.getSc  
    val rdd: RDD[Int] = sc.makeRDD(List(1, 2, 3, 4), 2)  
    var count: Long = 0L  
    // rdd.map(count += _)  
    rdd.foreach(num => { count += num })  
    // 计算的结果为 0  
    println(count)  
    sc.stop()  
}
```



解决方案：应该将每个 **executor** 执行的结果数据返回到 **Driver** 端进行聚合操作，返回最终结果数据



使用累加器计算结果

```
def main(args: Array[String]): Unit = {
    val sc: SparkContext = SparkUtil.getSc
    val rdd: RDD[Int] = sc.makeRDD(List(1, 2, 3, 4), 2)
    // var count: Long = 0L
    // 创建累加器
    val cnt: LongAccumulator = sc.longAccumulator("cnt")
    // rdd.map(count += _)
    // 累加数据
    rdd.foreach(num => { cnt.add(num) })
    // 计算的结果为
    println(cnt.value) // 10
    sc.stop()
}
```

上面使用累加器计算出我们要统计的结果数据，但是注意下面的操作

```
def main(args: Array[String]): Unit = {
    val sc: SparkContext = SparkUtil.getSc
    val rdd: RDD[Int] = sc.makeRDD(List(1, 2, 3, 4), 2)
    // 创建累加器
    val cnt: LongAccumulator = sc.longAccumulator("cnt")

    // 累加器计算结果
    /**
     * 注意:
     * 1 累加器数据属于全局变量，由行动算子触发执行，没有触发不执行累加 没算
     * 2 如果多次触发行动算子，累加器会执行多次
     * 3 建议将累加器的变化操作编写在行动算子中
     */
    val rdd2 = rdd.map(num => {
```

```
    cnt.add(num)
  })
  //计算的结果为
  // 执行行动算子 ,触发累加器计数
  rdd2.collect() // 触发累加
  rdd2.collect() // 触发累加
  println(cnt.value)
  sc.stop()
}
```

4.9 自定义累加器

4.9.1 案例需求背景

业务上需要对如下数据进行统计：比如统计每个 city 的用户数

```
"1,Mr.duan,18,beijing"
"2,Mr.zhao,28,beijing"
"b,Mr.liu,24,shanghai"
"4,Mr.nai,22,shanghai"
"a,Mr.liu,24,shanghai"
"6,Mr.ma"
```

同时，还想在业务统计的过程中，附带统计出原始数据中的脏数据条数，并按多种不正确的格式进行分别统计，如：id 字段无法数字化的条数，字段数量不够的条数，其他不正确的条数

4.9.2 自定义累加器

```
/**
 * @author 涛哥
 * @nick_name "deep as the sea"
 * @contact qq:657270652 wx:doit_edu
 * @site www.doitedu.cn
 * @date 2021-11-25
 * @desc IN 泛型，指的是在累加器上调 add() 方法所传入的参数类型
 *      OUT 泛型，指的是在累加器上调 value 所返回的数据的类型
 *      注意：累加器中用于存储值的数据结构，必须是线程安全的；
 */
```

```
*      因为累加器对象在一个 executor 中是被多个 task 线程共享;
*      本例中的 HashMap 其实是不合适的!!!!
*      应该改成 java.util.concurrent.ConcurrentHashMap
*/

class CustomAccumulator(valueMap: scala.collection.mutable.HashMap[String, Long]) extends
AccumulatorV2[(String, Int), scala.collection.mutable.HashMap[String, Long]] {

    // 判断累加器是否为初始状态
    override def isZero: Boolean = valueMap.isEmpty

    // 拷贝一个累加器对象的方法
    override def copy(): AccumulatorV2[(String, Int), scala.collection.mutable.HashMap[String, Long]]
= {

        val newValueMap = new mutable.HashMap[String, Long]()

        this.valueMap.foreach(kv=>newValueMap.put(kv._1,kv._2))

        new CustomAccumulator(newValueMap)
    }

    // 将累加器重置的方法
    override def reset(): Unit = this.valueMap.clear()

    // 对累加器更新值的方法
    override def add(tp: (String, Int)): Unit = {
        val key: String = tp._1
        val oldValue: Long = this.valueMap.getOrElse(tp._1, 0)

        this.valueMap.put(key, oldValue+tp._2)
    }

    // driver 端对各个 task 所产生的的累加器进行结果合并的逻辑
    override def merge(other: AccumulatorV2[(String, Int), mutable.HashMap[String, Long]]): Unit =
    {
        val otherValueMap: mutable.HashMap[String, Long] = other.value
        otherValueMap.foreach(kv => {
            val key: String = kv._1
            val selfValue: Long = this.valueMap.getOrElse(kv._1, 0)
            val otherValue: Long = otherValueMap(kv._1)

            this.valueMap.put(key, selfValue+otherValue )
        })
    }
}
```

```
}

// 从累加器上取值
override def value: scala.collection.mutable.HashMap[String, Long] = this.valueMap
}
```

4.9.3 业务代码中使用累加器

```
/**
 * @author 涛哥
 * @nick_name "deep as the sea"
 * @contact qq:657270652 wx:doit_edu
 * @site www.doitedu.cn
 * @date 2021-11-25
 * @desc 累加器，主要用于正常业务作业过程中的一些附属信息统计
 *       （比如程序遇到的脏数据条数，
 *       程序处理的数据总行数，
 *       程序处理的特定数据条数，
 *       程序处理所花费的时长）
 */
object E05_RDD 开发 API 边缘_累加器自定义 {
  def main(args: Array[String]): Unit = {
    val sc: SparkContext = SparkContextUtil.getSc("累加器")

    val id_malformed: LongAccumulator = sc.longAccumulator("id_malformed")
    val fields_notenough: LongAccumulator = sc.longAccumulator("fields_notenough")

    // 构造一个自定义的累加器
    val accumulator = new CustomAccumulator(new mutable.HashMap[String, Long]())
    // 将自定义累加器向 sparkContext 注册
    sc.register(accumulator)

    val info: RDD[String] = sc.parallelize(Seq(
      "1,Mr.duan,18,beijing",
      "2,Mr.zhao,28,beijing",
      "b,Mr.liu,24,shanghai",
      "4,Mr.nai,22,shanghai",
      "a,Mr.liu,24,shanghai",
      "6,Mr.ma")
    )
  }
}
```

```
val tuples = info.map(line => {
    var res: (Int, String, Int, String) = null
    try {
        val arr: Array[String] = line.split(",")
        val id = arr(0).toInt
        val name = arr(1)
        val age = arr(2).toInt
        val city = arr(3)
        res = (id, name, age, city)
    } catch {
        case e: ArrayIndexOutOfBoundsException => {
            fields_notenough.add(1)
            accumulator.add(("indexoutofBounds", 1))
        }
        case e: NumberFormatException => {
            id_malformed.add(1)
            accumulator.add(("numberFormatException", 1))
        }
        case _ => accumulator.add(("other", 1))
    }
    res
})

val res = tuples.filter(tp => tp != null)
    .groupBy(tp => tp._4)
    .map(tp => {
        (tp._1, tp._2.size)
    })

res.foreach(println)

// 查看累加器的数值
println(id_malformed.value)
println(fields_notenough.value)
println(accumulator.value)

sc.stop()
}
```

5 Spark 编程综合练习

“多易人才 app”产生了大量用户行为日志，文件存储在 hdfs 中：

```
/logdata/app_log_2021-06-05.log  
/logdata/app_log_2021-06-06.log  
/logdata/app_log_2021-06-07.log  
/logdata/app_log_2021-06-08.log
```

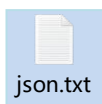
文件内容是 json 格式，如下所示

```
{"account": "WP8p", "appId": "cn.doitedu.app1", "appVersion": "3.2", "carrier": "中国联通",  
"deviceId": "TIIrjOf8UV75", "deviceType": "MI-7", "eventId": "search", "ip": "51.70.233.173", "latitude": 25.38  
572072852434, "longitude": 115.1716608614312, "netType": "WIFI", "osName": "android", "osVersion": "7.2", "proper  
ties": {"keywords": "AJbK SUcb"}, "releaseChannel": "车载安卓市场",  
"resolution": "1024*768", "sessionId": "LOxXhviacU2", "timeStamp": 1628240904060}
```

JUST DO IT
多易教育

5.1 各省份 uv 统计

从 HDFS（或本地磁盘）文件中，统计出每个省份的去重访客数（UV）



数据样例：

```
{"guid": "00762", "province": "贵州省", "url": "http://www.doitedu.cn/job/lmu.html", "device": "iphone8"}  
{"guid": "00927", "province": "辽宁省", "url": "http://www.doitedu.cn/job/ums.html", "device": "iphone10"}  
{"guid": "00251", "province": "北京", "url": "http://www.doitedu.cn/job/ggx.html", "device": "mi8"}  
{"guid": "00527", "province": "云南省", "url": "http://www.doitedu.cn/job/pwp.html", "device": "zte10"}  
{"guid": "00951", "province": "福建省", "url": "http://www.doitedu.cn/job/dvb.html", "device": "iphone8"}  
{"guid": "00671", "province": "广西省", "url": "http://www.doitedu.cn/job/kur.html", "device": "mi8"}  
{"guid": "00843", "province": "山东省", "url": "http://www.doitedu.cn/job/ypy.html", "device": "mate10"}  
{"guid": "00895", "province": "北京", "url": "http://www.doitedu.cn/job/hao.html", "device": "mi6"}
```

5.2 地理位置集成

处理需求中的日志数据：根据日志中的 `gps` 坐标，添加省、市、区 3 个字段：

这里有一份全国 `gps` 地理位置字典表可供使用：

ID	AREANAME	PARENTID	LEVEL	BD09_LAT	BD09_LNG
330127208	王阜乡	330127	4	29.841282569641734	118.9276270139204
330127210	宋村乡	330127	4	29.68968911824958	118.86868291656283
330127211	鸠坑乡	330127	4	29.676365304013252	118.78084930323426
330127212	浪川乡	330127	4	29.498060875067306	118.61175398087234
330127214	界首乡	330127	4	29.55611052373409	118.79513585255172
330127216	安乡乡	330127	4	29.42796863379595	118.83844659359879

5.2.1 开发步骤 1

将全国 `gps` 地理位置参考知识表加工，变成扁平结构

lat	lng	province	city	region
39.9223291372698	116.41369487374497	北京市	北京市	东城区
39.933261013519605	116.41694817338669	北京市	北京市	东城区
39.942102620515094	116.40734228624632	北京市	北京市	东城区
39.95046113360532	116.41900142040949	北京市	北京市	东城区
39.94812725991497	116.4305442069413	北京市	北京市	东城区
39.93675959657538	116.43047934928347	北京市	北京市	东城区
39.92684719848015	116.43692999042177	北京市	北京市	东城区
39.9201179325581	116.43415880955067	北京市	北京市	东城区
39.93653907090478	116.44829420280128	北京市	北京市	东城区
39.96249446905228	116.4303455017625	北京市	北京市	东城区
39.90017648387664	116.41364582806074	北京市	北京市	东城区
39.900713274268604	116.43383452036325	北京市	北京市	东城区
39.90522919111997	116.43798738673588	北京市	北京市	东城区
39.89824902585872	116.44845702067495	北京市	北京市	东城区
39.89313025958401	116.42883455189471	北京市	北京市	东城区
39.895610280075296	116.40548399846834	北京市	北京市	东城区
39.87204158807106	116.40840708459781	北京市	北京市	东城区
39.91074738030027	116.38992279045806	北京市	北京市	西城区

```
create table ref_zb
as
select
lev4.bd09_lat as lat,
lev4.bd09_lng as lng,
lev1.areaname as province,
lev2.areaname as city,
lev3.areaname as region
from t_md_areas lev4 join t_md_areas lev3 on lev4.level=4 and lev4.parentid = lev3.id
```

```
join t_md_areas lev2 on lev3.parentid=lev2.id  
join t_md_areas lev1 on lev2.parentid=lev1.id
```

5.2.2 开发步骤 2

发现一个问题：日志中的 **gps** 坐标，不太可能正好等于知识表中的参考点坐标
查询就很困难！

引入一个 **gps** 坐标查询技术： **geohash** 编码

将参考点表，加工成
geohash 码,省,市,区

将日志中的 **gps** 坐标，变成 **geohash** 码，查询参考点表即可实现需求；

5.3 漏斗模型计算

有分析师定义的业务漏斗模型：

步骤 1： 完成行为 **search**，且行为属性中有： **keyword = 咖啡**

步骤 2： 完成行为 **addCart**，且行为属性中有： **productId** 以 101 开头

步骤 3： 完成行为 **submitOrder**



需要计算出分析师所定义的业务漏斗模型中，每一个步骤的人数

5.4 事件归因分析

用户发生目标行为时，经常需要分析发生的原因，比如用户下单了雀巢咖啡，可能的原因有：

1. 浏览了 app 首页上的“雀巢咖啡”轮播图运营广告
2. 浏览了“猜你喜欢”上的雀巢咖啡推荐广告
3. 领取了“雀巢咖啡”专属优惠券

现在，就上面的模型为例，去分析 2021-08-06 号，发生了“雀巢咖啡”下单事件的人，究竟是因为上面 3 种待归因事件中的哪一种或哪几种引起的，或者每一种原因对目标所产生的影响权重

知识补充： 归因分析，在数据分析领域，有如下几种常见的归因策略：

- 首次触点归因
- 末次触点归因
- 线性归因
- 位置归因
- 时间衰减归因

请使用首次触点归因策略，来归因分析上述场景：

5.5 区间连续活跃分析

假设有 2021-08-05 日的用户活跃区间记录表

用户标识	首次访问时间	活跃区间起始日	活跃区间结束日
D00001	2021-08-01	2021-08-01	2021-08-04
D00001	2021-08-01	2021-08-06	9999-12-31
D00002	2021-08-03	2021-08-03	2021-08-03
D00002	2021-08-03	2021-08-05	2021-08-05

请根据 2021-08-06 日的用户行为日志，更新上面的连续活跃区间记录表

5.6 session 会话切割

由于“多易人才 app”采用了会话保持技术，所以就算用户把 app 放入后台很长时间后再唤醒，会话 id 也会保持不变，这对于后续的访问会话次数、时长等分析带来困扰

请根据用户的两次相邻行为之间是否间隔超过 30 分钟，来将原会话分割成多个会话（附新的会话 id）

6 RDD 深度理解

6.1 RDD 概要

在面向应用开发人员的层面，spark 最重要的一个编程抽象是：

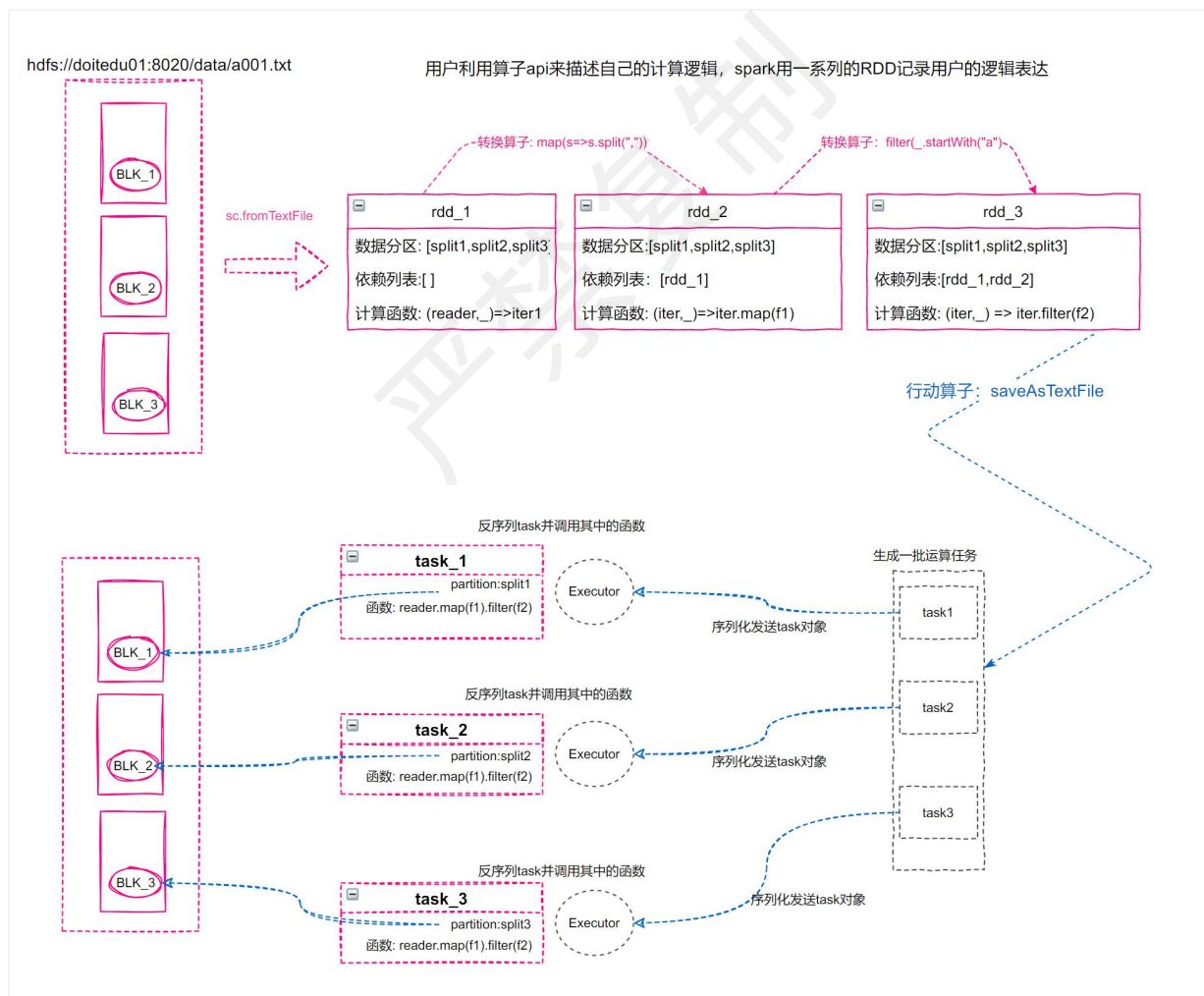
- 将数据抽象成一个 **RDD [弹性分布式数据集]**
- 在 RDD 上提供丰富的编程接口 **[各类算子]** [map/flatMap/filter/reduceByKey/....]

RDD 与 scala 中的各类集合，外在表现非常相似；但其内在实质却相去甚远！

RDD 对象中并不真实存储数据，而只是封装着 **“对数据读写操作的逻辑定义”**：

- 本 RDD 的“数据”将从哪里去读
- 读进来的数据需要做什么样的运算；
- 算完的结果需要写到哪里去

所以，RDD 的本质，更像 scala 中的 **迭代器**！



6.2 RDD 五大属性

1) compute 计算函数（描述本 RDD 的数据该如何计算出来）

一个用于对每个 partition 数据进行逻辑运算的函数：（本质上就是运算逻辑的迭代器）

每个 partition 的计算逻辑是相同的；

```
f: (TaskContext, Int, Iterator[T]) => Iterator[U]
```

2) 依赖 RDD 列表

```
var deps: Seq[Dependency[_]]
```

存储着当前 RDD 所依赖的一个或多个前序 RDD

3) 分区列表

```
private var partitions_ : Array[Partition]
```

Spark 是基于海量数据分布式计算的场景设计的；

会把数据分给多个 task 并行计算，因此，也就需要把数据划分成多个任务片（partition）

- 处理 HDFS 上的文件时，会按文件及偏移量范围划分 partition，通常一个 hdfs 的 block 块对应一个 partition，比如：

```
PARTITION_0 : /inputdata/aaa.txt ,0~128M  
PARTITION_1 : /inputdata/aaa.txt,128M~200M
```

- 处理 MySQL 等数据库中的数据时，会按照用户指定的某个字段的取值范围和指定的分区数进行 partition 划分，比如：

```
PARTITION_0 : from xdb.table_a where id < 100000  
PARTITION_1 : from xdb.table_a where id >=100000 and id<200000  
PARTITION_2 : from xdb.table_a where id >=200000
```

4) [可选] key-value 类型 RDD 的分区器

```
val partitioner: Option[Partitioner]
```

5) [可选] 每个分区的首选计算执行位置

```
final def preferredLocations(split: Partition): Seq[String]
```

6.3 RDD 转换算子的 lazy 原理

6.3.1 scala 迭代器回顾

6.3.1.1 Iterator 接口

```
trait Iterator[+A] extends TraversableOnce[A] {  
  def hasNext: Boolean  
  def next(): A  
}
```

6.3.1.2 Iterable 接口

```
trait Iterable[+A] extends Traversable[A]  
  with GenIterable[A]  
  with GenericTraversableTemplate[A, Iterable]  
  with IterableLike[A, Iterable[A]] {  
  
  def iterator: Iterator[A]  
}
```

6.3.2 scala 迭代器 lazy 现象及原理

6.3.2.1 lazy 现象

```
val iter = List(1, 2).iterator  
val iter2 = iter.map(x=>{println("第 1 级 map 算子"); x*10})  
val iter3 = iter2.map(x=>{println("第 2 级 map 算子"); x+1000})  
iter3.foreach(println)
```

上面代码中，如果把 `foreach` 去掉，会发现，运行程序后没有任何打印输出；

而把 `foreach` 这句代码加上，则会有如下输出：

```
第 1 级 map 算子  
第 2 级 map 算子  
1010
```

```
第 1 级 map 算子  
第 2 级 map 算子  
1020
```

在 scala 中的迭代器上调 map/flatMap/groupBy 等算子时，运算逻辑并不会真正执行，而在调用 foreach 这样的算子后才触发整个算子链条的执行（所谓 lazy 特性）；

这就类似 RDD 上的所谓“转换算子”和“行动算子”

6.3.2 lazy 原理

所谓 lazy 特性，原理其实非常简单，看 map 算子的源码：

```
def map[B](f: A => B): Iterator[B] = new AbstractIterator[B] {  
  def hasNext = self.hasNext  
  def next() = f(self.next())  
}
```

- 1) 迭代器要真正执行计算，需要调用 hasNext 判断是否还有数据，然后 next 取数返回；
- 2) scala 迭代器的 map 算子中，并没有调 hasNext，也没有调 next，所以根本没有执行计算；
- 3) map 方法只是创建一个新的迭代器返回，新的迭代器（子迭代器）中：
 - hasNext 的实现是直接调父迭代器的 hasNext
 - next()是在父迭代器 next()的返回结果上，应用了 map 算子传入的函数 f

如果在 iter1 上连续地调用“转换算子”：

```
val iter2 = iter1.map( f1 )  
val iter3 = iter2.map( f2 )  
// .....  
val iter100 = iter99.map( f99 )
```

它不会真正执行计算；但形成了一个迭代器的链条（也就是数据处理的逻辑链条）：

iter100 中的 next()方法，就是：

```
def next() = {  
    f99 (f98 ( f97(... f3(f2(f1(iter1.next ))) )))  
}
```

因此，当对 `iter100` 调用 `next()` 方法时，就会从 `iter1.next` 拿到数据，然后依次应用函数 `f1, f2, f3...` 的处理，得到最终结果后作为 `iter100.next()` 的返回值；

6.3.3 源头迭代器的理解

迭代器链条的最源端，`hasNext` 和 `next` 是没有“父迭代器”可用的，那么它是什么样子呢？

6.3.3.1 抽象封装需求

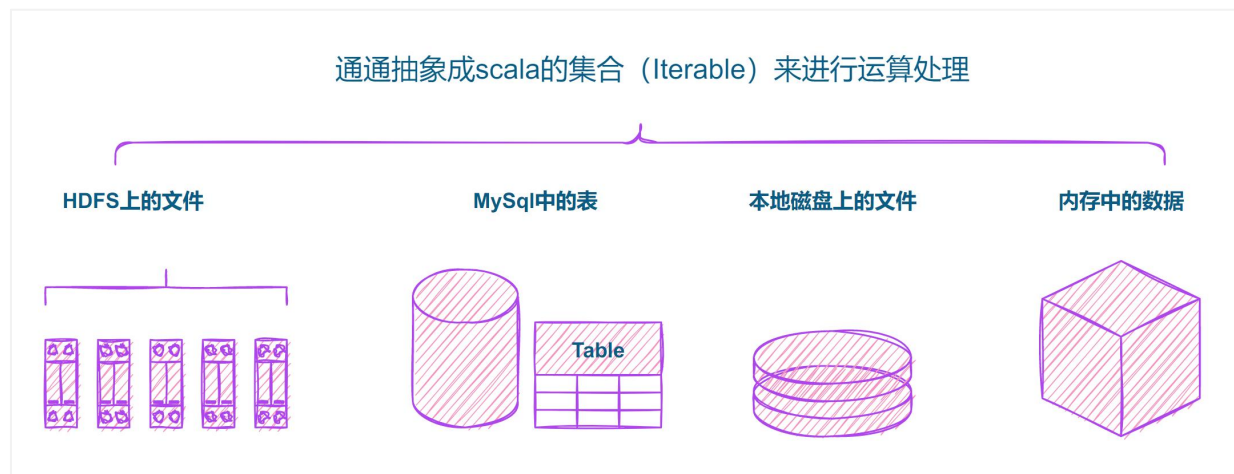
思考一个场景，有如下数据，需要计算每种角色的人数，最大战力，平均战力等

```
1, 张飞, 坦克, 500  
2, 关羽, 战士, 600  
3, 赵云, 刺客, 600  
4, 马超, 战士, 400  
5, 张苞, 坦克, 300  
6, 黄忠, 刺客, 400
```

而数据可能以各种形式存储在各种数据源中：

- `hdfs` 上的文件
- 本地磁盘上的文件
- 内存中的数据
- `mysql` 中的表

现在，你能否设计一个机制，屏蔽掉底层的各种存储差异，统一使用 `scala` 迭代器的方式进行处理？



```
// 从本地磁盘文件加载数据，抽象成 RDD
val rdd1: RDD = Context.fromTextFile("d:/data.txt")

// 从数据库表加载数据，抽象成 RDD
val rdd2: RDD = Context.fromDbTable(
  "jdbc:mysql://hdp01:3306/realtimedw",
  "root",
  "ABC123abc.123",
  "data")

// 计算每一种角色的人数
// 磁盘文件来的数据 rdd1 和数据库来的数据 rdd2，完全使用相同方式进行处理
rdd1.map(_._split(","))
  .groupByKey(_._2)
  .mapValues(_._size)
  .foreach(println)

rdd2.map(_._split(","))
  .groupByKey(_._2)
  .mapValues(_._size)
  .foreach(println)
```

```
D:\devapps\jdk1.8.0_202\bin\java.exe ...
```

```
(战士,3)
```

```
(坦克,2)
```

```
(刺客,2)
```

```
=====
```

```
(战士,3)
```

```
(坦克,2)
```

```
(刺客,2)
```

6.3.3.2 基础封装实战

6.3.3.2.1 将本地磁盘文件封装成迭代器

```
class FileIterator(val path: String) extends Iterator[String] {  
  
    private val br = new BufferedReader(new FileReader(path))  
  
    var line: String = null  
  
    override def hasNext: Boolean = {  
        line = br.readLine()  
        line != null  
    }  
  
    override def next(): String = line  
}  
  
class FileIterable(val path:String) extends Iterable[String] {  
    override def iterator: Iterator[String] = new FileIterator(path)  
}
```

6.3.3.2.2 MySQL 表封装成迭代器

```
package cn.doitedu.spark_sixiang  
  
import java.sql.{Connection, DriverManager, PreparedStatement, ResultSet}  
  
class TableIterator(val url:String ,  
                    val tableName:String,  
                    val userName:String,  
                    val password:String  
                    ) extends Iterator[(Int,String,String,Int)]{  
  
    private val conn: Connection = DriverManager.getConnection(url,userName,password)  
    private val stmt: PreparedStatement = conn.prepareStatement("select id,name,role,battel from battel")
```

```
private val rs: ResultSet = stmt.executeQuery()

override def hasNext: Boolean = rs.next()

override def next(): (Int, String, String, Int) = {
    val id: Int = rs.getInt(1)
    val name: String = rs.getString(2)
    val role: String = rs.getString(3)
    val battel: Int = rs.getInt(4)
    (id,name,role,battel)
}

class TableIterable(val url:String ,
                    val tableName:String,
                    val userName:String,
                    val password:String
                    ) extends Iterable[(Int,String,String,Int)] {
    override def iterator: Iterator[(Int, String, String, Int)] = new
    TableIterator(url,tableName,userName,password)
}
```

6.3.4 RDD 转换算子的 lazy 原理

Rdd 转换算子的 lazy 原理，与 scala 的迭代器算子的 lazy 原理很类似

Scala 迭代器算子： map/flatMap 等，方法的内部实现只是创建了一个新的迭代器返回（并没有去调用迭代器的 hasNext 和 next 进行迭代计算）

Rdd 的转换算子： map/flatMap 等，方法的内部实现只是创建了一个新的 RDD 返回（并没有去调用 RDD 内部的迭代器的 hasNext 和 next 进行迭代计算）

6.4 RDD 全真模拟（有闲情可了解）

需求说明

可以加载各类数据源，得到一个抽象集合

在抽象集合上，提供一些 **transformation** 算子，可以让用户传入计算逻辑，但并不真正执行

在抽象集合上，提供一些 **action** 算子，用户一旦触发，则会执行前阶段所定义好的完整逻辑

代码实战

6.4.1 定义抽象集合的 **trait**

```
trait Rdd {  
  var dep: List[Rdd]  
  val iter: Iterator[String]  
  def map(f: String => String): MapRdd  
  def compute(): Iterator[String]  
  def foreach(f:String=>Unit):Unit  
}
```

6.4.2 定义集合类的具体实现

内存源 RDD

```
class SeqRdd(val c: Seq[String]) extends Rdd {  
  override val iter = compute()  
  override var dep: List[Rdd] = Nil  
  
  override def compute(): Iterator[String] = {  
    c.iterator  
  }  
  
  override def map(f: String => String): MapRdd = {  
    new MapRdd(iter => iter.map(f), List(this))  
  }  
  
  override def foreach(f: String => Unit): Unit = iter.foreach(f)  
}
```

文件源 RDD

```
class FileRdd(val path: String) extends Rdd {  
  override val iter = compute()  
  override var dep: List[Rdd] = Nil  
  
  override def compute(): Iterator[String] = {  
    new FileIterator(path)  
  }  
  
  override def map(f: String => String): MapRdd = {  
    new MapRdd(iter => iter.map(f), List(this))  
  }  
  
  override def foreach(f: String => Unit): Unit = iter.foreach(f)  
  
  class FileIterator(val s: String) extends Iterator[String] {  
    private val br = new BufferedReader(new FileReader(s))  
    private var line: String = _  
    private var flag: Boolean = false  
  }
```

```
override def hasNext: Boolean = {  
    line = br.readLine()  
    if(line != null) flag = true else flag=false  
    flag  
}  
  
override def next(): String = {  
    line  
}  
}  
}
```

中间映射 RDD

```
class MapRdd(val f: Iterator[String] => Iterator[String],  
             var dep: List[Rdd]) extends Rdd {  
    def compute(): Iterator[String] = {  
        f(dep(0).iter)  
    }  
  
    def map(f: String => String): MapRdd = {  
        new MapRdd(iter => iter.map(f), List(this))  
    }  
  
    override val iter: Iterator[String] = compute()  
  
    override def foreach(f: String => Unit): Unit = iter.foreach(f)  
}
```

6.4.3 定义创建集合的工具类

```
class Context{  
    def textFile(path:String):FileRdd = {  
        new FileRdd(path)  
    }  
  
    def makeRdd(seq:Seq[String]):SeqRdd = {  
        new SeqRdd(seq)  
    }  
}
```

```
def runJob(rdd:Rdd): Unit ={  
    val iter = rdd.iter  
    iter.foreach(println)  
}  
}
```

6.4.4 测试代码开发及运行

```
object Test {  
    def main(args: Array[String]): Unit = {  
        val sc = new Context  
        val rdd = sc.textFile("d:/a.txt")  
        val res = rdd.map(_.toUpperCase())  
            .map("xx - " + _)  
            .map(_ ++ " _")  
  
        res.foreach(println)  
    }  
}
```

7 Spark 的 job 调度概念

7.1 分布式运算模式下的新要求

观察如下 wordcount 代码

```
val rdd1 = sc.textFile("data/wordcount.txt")  
val rdd2 = rdd1.flatMap(_.split(" "))  
val rdd3 = rdd2.map(_._1)  
val rdd4 = rdd3.reduceByKey(_+_)
```

如果是单机程序的模式运行，其实用 `scala` 的集合和迭代器算法就轻松完成：

一旦要以分布式程序的模式来运行，就会引入一系列需要解决的问题：如，并行度 `partition` 设置、`stage` 划分、`task` 任务调度等功能：

到此环节，咱们可以对 RDD 的设计思想进行进一步升华：

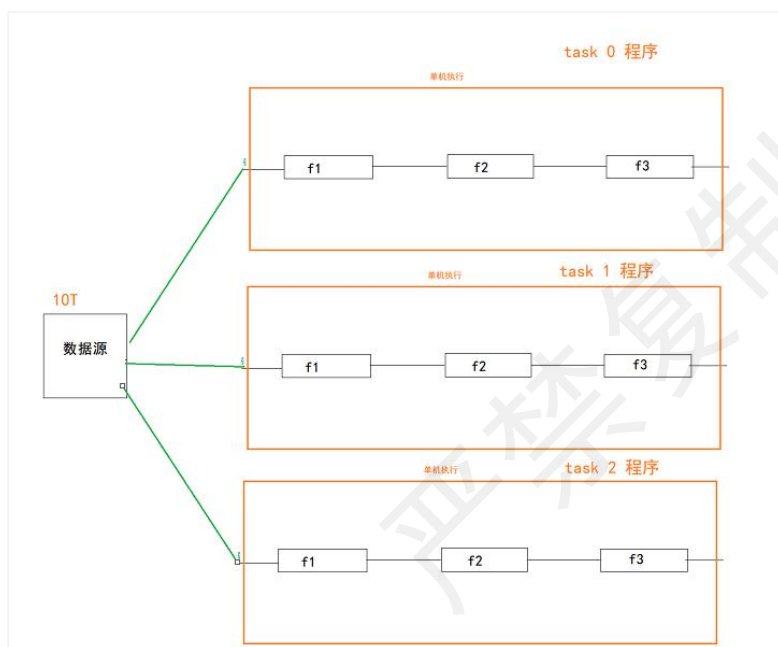
RDD 设计思想 ==> scala 迭代器“描绘”数据运算逻辑 + 分布式任务调度功能封装

7.2 分布式执行引入的问题

7.2.1 问题 1：任务分片

如果用户所编写的运算逻辑链，放置到多个节点上（多个 task）并行执行，

- 会引来一个显见问题：数据会被重复计算处理！

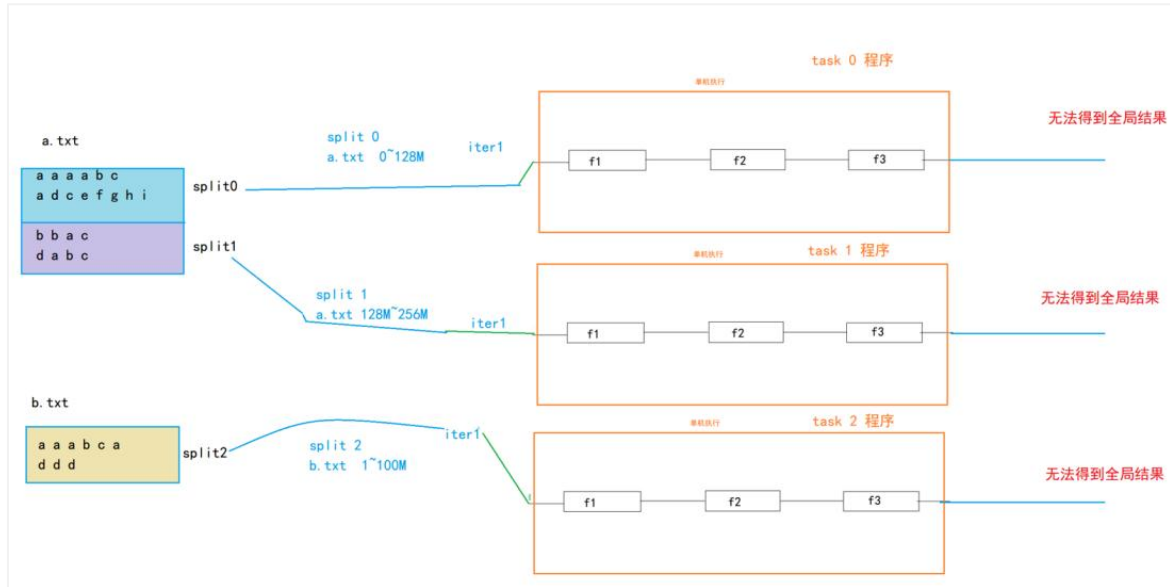


引入任务分片机制，让每个 task 都只负责读取和处理各自所负责的数据分区

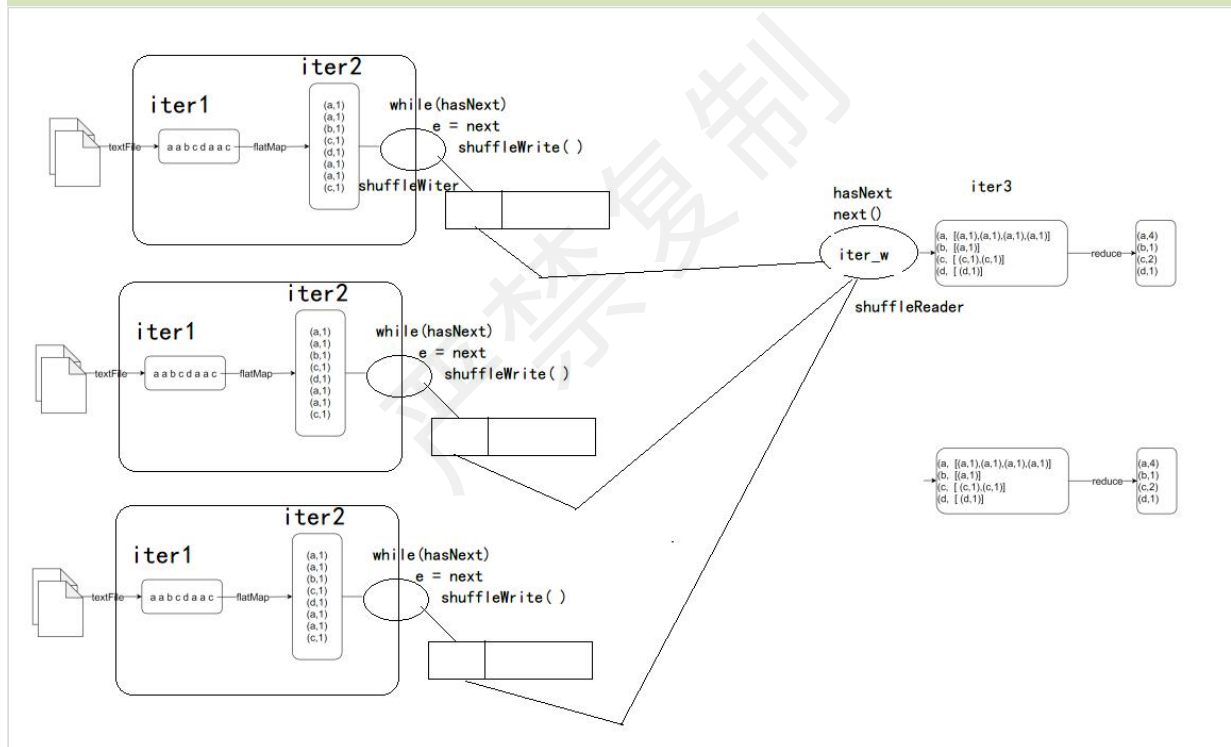
7.2.2 问题 3：shuffle

一旦引入任务分片机制（分区）后，又引入了一个新的问题

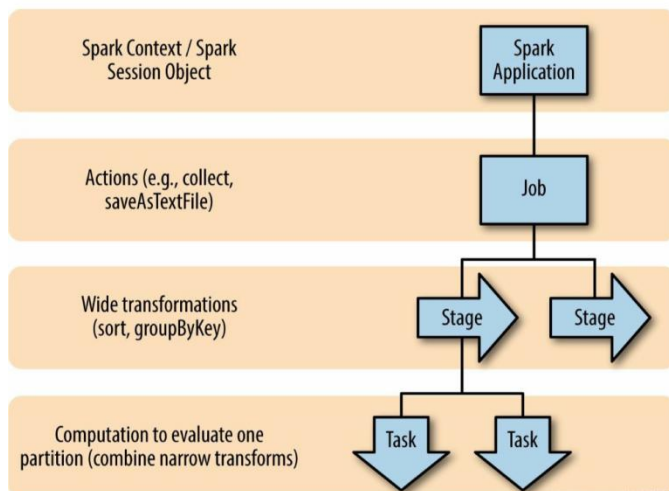
- 每个 task 都只读取了数据源的一部分，无法得出全局结果！



将运算逻辑链划分成多个 stage) 每个 stage 形成独立 task 任务，上下游衔接借助数据的 shuffle



spark 中 rdd 所定义的逻辑链条，就转化成了一个 stage，而每一个 stage 中的“迭代器链”可以交由一台机器去执行（task），而且，同一个 stage 可以交给多台机器（task）去并行执行；



7.2.3 问题 3：行动算子的重新设计

在普通的 `scala` 迭代器中，一旦调用行动算子，则整个迭代器链条将立即执行

分布式运算模式下，需要将“行动算子”的功能改造成：对整个逻辑链条的分布式任务生成及提交、调度（即 job 调度）

Return the number of elements in the RDD.

```
def count(): Long = sc.runJob( rdd = this, Utils.getIteratorSize _).sum
```

Applies a function f to all elements of this RDD.

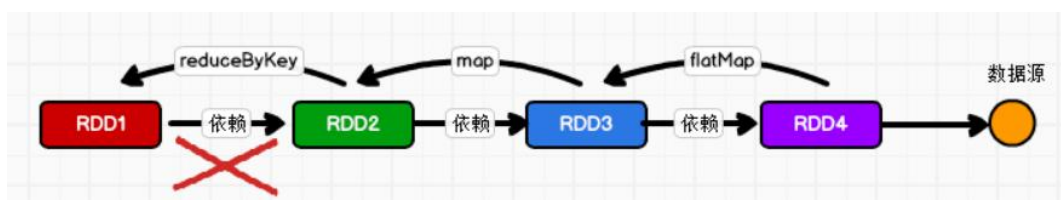
```
def foreach(f: T => Unit): Unit = withScope {
  val cleanF = sc.clean(f)
  sc.runJob( rdd = this, (iter: Iterator[T]) => iter.foreach(cleanF))
}
```

`sc.runJob` 会把用户通过 RDD 定义的运算链，转化成一系列 stage，及 task 计算任务并提交到集群执行；

7.3 rdd 的血缘和依赖

7.3.1 血缘与依赖的概念

- 相邻的两个 RDD 的关系称之为依赖关系,新的 RDD 依赖于旧的 RDD
- 一系列连续的依赖关系,称为血缘关系



每个 RDD 会保存依赖关系,但每个 RDD 不会保存数据,如上图,在 `reduceByKey` 过程中出现错误时,虽然 RDD2 不会保存数据,但可以根据依赖链条(即血缘关系)从数据源重新读取进行计算。

7.3.2 Dependency 依赖关系

7.3.2.1 Dependency 概念

Dependency 不仅仅描述父子血缘,更关键描述了父子 RDD 的 **partitions** 之间的依赖关系;

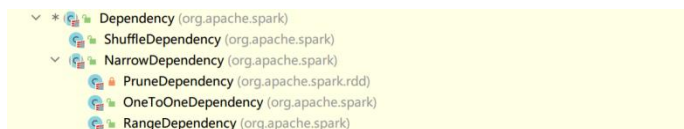
Dependency 是判断是否需要划分 **stage** (即是否需要 **shuffle**) 的依据!

```
abstract class RDD[T: ClassTag]({
  @transient private var _sc: SparkContext,
  @transient private var deps: Seq[Dependency[_]]
}) extends Serializable with Logging {
```

- Dependency 抽象类定义

```
abstract class Dependency[T] extends Serializable {
  def rdd: RDD[T]
}
```

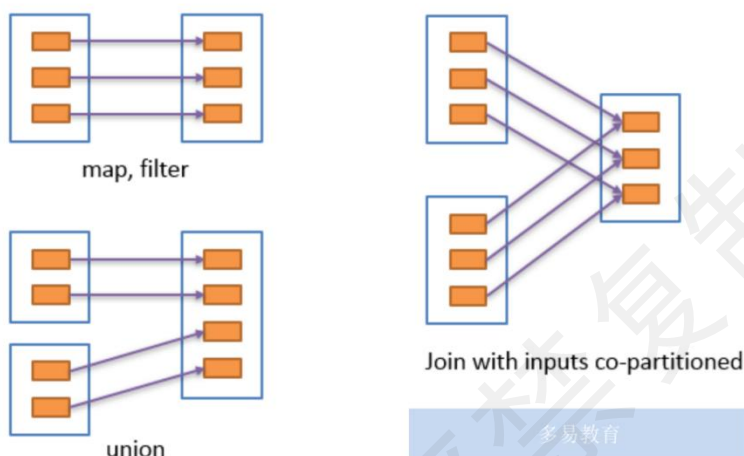
- Dependency 的实现类:
 - ShuffleDependency (宽依赖)
 - NarrowDependency 依赖 (窄依赖)



7.3.2.2 窄依赖

指的是父 RDD 中的一个分区最多只会被子 RDD 中的一个分区使用，意味着子 RDD 的计算逻辑在同一个 task 中执行，不需要经过 Shuffle 来重组数据。

窄依赖包括两种：一对一依赖（OneToOneDependency）和范围依赖（RangeDependency）



● 窄依赖的核心特点

父 RDD 的一个分区，一定只会被子 RDD 的一个分区所依赖；

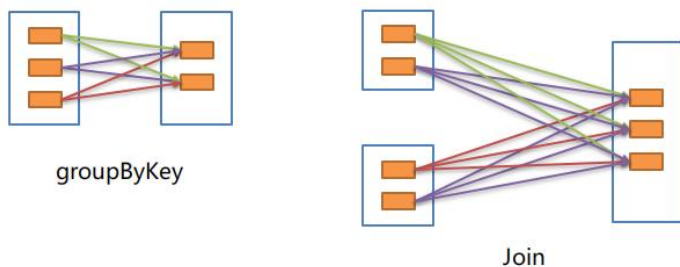
子 RDD 的一个分区，可以依赖父 RDD 的多个分区

7.3.2.3 宽依赖

指的是父 RDD 的一个分区可能会被子 RDD 的所有分区所依赖；

因而父 RDD 一个分区的数据需要被分割成多份，来形成子 RDD 的各个分区；

因此宽依赖也意味着从父 RDD 计算出子 RDD 的过程中，需要一个 Shuffle 过程。



宽依赖在源码中只有一种定义： ShuffleDependency

```
class ShuffleDependency[K: ClassTag, V: ClassTag, C: ClassTag](
  @transient private val _rdd: RDD[_ <: Product2[K, V]], 所依赖的父RDD
  val partitioner: Partitioner, 分区器
  val serializer: Serializer = SparkEnv.get.serializer,
  val keyOrdering: Option[Ordering[K]] = None, Key排序比较器
  val aggregator: Option[Aggregator[K, V, C]] = None, 聚合算子
  val mapSideCombine: Boolean = false) 是否开启map端聚合
  extends Dependency[Product2[K, V]] {

  if (mapSideCombine) {
    require(aggregator.isDefined, "Map-side combine without Aggregator specified!")
  }
  // 返回的就是所依赖的父RDD
  override def rdd: RDD[Product2[K, V]] = _rdd.asInstanceOf[RDD[Product2[K, V]]]
```

- 1) 记录了本 RDD 所依赖的父 RDD
- 2) 记录了生成本 RDD 的 Partitioner
- 3) 记录了生成本 RDD 时要使用的 Key 排序比较器
- 4) 记录了生成本 RDD 时所需要的聚合算子
- 5) 记录了生成本 RDD 时，上游任务是否需要开启 map 端聚合

● 宽依赖的核心特点

父 RDD 的一个分区，会被子 RDD 的多个（所有）分区所依赖

7.3.3 Dependency 的形成

7.3.3.1 Dependency 形成举例

```
val rdd4: RDD[(String, Int)] = rdd3.reduceByKey(_ + _, 2)
```

`reduceByKey` 这个算子生成了 `rdd4` ；

本质来说，`rdd4` 就是由 `reduceByKey` 方法构造出来的；

因而，`rdd4` 与 `rdd3` 的依赖关系，就是由 `reduceByKey` 构造 `rdd4` 时决定的

那么，`rdd1.reduceByKey(_+_)` 是否一定会让返回的结果 `RDD2` 与调用 `RDD1` 形成宽依赖关系呢？

并不是！看源码，如下是 `reduceByKey` 最终返回结果的关键源码：

```
if (self.partition == Some(partitioner)) {
  self.mapPartitions(iter => {
    val context = TaskContext.get()
    new InterruptibleIterator(context, aggregator.combineValuesByKey(iter, context))
  }, preservesPartitioning = true)
} else {
  new ShuffledRDD[K, V, C](self, partitioner)
    .setSerializer(serializer)
    .setAggregator(aggregator)
    .setMapSideCombine(mapSideCombine)
}
```

- 如果走的是 `else`（父子 RDD 的 `partitioner` 不同）

返回一个 `ShuffledRDD`

而 `ShuffledRDD` 里面的 `dependency` 就是一个宽依赖：`ShuffleDependency`

```
override def getDependencies: Seq[Dependency[_]] = {
  val serializer = userSpecifiedSerializer.getOrElse {
    val serializerManager = SparkEnv.get.serializerManager
    if (mapSideCombine) {
      serializerManager.getSerializer(implicitly[ClassTag[K]], implicitly[ClassTag[C]])
    } else {
      serializerManager.getSerializer(implicitly[ClassTag[K]], implicitly[ClassTag[V]])
    }
  }
  List(new ShuffleDependency(prev, part, serializer, keyOrdering, aggregator, mapSideCombine))
}
```

- 如果走的是 `if` 分支（父子 RDD 的 `partitioner` 相同）

返回一个 `MapPartitionsRDD`

```
new MapPartitionsRDD(
  prev = this,
  (context: TaskContext, index: Int, iter: Iterator[T]) => cleanedF(iter),
  preservesPartitioning)
```

这里，看似没有传递 `Dependency` 关系传入 `MapPartitionsRDD` 的构造方法，但是，这个构造方法，会调用父抽象类 `RDD` 的构造方法

```

Order-Sensitive.
private[spark] class MapPartitionsRDD[U: ClassTag, T: ClassTag](
  var prev: RDD[T],
  f: (TaskContext, Int, Iterator[T]) => Iterator[U], // (TaskContext, partition i
  preservesPartitioning: Boolean = false,
  isFromBarrier: Boolean = false,
  isOrderSensitive: Boolean = false)
  extends RDD[U](prev) {

```

而 RDD 的构造函数，会再调另一个辅助构造函数，并传入了一个 `OneToOneDependency` 依赖属性

```

Construct an RDD with just a one-to-one dependency on one parent
def this(@transient oneParent: RDD[_]) =
  this(oneParent.context, List(new OneToOneDependency(oneParent)))

```

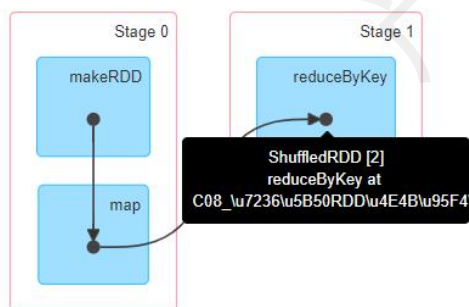
7.3.3.2 Dependency 形成实证

- 验证 `reduceByKey` 返回的 `rdd` 中的 `Dependency`

1) 如下代码，`reduceByKey` 将返回一个 `ShuffledRDD`（形成宽依赖关系）

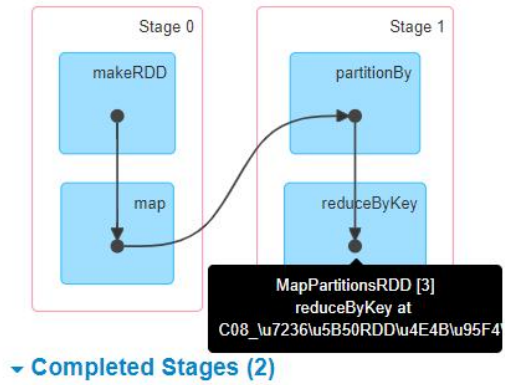
```
val res = rdd1.map(x=>(x,10)).reduceByKey( _ + _,2 )
```

▶ Event Timeline
▼ DAG Visualization



2) 如下代码，`reduceByKey` 将返回一个 `MapPartitionsRDD`（形成窄依赖关系）

```
val res = rdd1.map(x=>(x,10)).partitionBy(new HashPartitioner(2)).reduceByKey( _ + _,2 )
```



结论：子 RDD 和父 RDD 之间的依赖关系类型，是由算子在构造子 RDD 时决定

比如，算子返回 `ShuffledRDD`，父子之间就是 `ShuffleDependency`（宽依赖）

比如，算子返回 `MapPartitionsRDD`，父子之间就是 `OneToOneDependency`（窄依赖）

JUST DO IT
多易教育

7.4 Job 调度中的核心概念

Spark 中的 Job 调度：是指 spark 框架底层根据 rdd 逻辑链条，划分 stage，生成 task 并提交执行的过程；

7.4.1 名词术语理解

- **Application(应用)**：创建一个 sparkContext，就生成了一个 application

- **job**：一个行动算子就触发了一个 job（此说法：直观但不精确）

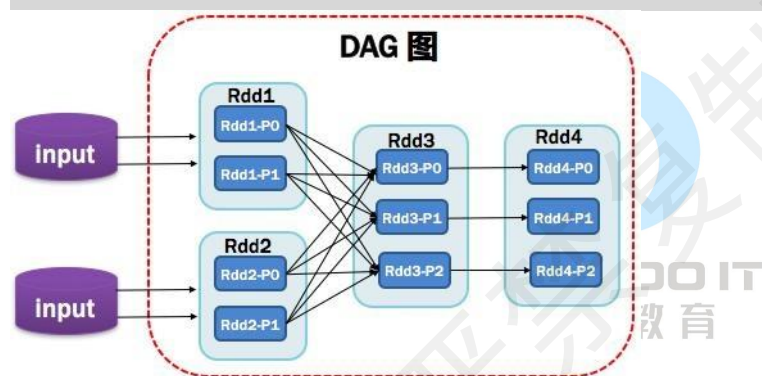
有些行动算子，可能触发多个 job，比如 `take()`；

有些 transformation 算子，也可能会触发 job，比如 `sortByKey()` 算子[因为它要用 `rangepartitioner`]

一个 job 就是一个 dag 的运算流程（触发了一次 `sc.runJob()` 就是一次 job）

而每一个 job 中，都包含如下概念：

DAG / stage / taskSet / task



- DAG

DAG(Directed Acyclic Graph)叫做有向无环图，是的一系列 RDD 转换关系的描述

- Stage（阶段）

以 shuffle 为分界线，将 DAG 转换逻辑从整体划分成段，每一段就称之为一个 Stage

（本质就是：一个 stage 中的 rdd 计算逻辑可以放在一个 task 上执行）

- Task/Task Set

每一个 stage 中包含的逻辑链条段，需要提交集群去并行执行，每个执行实例就称之为一个 task；

而这个 stage 所转化出来的这一批 task，称之为 task set（一个 taskset 中所有 task 执行的计算逻辑都相同，不同的只是负责的数据分区不同）；task set 中 task 个数，由这个 stage 的 finalRdd 分区数决定！

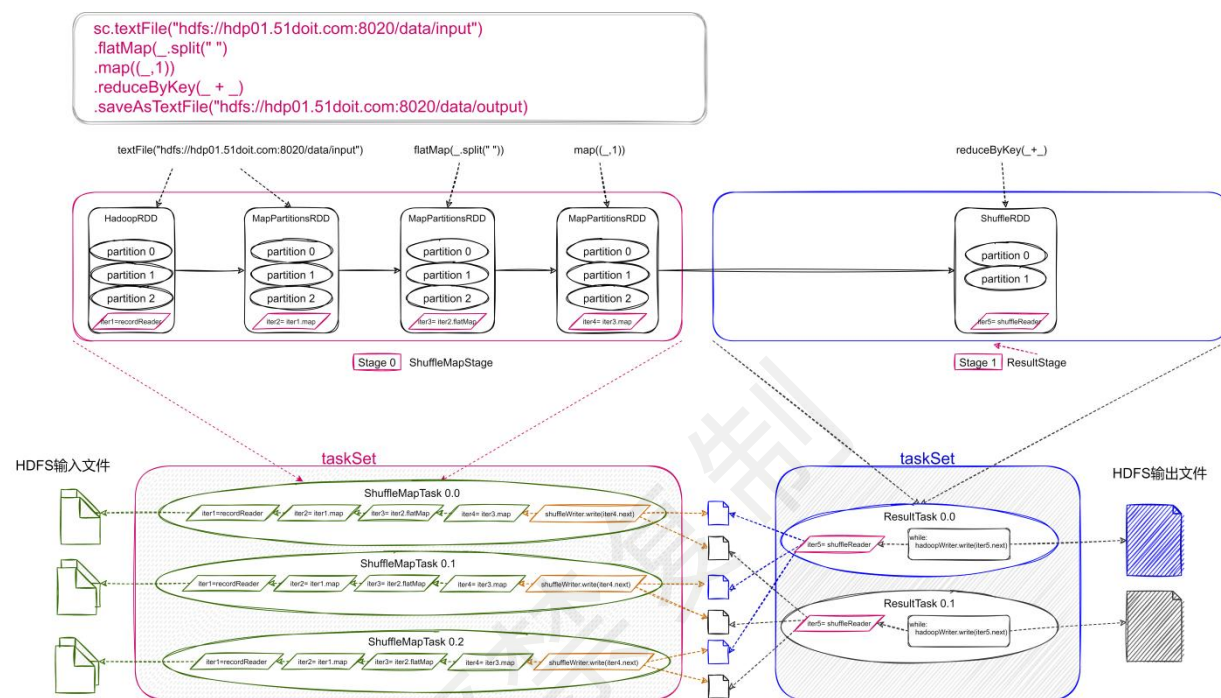
（task，本质上是一个类，它封装了调用 rdd 迭代器的代码以及将计算结果输出的代码）

Task 在 spark 内部共有 2 种: **shuffleMapTask** 和 **resultTask**

最后一个 stage 所产生的 task, 是 resultTask

其他 stage 所产生的 task, 都属于 shuffleMapTask

7.4.2 核心概念示意图



8 Partition (分区) 与 Partitioner (分区器)

分区 Partition : 分区代表 RDD 数据集的一部分; 每个分区的数据都需要一个 task 并行实例来计算; RDD 有几个分区, 则在计算数据时就需要几个 task 并行度!

分区的描述要素:

- 分区号
- 分区数据所在位置
 - 源头 RDD: 数据的位置是用数据源的相关特性来描述的;
 - 比如文件路径和偏移量; 表和 id 值范围;
 - 中间 RDD: 数据的位置是用父 RDD 的分区依赖关系描述;

分区器 Partitioner : 描述的是 RDD 的数据在各个分区之间的分布规则！

最常用的分区器: HashPartitioner;

(数据按照 key 的 $\text{hashCode} \% \text{分区数} = \text{partitionid}$)

分布式计算, 基本上离不开任务划分, 任务划分就离不开数据分片 (只不过有些框架中称之为 Split, 有些框架中称之为 Partition)

在 spark 中, partitions 是 RDD 的五大属性之一, 尤可见其重要性!

8.1 RDD 必有 Partitions

因为一个 rdd 将来要分成多个并行的 task 去执行计算,

那么, rdd 必须描述自己有哪些 Partition! 并描述分区的数据从何而来;

- File 类的 Partition: /input/a.txt 0 ~ 128M
- Seq 集合类的 Partition: 直接就存了 Seq 的一段数据
- DB 类的 Partition: db.table where id > 10000 and id < 2000

8.2 RDD 的 partitions 数是如何决定的

- 数据源 RDD 的分区数, 由数据源的读取器决定

举例: `sc.textFile` 产生的 rdd, 分区数是由 `TextInputFormat.getInputSplits()` 方法决定的;

`getInputSplits()` 方法的分区逻辑: 挨个遍历输入目录下的文件, 默认按 `blocksize` 进行切片;

自己思考: `new JdbcRDD(conn,rowMap,sql,lowerBound,upperBound,numPartitions)`

自己思考: `sc.parallelize(1 to 100000,3)`

- 后续的窄依赖子 RDD (`map/flatMap/mappartitions/filter/mapvalues`), 分区数是一路传承不改变的;

特例: `val rdd2 = rdd1.coalesce(2,false)`

rdd2 中的一个分区, 映射了 rdd1 的多个固定的分区;

比如 rdd2 的 p0 映射了 rdd1 的 p0,p3,p4

比如 rdd2 的 p1 映射了 rdd1 的 p1,p2

- 后续宽依赖子 RDD，分区数是由 shuffle 算子 `reduceByKey(f,4) /groupBy(4) /groupByKey(4) /cogroupByKey(4) /join(rdd2,4)` 传入“分区数参数”来决定的

- 在可以传入一个“分区数参数”来决定 rdd 分区数的算子中，如果没有传入“分区数参数”，则如何决定分区数呢？【策略：默认并行度 `defaultParallelism()`】

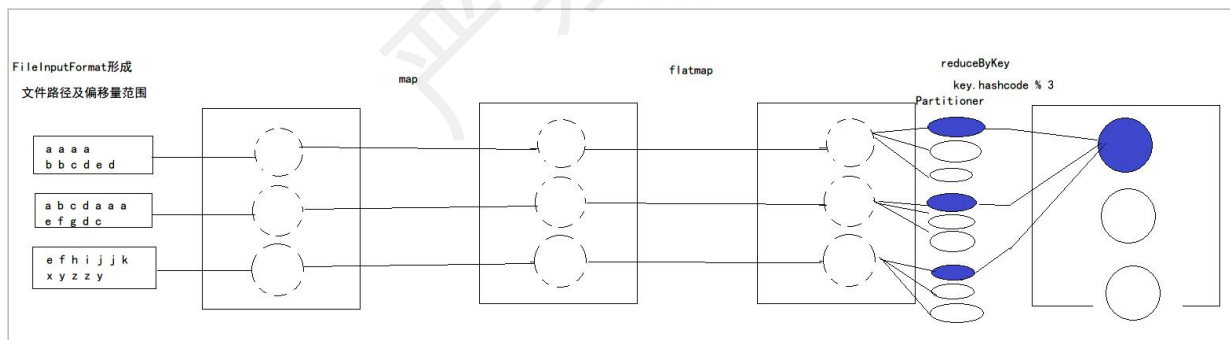
优先根据 `spark.default.parallelism` 参数值作为默认分区数

如果参数没配，则参考运行时能用的 cpu 核数：

(local 运行模式下: `getInt("spark.default.parallelism", totalCores)`)

集群运行模式: `getInt("spark.default.parallelism", math.max(totalCoreCount.get(), 2))`

8.3 RDD 不一定有 Partitioner (分区器)



数据源 rdd，会通过**数据源读取器**的内在机制进行分区，不需要 Partitioner；

而中间的窄依赖 rdd，直接一对一依赖于前面 rdd 的分区，所以没有发生任何数据分布变化，也就不需要所谓的 Partitioner；

而要 shuffle 的时候，需要把上游分区中的数据混洗成下游的多个分区，就需要一个分区规则 Partitioner 来决定一条数据究竟该放到下游的哪一个分区！

8.4 RDD 的 Partitioner 是如何决定的

- 1) 源头 RDD 的分区数是由数据源的读取器机制内部决定的，而且通常没有分区器；
- 2) 窄依赖子 RDD 分区器通常为 None，也可以选择让它保留父 RDD 的分区器；
- 3) 宽依赖 RDD（ShuffledRDD）的分区数、分区器：
 - a. 可以由算子传入 `reduceByKey(_ + _, 4)`，分区器就是 `HashPartitioner`；
 - b. 可以由算子传入分区器 `reduceByKey(new HashPartitioner(4), _ + _)`；
 - c. 可以什么都不传，则调用 `Partitioner.defaultPartitioner()` 这个方法来得得到一个分区器：

```
// 求默认分区器的策略方法
def defaultPartitioner(rdd: RDD[_], others: RDD[_]*): Partitioner = {
  // 将所有传入的父 rdd 组成一个 list
  val rdds = (Seq(rdd) ++ others)
  // 筛选出父 RDD 中拥有 "分区器"的 rdd
  val hasPartitioner = rdds.filter(_.partitioner.exists(_.numPartitions > 0))

  // 找到拥有最大分区器的 rdd
  val hasMaxPartitioner: Option[RDD[_]] = if (hasPartitioner.nonEmpty) {
    Some(hasPartitioner.maxBy(_.partitions.length))
  } else {
    None
  }

  // 找默认分区数，有参数配置的，就用参数配置的值； 没有的话，就取上游所有 rdd 中的最大分区数
  val defaultNumPartitions = if (rdd.context.conf.contains("spark.default.parallelism")) {
    rdd.context.defaultParallelism
  } else {
    rdds.map(_.partitions.length).max
  }

  // 如果 (存在拥有分区器的父 rdd && [ 父 RDD 中的最大分区数/父 RDD 中的最大分区器的分区数 < 10 倍 || 默认分区数<=最大分区器的分区数 ])
  // 就用上游的那个最大分区器作为子 RDD 的分区器

  // 否则，就新创建一个 HashPartitioner 分区器，分区数=默认分区数 (参数, 上游的最大分区数)
  if (hasMaxPartitioner.nonEmpty && (isEligiblePartitioner(hasMaxPartitioner.get, rdds) ||
    defaultNumPartitions <= hasMaxPartitioner.get.getNumPartitions)) {
    hasMaxPartitioner.get.partition.get
  }
}
```

```

    } else {
        new HashPartitioner(defaultNumPartitions)
    }
}

```

8.5 分区器/分区数的简明结论

源头 RDD 的分区数是由数据源的读取器机制内部决定的，而且通常没有分区器；

窄依赖 RDD 总是沿袭前一个 RDD 的分区数，（分区器为 None，也可以选择让它保留前一个分区器）

宽依赖 RDD（ShuffledRDD）的分区数、分区器：

可以由算子传入分区数 `reduceByKey(_ + _, 4)`，分区器就是 `HashPartitioner`

可以由算子传入分区器 `reduceByKey(new HashPartitioner(4), _ + _)`

可以什么都不传：

调用 `Partitioner.defaultPartitioner()` 这个方法来得一个分区器

如果：上游有分区器，[且，最大分区器分区数>默认并行度 or 上游最大分区数/最大分区器分区数<10]

- 选上游最大分区器（分区数）

否则：

- 用 `HashPartitioner`；分区数优先用 `spark.default.parallelis`，如无，则用上游最大分区数；

8.6 shuffle 算子不一定需要 shuffle 的原因

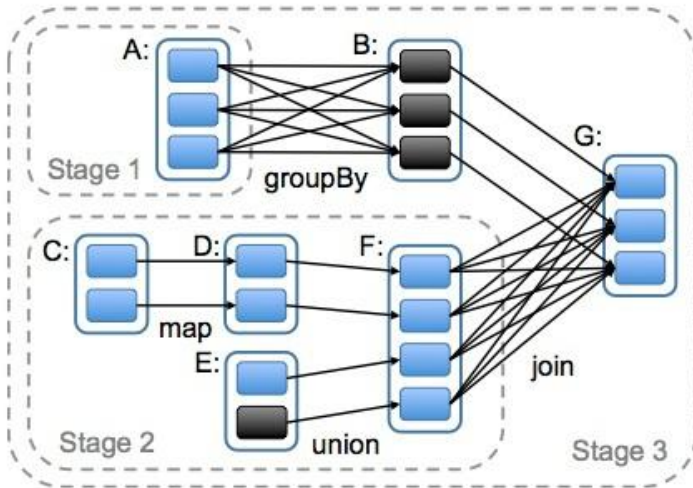
观察现象：

如下情况中，多个“shuffle 算子”都没有真正生成 shuffle 依赖

```

val rd11 = rdd1.map(_._1)
    .groupByKey(tp => tp._1)
val rd21 = rdd2.map(_._1)
    .groupByKey(tp => tp._1)
// join 没有引起 shuffle
rd11.join(rd21).count()
// cogroup 没有引起 shuffle
rd11.cogroup(rd21).count()
// reduceByKey 也没有引起 shuffle
rd11.reduceByKey((iter1,iter2)=>iter1)

```



内在原因：

join/reduceByKey/cogroup 等算子的底层原理中都会判断：

上游 RDD 和下游 RDD，如果分区器和分区数都相等，则执行分组聚合或者 join 等逻辑时，数据在上下游的分区分布规则完全相同，因而也就不需要 Shuffle，算子也就不会返回 ShuffledRDD（也就没有 ShuffleDependency），而是返回 MapPartitionsRDD 即可；

JUST DO IT
多易教育

9 Spark 程序分布式运行模式

spark 程序分布式运行的模式有很多种：

Standalone 模式 (Client/cluster)：在 spark Standalone 集群上运行

Yarn 模式 (Client/cluster)：在 yarn 上运行

Mesos/k8s：在 mesos 集群或者 k8s 集群上运行

9.1 spark 运行时主要角色

- SparkSubmit (进程)

应用提交的客户端程序

- Driver (线程)

含有 SparkContext 实例的那个线程

(功能：利用 sparkcontext 做 stage 切分，生成 taskset，调度 task)

- ApplicationMaster (进程) (yarn-cluster 模式下有)

类同于 MapReduce 中的 MRAppMaster (local 模式中是 LocalJobRunner)

是 yarn 的规范中要求的用户程序的主管进程！

ApplicationMaster 负责申请容器启动 executor，及调用 Driver 线程的功能：

- ExecutorLauncher (yarn-client 模式下有)

作为一个简化的 applicationMaster 运行在容器中，只负责向 RM 申请容器申请启动 executor；

- Executor (进程)

Executor 是 Task 的具体执行者；

它负责调用 task 中的 runTask 方法来执行 task 的运算逻辑；

(executor 是进程，task 是被线程所调用的对象)

- Task (对象)

Task 中就是封装了对 FinalRDD 的迭代器进行循环调用的逻辑；

Task 分为 ShuffleMapTask 和 ResultTask

ShuffleMapTask:

往下游输出，是用 ShuffleWriter 写 shuffle 溢出文件；

ResultTask

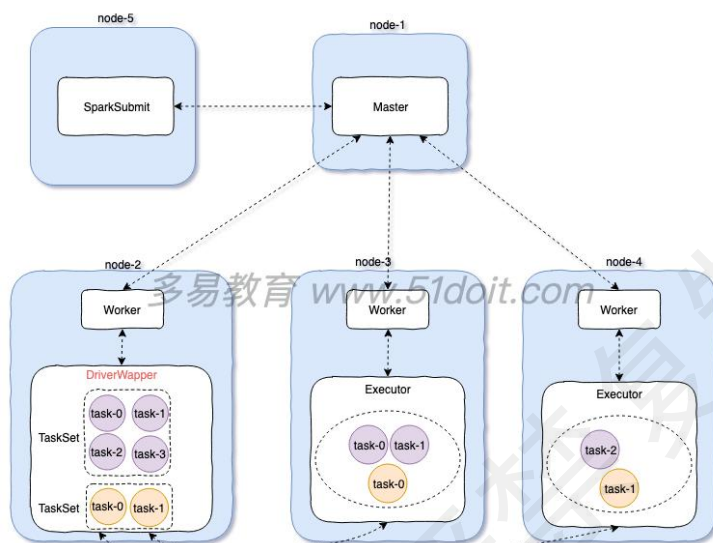
往下游输出，是用外部存储系统的客户端 (api) 向外部系统输出最终结果；

(shuffleMapTask、resultTask 从哪里读取数据？如果该 task 是整个 DAG 中第一个 stage 的 task，则迭代器是外部存储系统的 reader；如果是中间 stage 的 task，则迭代器就是一个 shuffleReader)

9.2 Spark StandAlone 的执行模式

9.2.1 spark standalone 资源调度集群安装

Spark standalone 是一个类似于 yarn 的资源调度集群



安装步骤:

1. 上传安装包解压
2. 修改 spark-env.sh

```
export JAVA_HOME=/opt/apps/jdk
export HADOOP_CONF_DIR=/opt/apps/hadoop/etc/hadoop
export YARN_CONF_DIR=/opt/apps/hadoop/etc/hadoop
```

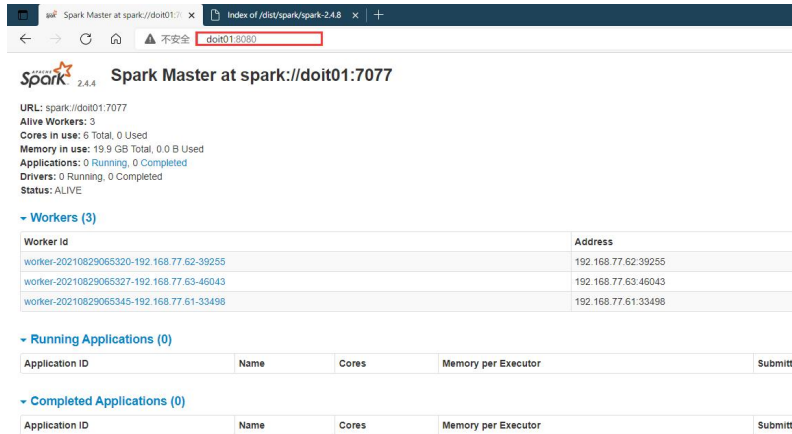
3. 修改 slaves 文件, 添加 worker 节点

```
doit01
doit02
doit03
```

4. 将安装包分发到其他节点

5. 启动 standalone 集群

sbin/start-all.sh

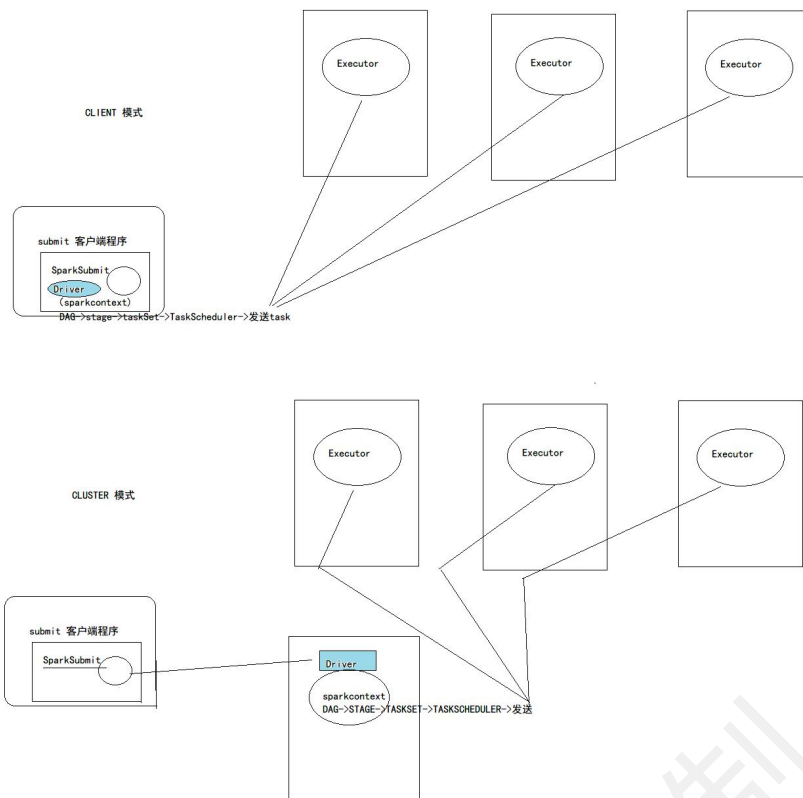


此界面是 standalone 集群的监控界面
而每一个 application 都有一个自己的监控页面

9.2.2 client 模式 vs cluster 模式

client 模式: Driver 是在 SparkSubmit 进程中

cluster 模式: Driver 是在 Worker 节点上运行



9.2.3 提交方式

9.2.3.1 spark-submit 脚本方式提交

- 提交为 **client** 模式

程序 jar 包可以放在本地磁盘

```
# 参数解析
bin/spark-submit --master spark://doit01:7077 \
--deploy-mode client \
--class cn.doitedu.spark.WordCount \
--name "帅无边男人的帅无边程序" \
# --jars /root/mysql-connector-java-8.0.25.jar,/root/fastjson-1.2.78.jar 给 driver 和 executor 添加依赖
# --packages com.alibaba:fastjson:1.2.78,mysql:mysql-connector-java:8.0.25 给 driver 和 executor 添加依赖
# --files /root/user.txt 添加缓存文件到每一个 executor 的工作目录中
# --conf spark.default.parallelism=200 定义 spark 的一些参数属性
# --properties-file /root/spark-confs.properties 定义 spark 的一些参数属性
--driver-memory 1G \
# --driver-java-options 为 driver 设置一些 jvm 参数
```

```
# --driver-library-path /root/lib/ 给 driver 添加依赖
# --driver-class-path /root/a.jar,/root/b.jar 给 driver 添加依赖
--executor-memory 2G \
--executor-cores 1 # 每个 executor 用的核数

--num-executors 10 # yarn cluster 模式下指定 executor 的总个数
# --driver-cores 1 在 cluster 模式下, driver 程序所用的 cpu 核数
--supervise #在 standalone 的 cluster 模式下, Driver 失败会自动重启
--total-executors-cores # 整个 application 的所有 executor 的总核数
```

```
##### 完整示例 #####
# Usage: spark-submit [options] <app jar | python file | R file> [app arguments]
bin/spark-submit --master spark://doit01:7077 \
--deploy-mode client \
--class cn.doitedu.spark.WordCount \
--name "帅无边男人的帅无边程序" \
--driver-memory 1G \
--executor-memory 2G \
--executor-cores 1 \
--total-executor-cores 2 \
/root/sparktest.jar hdfs://doit01:8020/sparktest/wordcount/input
hdfs://doit01:8020/sparktest/wordcount/output
```

运行起来后, 集群中会看到这两种进程:

```
[root@doit01 spark]# jps
4464 Master
4546 Worker
8291 SparkSubmit
8358 CoarseGrainedExecutorBackend
5544 DataNode
8378 Jps
5407 NameNode
```

运行中出现过 3 个问题:

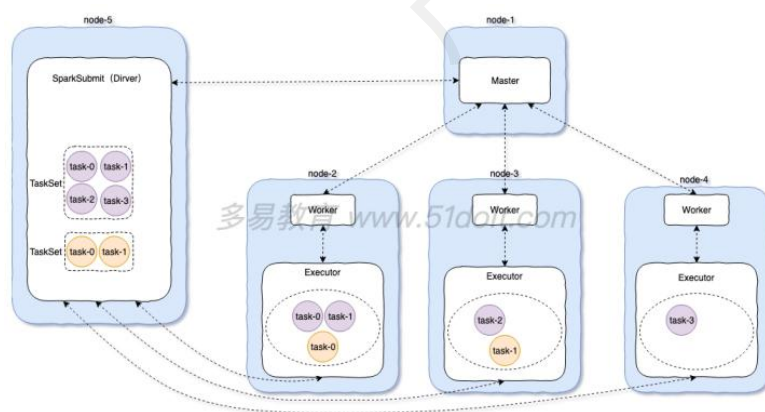
1. 缺少类 (缺少的是 scala 的类) 原因: 缺少 scala 依赖包 解决: 将 scala 依赖打入程序 jar
2. Scala 中的某类找不到某方法 (版本冲突) 原因: 集群环境是 spark-scala_2.11, 而程序编译依赖的是 scala_2.12 解决: 修改程序的编译依赖为 scala_2.11
3. 内存溢出 OutOfMemory 分析: stage0 就溢出, 也就是 LineRecordReader.next 方法就溢出 原因: 一行太长 (整个 4G 文件就只有一行)

● 提交为 **cluster** 模式

需要将程序的 jar 包上传到 hdfs。Driver 运行在集群中, 不在 SparkSubmit 进程中。

运行起来后，集群中会出现如下独立的进程：

```
[root@doit02 ~]# jps
1652 Worker
2535 SecondaryNameNode
3611 CoarseGrainedExecutorBackend
3660 Jps
2429 DataNode
[root@doit02 ~]#
```



9.2.3.2 idea 方式提交

```
import org.apache.spark.rdd.RDD
import org.apache.spark.{SparkConf, SparkContext}

object SparkDemo {

  def main(args: Array[String]): Unit = {

    val conf: SparkConf = new SparkConf().setAppName("SparkDEMO")
      .setMaster("spark://doitedu01:7077")
      .set("spark.yarn.jars", "hdfs://doitedu01:8020/system/spark/yarn/jars/*")
      .setJars(List("/home/doitedu/SparkLearn.jar"))

    val sc: SparkContext = new SparkContext(conf)

    val dataRDD: RDD[Int] = sc.parallelize(Array(1,2,3,4))

    val result = dataRDD.map(res => res * 2)
    result.collect().foreach(println(_))
    print(result.count())
  }
}
```

9.2.3.3 REST 方式提交

```
curl -X POST http://node-1.51doit.cn:6066/v1/submissions/create --header
"Content-Type:application/json;charset=UTF-8" --data '{
  "action": "CreateSubmissionRequest",
  "clientSparkVersion": "2.3.3",
  "appArgs": ["hdfs://node-1.51doit.cn:9000/wc", "hdfs://node-1.51doit.cn:9000/out6666"],
  "appResource": "hdfs://node-1.51doit.cn:9000/jars/spark-test.jar",
  "environmentVariables": {
    "SPARK_ENV_LOADED": "1"
  },
  "mainClass": "cn._51doit.spark.day01.WordCount",
  "sparkProperties": {
    "spark.jars": "hdfs://node-1.51doit.cn:9000/jars/spark-test.jar",
```

```
"spark.driver.supervise": "false",  
"spark.app.name": "WordCount",  
"spark.eventLog.enabled": "false",  
"spark.submit.deployMode": "cluster",  
"spark.master": "spark://node-1.51doit.cn:6066"  
}  
'
```



9.3 Spark On Yarn 模式

9.3.1 YARN 要点回顾

9.3.1.1 YARN 的基本组成

- Resource Manager(RM)

全局的资源管理器,负责整个系统的资源管理和分配,由调度器(ResourceScheduler)和应用管理器(Applications Manger,ASM)组成

- 调度器(ResourceScheduler)

调度器根据容量,队列等限制条件,将系统中的资源分配给各个正在运行的应用程序.调度器不参与任何应用程序的具体工作,仅根据应用程序的资源需求进行资源分配.调度器是个可拔插的组件,用户可根据自己的需要设计新的调度器.

- 应用程序管理器(ASM)

应用程序管理器负责整个系统中所有应用程序,包括应用程序的提交,与调度器协商资源以启动 ApplicationMaster(AM),监控 AM 运行状态并在失败时重启它

- Node Manager(NM)

NM是每个节点上的资源和任务管理器.一方面,它会定时的向RM汇报本节点上的资源使用情况和各个 Container 的运行状态;另一方面,它接收并处理来自AM的 Container 启动\停止等请求 Container

- Application Master(AM)

用户提交的每个应用程序均包含一个 AM,主要功能包括:

- ① 与 RM 调度器协商以获取资源
- ② 将得到的资源进一步分配给内部的任务
- ③ 与 NodeManager(NM)通信,以启动\停止任务
- ④ 监控当前任务对应的容器运行状态,并在任务运行失败时重新为任务申请资源重启任务

9.3.1.2 YARN 的调度策略

← → ↺ ↻ ⚠ 不安全 | doit01:8088/cluster/scheduler?openQueues=Queue:%20default

 **NEW,NEW_SAVING,SUBMITTED,ACCEPTED,RUNNING**

Cluster

- About
- Nodes
- Node Labels
- Applications
- NEW
- NEW_SAVING
- SUBMITTED
- ACCEPTED
- RUNNING
- FINISHED
- FAILED
- KILLED
- Scheduler**

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total
0	0	0	0	0	0 B	24 GB

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy
3	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:1024, vCores:1>	<memory:8192, vCores:4>

Dump scheduler logs 1 min

Application Queues

Legend: Capacity Used Used (over capacity) Max Capacity Users Requesting Resources Auto Capacity

Queue: root

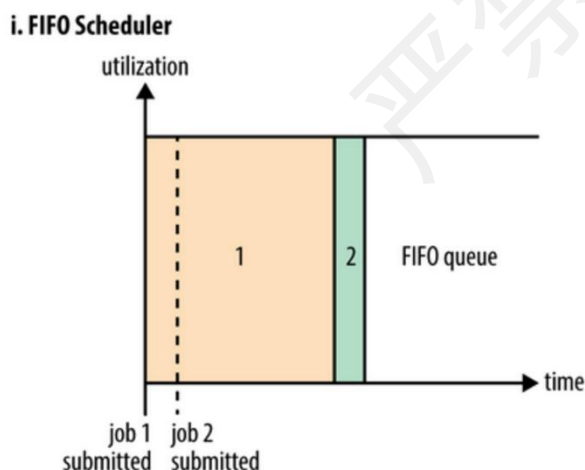
Queue: default

Show 20 entries

ID	User	Name	Application Type	Queue	Application Priority	StartTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU Vcores	Allocated Memory MB	Reserved CPU Vcores
No data available in table													

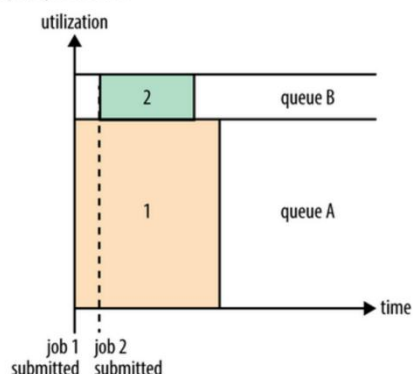
Showing 0 to 0 of 0 entries

9.3.1.2.1 fifo 调度策略



9.3.1.2.2 capacity 调度策略

ii. Capacity Scheduler



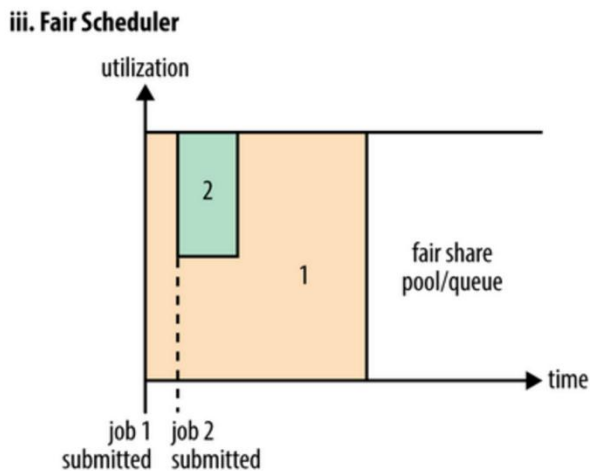
Capacity 调度器的配置文件，文件名为 `capacity-scheduler.xml`。在这个配置中，在 `root` 队列下面定义了两个子队列 `prod` 和 `dev`，分别占 40% 和 60% 的容量。需要注意，一个队列的配置是通过属性 `yarn.scheduler.capacity.<queue-path>.<sub-property>` 指定的，`<queue-path>` 代表的是队列的继承树，如 `root.prod` 队列，`<sub-property>` 一般指 `capacity` 和 `maximum-capacity`。

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>yarn.scheduler.capacity.root.queues</name>
    <value>prod, dev</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.dev.queues</name>
    <value>eng, science</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.prod.capacity</name>
    <value>40</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.dev.capacity</name>
    <value>60</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.dev.maximum-capacity</name>
    <value>75</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.dev.eng.capacity</name>
    <value>50</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.dev.science.capacity</name>
    <value>50</value>
  </property>
</configuration>
```



capacity-scheduler.xml

9.3.1.2.3 fair 调度策略



调度器的使用是通过 `yarn-site.xml` 配置文件中的 `yarn.resourcemanager.scheduler.class` 参数进行配置的，默认采用 Capacity Scheduler 调度器。如果我们要使用 Fair 调度器，需要在这个参数上配置 FairScheduler 类的全限定名：

```
org.apache.hadoop.yarn.server.resourcemanager.scheduler.fair.FairScheduler。
```

Fair 调度器的配置文件位于类路径下的 `fair-scheduler.xml` 文件中，这个路径可以通过 `yarn.scheduler.fair.allocation.file` 属性进行修改。

```
<?xml version="1.0"?>
<allocations>
  <defaultQueueSchedulingPolicy>fair</defaultQueueSchedulingPolicy>

  <queue name="prod">
    <weight>40</weight>
    <schedulingPolicy>fifo</schedulingPolicy>
  </queue>

  <queue name="dev">
    <weight>60</weight>
    <queue name="eng" />
    <queue name="science" />
  </queue>

  <queuePlacementPolicy>
    <rule name="specified" create="false" />
    <rule name="primaryGroup" create="false" />
    <rule name="default" queue="dev.eng" />
  </queuePlacementPolicy>
</allocations>
```

9.3.2 On Yarn 必要配置

1) 需要在所有节点的/etc/profile 中配置 HADOOP_CONF_DIR 的目录, 目的是为了让 Spark 找到 core-site.xml、hdfs-site.xml 和 yarn-site.xml、capacity-scheduler.xml, 不然会报错:

```
export HADOOP_CONF_DIR=/bigdata/hadoop-3.2.1/etc/hadoop/
```

2) 关闭内存资源检测

修改 yarn-site.xml (使用虚拟机环境、生产环境不需要修改)

```
<property>
  <name>yarn.nodemanager.pmem-check-enabled</name>
  <value>>false</value>
</property>

<property>
  <name>yarn.nodemanager.vmem-check-enabled</name>
  <value>>false</value>
</property>
```

参数说明:

yarn.nodemanager.pmem-check-enabled

是否启动一个线程检查每个容器正使用的物理内存量, 如果任务超出分配值, 则直接将其杀掉, 默认是true。

yarn.nodemanager.vmem-check-enabled

是否启动一个线程检查每个任务正使用的虚拟内存量, 如果任务超出分配值, 则直接将其杀掉, 默认是 true

3) 修改 capacity-scheduler.xml

```
<property>
  <name>yarn.scheduler.capacity.resource-calculator</name>
  <!--
    <value>org.apache.hadoop.yarn.util.resource.DefaultResourceCalculator</value>
  -->
  <value>org.apache.hadoop.yarn.util.resource.DominantResourceCalculator</value>
</property>
```

DefaultResourceCalculator 进行资源调度时只考虑内存, 不适合 spark 的一个 Executor 可以用多核并行运行多个 task 的理念;

4) 重新分发到 yarn 中的各个节点:

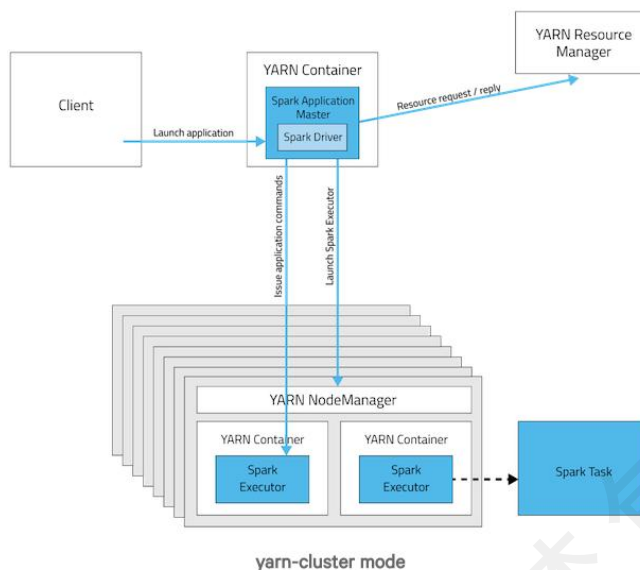
```
for i in 2 3; do scp capacity-scheduler.xml yarn-site.xml node-$i.51doit.cn:$PWD ; done
```

然后, 启动 hdfs 和 yarn 集群。注意: 要保证 yarn 集群的各个节点的时间是同步的。否则回报错

9.3.3 Yarn-client 与 yarn-cluster

9.3.3.1 通过 spark-submit 提交 cluster 模式

```
/bigdata/spark-2.3.3/bin/spark-submit --master yarn --deploy-mode cluster --executor-memory 1g
--executor-cores 2 --num-executors 3 --class cn._51doit.spark.day01.WordCount
/root/spark10-1.0-SNAPSHOT.jar hdfs://node-1.51doit.cn:9000/wc hdfs://node-1.51doit.cn:9000/out01
```

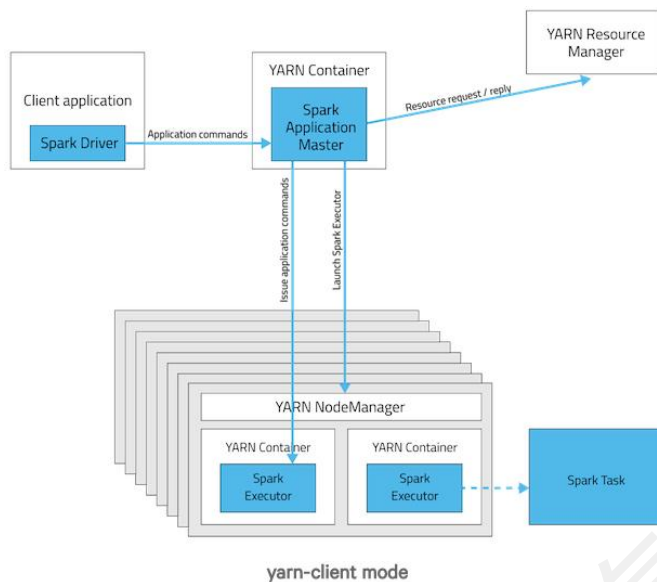


```
[root@doit01 ~]# jps
2018 ResourceManager
3139 CoarseGrainedExecutorBackend
3219 Jps
1686 DataNode
3015 SparkSubmit
1545 NameNode
2252 NodeManager
3119 CoarseGrainedExecutorBackend
[root@doit01 ~]#
```

```
[root@doit02 ~]# jps
2082 ApplicationMaster driver
2231 Jps
1528 SecondaryNameNode
2152 CoarseGrainedExecutorBackend
1609 NodeManager
1420 DataNode
[root@doit02 ~]#
```

9.3.3.2通过 spark-submit 提交为 client 模式

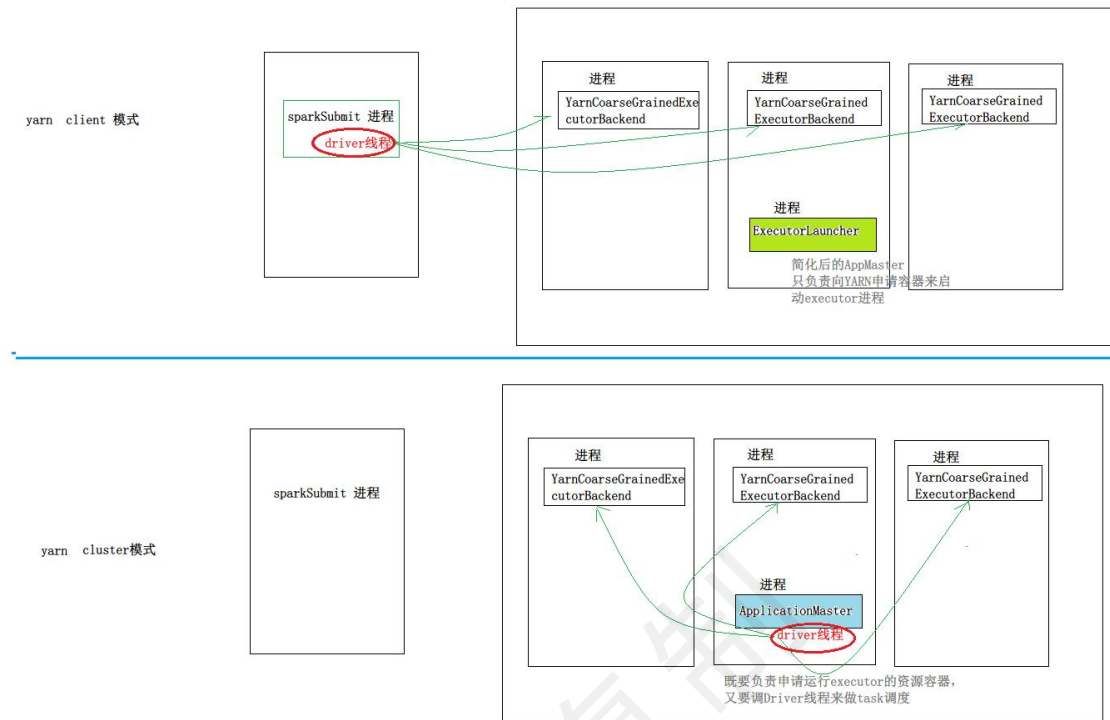
```
/bigdata/spark-2.3.3/bin/spark-submit --master yarn --deploy-mode client --executor-memory 1g  
--executor-cores 2 --num-executors 3 --class cn._51doit.spark.day01.WordCount  
/root/spark10-1.0-SNAPSHOT.jar hdfs://node-1.51doit.cn:9000/wc hdfs://node-1.51doit.cn:9000/out01
```



```
[root@doit01 ~]# jps  
2018 ResourceManager  
2611 SparkSubmit driver  
2851 Jps  
1686 DataNode  
1545 NameNode  
2761 CoarseGrainedExecutorBackend  
2762 CoarseGrainedExecutorBackend  
2252 NodeManager
```

```
[root@doit02 ~]# jps  
1777 ExecutorLauncher appMaster  
1528 SecondaryNameNode  
1928 Jps  
1609 NodeManager  
1420 DataNode  
1820 CoarseGrainedExecutorBackend executor  
1853 CoarseGrainedExecutorBackend
```

9.3.3.3 Yarn-client 模式和 yarn-cluster 模式对比



9.3.3.4 通过 IDEA 提交（测试用）

```
import org.apache.spark.rdd.RDD
import org.apache.spark.{SparkConf, SparkContext}
object SparkDemo {
  def main(args: Array[String]): Unit = {

    val conf: SparkConf = new SparkConf().setAppName("SparkDEMO")
      .setMaster("yarn")
      .set("spark.yarn.jars", "hdfs://doitedu01:8020/system/spark/yarn/jars/*")
      .setJars(List("/home/doitedu/SparkLearn.jar"))
    val sc: SparkContext = new SparkContext(conf)

    val dataRDD: RDD[Int] = sc.parallelize(Array(1,2,3,4))

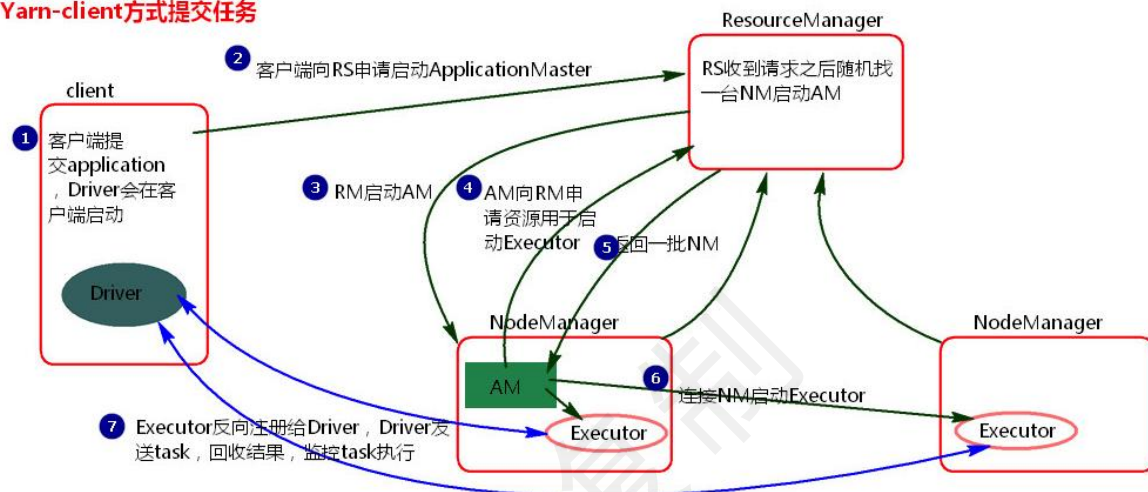
    val result = dataRDD.map(res => res * 2)
    result.collect().foreach(println(_))
  }
}
```

```
print(result.count())
}
}
```

9.3.4 on yarn 的启动执行流程

● client 模式

Yarn-client方式提交任务

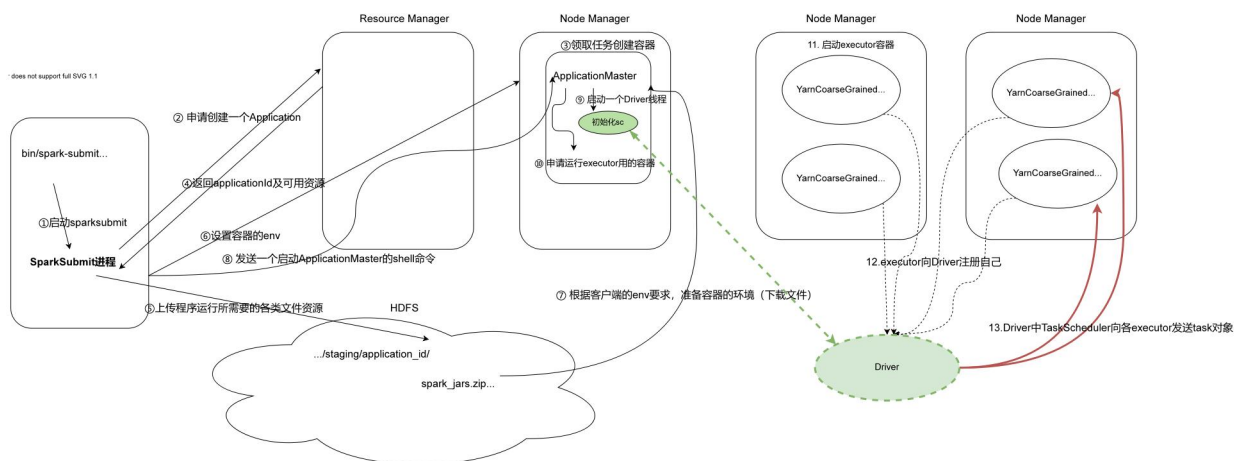


- 1) 客户端提交一个 Application, 在客户端启动一个 Driver 进程。
- 2) Driver 进程会向 ResourceManager 发送请求启动 ApplicationMaster 的资源。
- 3) ResourceManager 收到请求, 随机选择一台 NodeManager, 然后该 NodeManager 到 HDFS 下载 jar 包和配置, 接着启动 ApplicationMaster【ExecutorLuacher】。这里的 NodeManager 相当于 Standalone 中的 Worker 节点。
- 4) ApplicationMaster 启动后, 会向 ResourceManager 请求一批 container 资源, 用于启动 Executor。
- 5) ResourceManager 会找到一批符合条件 NodeManager 返回给 ApplicationMaster, 用于启动 Executor。
- 6) ApplicationMaster 会向 NodeManager 发送请求, NodeManager 到 HDFS 下载 jar 包和配置, 然后启动 Executor。
- 7) Executor 启动后, 会反向注册给 Driver, Driver 发送 task 到 Executor, 执行情况和结果返回给 Driver 端

Yarn-cluster 和 Yarn-client 区别就是 Yarn-cluster 的 driver 是在集群节点中随机选取启动, Yarn-client 的 driver 端是在任务提交的客户端中启动。

注意: 以后工作中提交 spark 任务一般都使用 cluster 模式, 但是 spark-shell、spark-sql 必须采用 client

● cluster 模式



9.3.5 Spark on yarn 启动流程源码解析

bin/spark-submit 脚本

启动 java 类: SparkSubmit(main)

sparksubmit 对象.doSubmit() // 判断你是要 submit, 还是 stauts, 还是 kill

sparksubmit 对象.submit()
--> doRunMain()

sparksubmit 对象.runMain() // 根据集群、部署模式, 来创建一个真正的应用提交器
--> 找到对应 yarn-cluster 模式的提交器: org.apache.spark.deploy.yarn.YarnClusterApplication
--> 用反射手段, 创建 YarnClusterApplication 的实例 (变量名 app)
--> 调用 app.start()

YarnClusterApplication.start()
--> new Client().run()

Client.run() 方法
--> this.appId = submitApplication() 提交应用

Client.submitApplication() 提交程序的核心流程

```
--> val newApp = yarnClient.createApplication() // 申请创建一个程序（申请了一个容器用来运行
ApplicationMaster）

--> val containerContext = createContainerLaunchContext(newAppResponse) // 为容器准备各种环境
参数（上传程序所需的文件资源）

--> val appContext = createApplicationSubmissionContext(newApp, containerContext) // 为
ApplicationMaster 的启动准备各种程序参数，启动命令

--> yarnClient.submitApplication(appContext) // 向 yarn 发送 applicationmaster 启动所需的所有信
息

ApplicationMaster.main() 启动进程
ExecutorLauncher .main() 启动进程
--> 调用 ApplicationMaster 的 main 方法

ApplicationMaster 的 main 方法
--> 调用 ApplicationMaster 的 run 方法

ApplicationMaster 的 run 方法
-- 判断 deploy-mode 是 cluster 还是 client
--> 如果是 cluster，则运行 runDriver() // 会启动 Driver 线程（调用户自己类的 main 方法初始化 sc）
// 然后申请 executor 资源容器
--> 如果是 client，则运行 runExecutorLauncher // 只做 executor 资源容器的申请启动行为
```

JUST DO IT
多易教育

10 Job 调度全流程(task 调度)

10.1 构造并初始化 SparkContext

在 Driver 线程中，run 方法调用的就是 UserClass(用户主类) 的 main 方法

而用户主类的 main 方法中，首先会初始化 SparkContext

初始化 SparkContext 的过程中，会初始化两个重要成员：DagScheduler 和 TaskScheduler

还会创建于 executor 通信用的 backend！

创建之后，就等待 executor 来注册！

10.2 创建 RDD 构建 DAG

原始的 RDD 通过一系列的转换就形成了 DAG，根据 RDD 之间的依赖关系的不同将 DAG 划分成不同的 Stage，对于窄依赖，partition 的转换处理在同一个 Stage 中完成计算。对于宽依赖，由于有 Shuffle 的存在，只能在 parent stage 处理完成后，才能开始接下来的计算，因此宽依赖是划分 Stage 的依据。

10.3 触发行动算子

job 及 task 的调度自此开始

sc.runJob()

10.4 切分 Stage，生成 Task 和 TaskSet

DAGScheduler 首先根据调用行动算子的 RDD (Final RDD)，调用 createResultStage 方法来生成 ResultStage

```
Create a ResultStage associated with the provided jobid.
private def createResultStage(
  rdd: RDD[_],
  func: (TaskContext, Iterator[_]) => _,
  partitions: Array[Int],
  jobId: Int,
  callSite: CallSite): ResultStage = {
  checkBarrierStageWithDynamicAllocation(rdd)
  checkBarrierStageWithNumSlots(rdd)
  checkBarrierStageWithRDDChainPattern(rdd, partitions.toSet.size)
  val parents = getOrCreateParentStages(rdd, jobId)
  val id = nextStageId.getAndIncrement()
  val stage = new ResultStage(id, rdd, func, partitions, parents, jobId, callSite)
  stageIdToStage(id) = stage
  updateJobIdStageIdMaps(jobId, stage)
  stage
}
```

每个 Stage 对象中, 最关键的属性是: 这个 Stage 最后一个 RDD (每个 RDD 都持有父 RDD 的引用)!

将来根据 Stage 生成的 Task, 就是用这个 final rdd 的迭代器来调用执行整个迭代器链!

10.5 提交 stage

首先, DAGScheduler 会将上一步 (划分 stage) 得到 finalStage, 拿去提交 task; 提交逻辑会检查当前要提交的 stage 是否还有未提交的父 stage, 如果有, 就得先提交父 stage!

(典型的递归算法)

```
private def submitStage(stage: Stage) {
  val jobId = activeJobForStage(stage)
  if (jobId.isDefined) {
    logDebug( msg = s"submitStage($stage (name=${stage.name}); " +
      s"jobs=${stage.jobIds.toSeq.sorted.mkString(",")})")
    if (!waitingStages(stage) && !runningStages(stage) && !failedStages(stage)) {
      val missing = getMissingParentStages(stage).sortBy(_.id)
      logDebug( msg = "missing: " + missing)
      if (missing.isEmpty) {
        logInfo( msg = "Submitting " + stage + " (" + stage.rdd + "), which has no missing parents")
        submitMissingTasks(stage, jobId.get)
      } else {
        for (parent <- missing) {
          submitStage(parent)
        }
        waitingStages += stage
      }
    }
  }
} else {
  abortStage(stage, reason = "No active job for stage " + stage.id, None)
}
```

提交task

foreach() 行动算子

```
-> sc.runJob()
-> dagScheduler.runJob()
```

dagScheduler 的 runJob()

```
-> submitJob()
```

dagScheduler 的 submitJob()

```
-> eventProcessLoop.post(JobSubmitted)
```

DAGSchedulerEventProcessLoop 收到消息

```
->doOnReceive()
```

DAGSchedulerEventProcessLoop 的 doOnReceive()

```
-> dagScheduler.handleJobSubmitted
```

DagScheduler 的 handleJobSubmitted 方法

```
-> finalStage = createResultStage(finalRDD, func, partitions, jobId, callSite) // 根据 finalRDD  
创建 finalStage
```

```
-> submitStage(finalStage) // 递归提交, 永远是从前面的 stage 开始提交
```

submitStage()

```
-> submitMissingTasks(stage, jobId.get)
```

submitMissingTasks 方法

```
-> 把 stage 的 finalRDD 信息序列化后广播给每个 executor
```

```
-> 根据要提交的 stage 的类型, 以及需要计算的分区个数, 生成相应类型相应个数的 Task 对象
```

```
-> taskScheduler.submitTasks(new TaskSet(tasks.toArray, stage.id, jobId...))
```

10.6 TaskScheduler 提交 TaskSet

TaskSchedulerImpl.submitTasks

```
-> 将 task 对象序列化, 发给 executor
```

```
-> task 发给 executor, 也是有调度策略的: FIFO
```

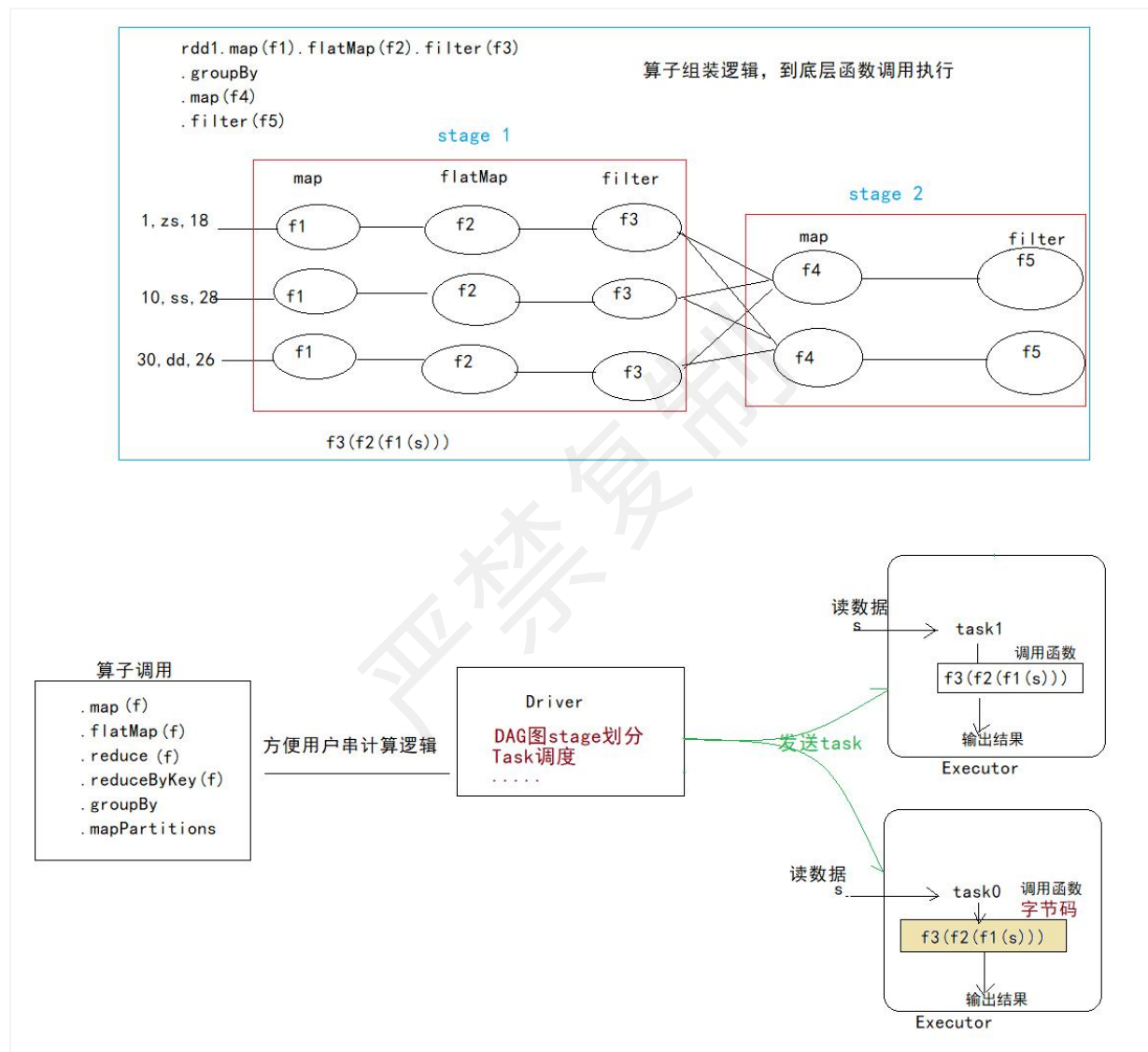
10.7 在 Executor 中执行 Task

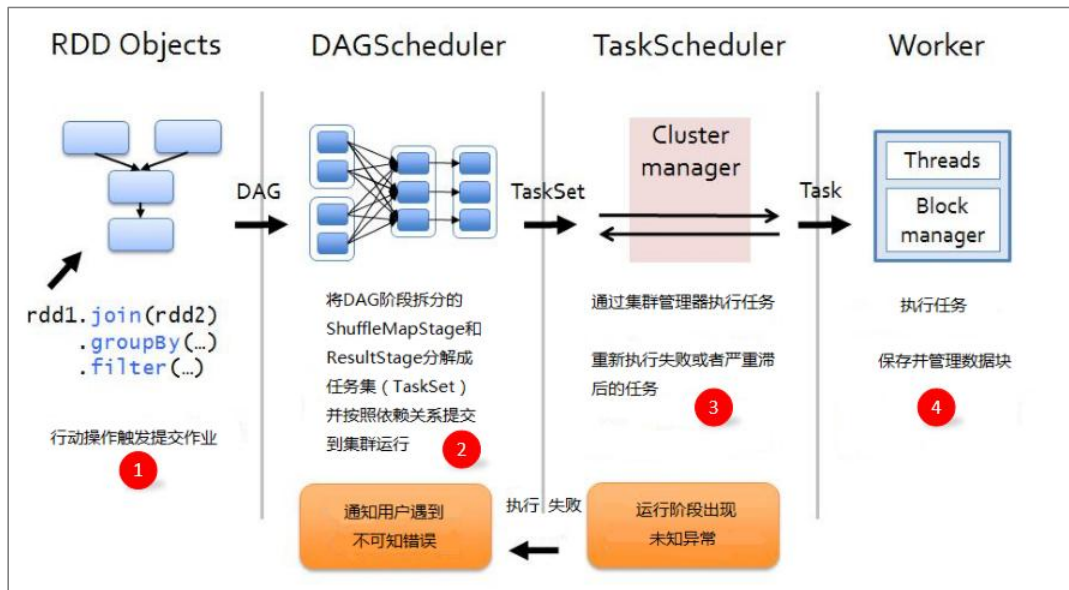
Executor 收到 Task 对象并反序列化后, 会将 Task 包装成一个 TaskRunner 类以便放入线程池执行; 而线程执行时, 调用的就是 Task 的 runTask 方法, 而 runTask 方法中, 拿到这个 task 的 rdd 的迭代器, 然后将迭代器传入一个 ShuffleWriter.write(records)!

而 shuffleWriter.write 方法中, 就是开始“迭代”这个迭代器

```
3 | }  
  | while (records.hasNext) {  
  |   addElementsRead()  
  |   kv = records.next()  
  |   map.changeValue((getPartition(kv._1), kv._1), update)  
  |   maybeSpillCollection(usingMap = true)  
  | }
```

10.8 job 执行全流程关键步骤总结





`foreach()` 行动算子

```
-> sc.runJob()
-> dagScheduler.runJob()
```

`dagScheduler` 的 `runJob()`

```
-> submitJob()
```

`dagScheduler` 的 `submitJob()`

```
-> eventProcessLoop.post(JobSubmitted)
```

`DAGSchedulerEventProcessLoop` 收到消息

```
-> doOnReceive()
```

`DAGSchedulerEventProcessLoop` 的 `doOnReceive()`

```
-> dagScheduler.handleJobSubmitted
```

`DagScheduler` 的 `handleJobSubmitted` 方法

```
-> finalStage = createResultStage(finalRDD, func, partitions, jobId, callSite) // 根据 finalRDD 创建 finalStage
-> submitStage(finalStage) // 递归提交, 永远是先从前面的 stage 开始提交
```

```
submitStage()
```

```
-> submitMissingTasks(stage, jobId.get)
```

submitMissingTasks 方法

```
-> 把 stage 的 finalRDD 信息序列化后广播给每个 executor
```

```
-> 根据要提交的 stage 的类型，以及需要计算的分区个数，生成相应类型相应个数的 Task 对象
```

```
-> taskScheduler.submitTasks(new TaskSet(tasks.toArray, stage.id, jobId...))
```

TaskSchedulerImpl.submitTasks

```
-> 将 task 对象序列化，发给 executor
```

```
-> task 发给 executor，也是有调度策略的：FIFO
```

Executor 收到 task 对象，就反序列化

并将反序列化出来的 task 对象封装到一个 TaskRunner 对象中

然后把这个 TaskRunner 对象放入线程池执行

TaskRunner 的 run 方法，就是调 task 对象的 runTask 方法

task 对象的 runTask 方法就要看是哪种 task 了 (shuffleMapTask, resultTask)

shuffleMapTask 的 runTask 方法中，调 ShuffleWriter.write(finalRDD.iterator)

ShuffleWriter.write(iterator) 具体实现，要看是哪一种 ShuffleWriter 实现类

```
while(iterator.hasNext())
```

```
    kv = iterator.next
```

```
// 不同实现类的不同之处就在后续将 kv 放到哪里去缓存，以及缓存满了以后怎么溢出
```

11 Shuffle Writer 机制详解

~~11.1~~ spark shuffle 演进的历史（了解）

Spark 0.8 及以前 Hash Based Shuffle

Spark 0.8.1 为 Hash Based Shuffle 引入 File Consolidation 机制

Spark 0.9 引入 ExternalAppendOnlyMap

Spark 1.1 引入 Sort Based Shuffle，但默认仍为 Hash Based Shuffle

Spark 1.2 默认的 Shuffle 方式改为 Sort Based Shuffle

Spark 1.4 引入 Tungsten-Sort Based Shuffle

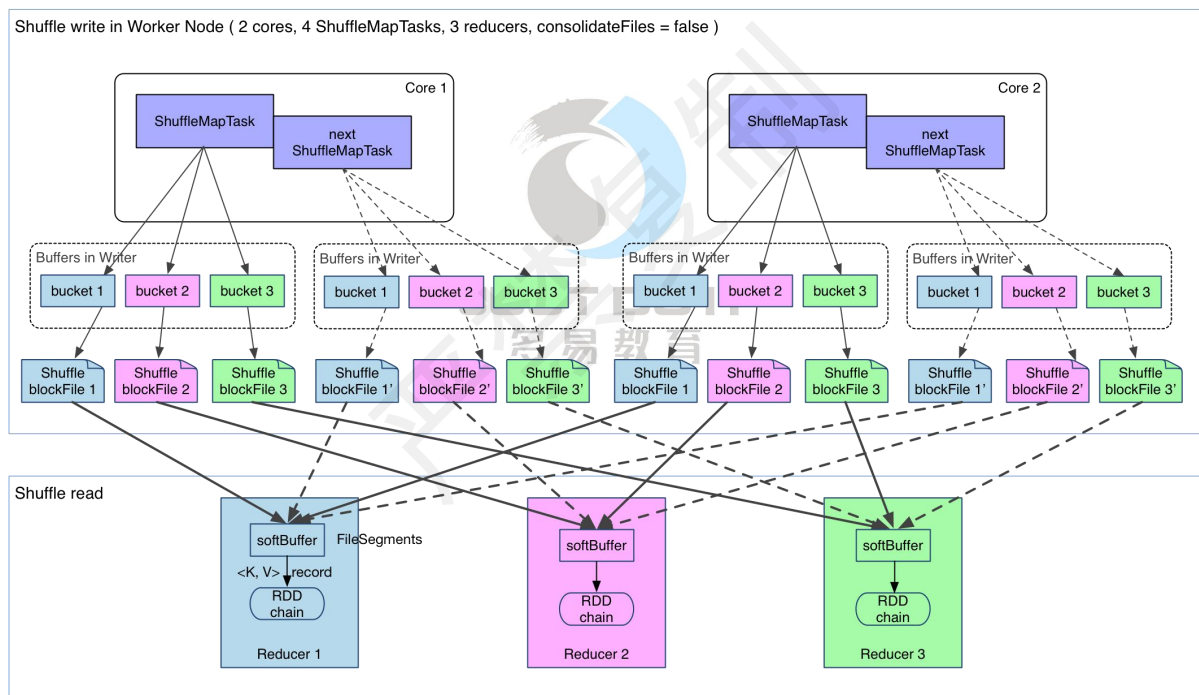
Spark 1.6 Tungsten-sort 并入 Sort Based Shuffle

Spark 2.0 Hash Based Shuffle 退出历史舞台

11.2 HashShuffleManager (已弃用)

● 优化前的

在 shuffle write 前，应用分区器，根据对应的分区规则，计算出数据 partition 编号，然后将数据写入 bucket 内存中，当数据达到一定大小或数据全部处理完后，将数据溢写持平持久化。之所以要持久化，一方面是要减少内存存储空间压力，另一方面也是为了容错降低数据恢复的代价。



上图有 2 个 Executor，每个 Executor 有 1 个 core，total-executor-cores 数为 2，每个 task 的执行结果会被溢写到本地磁盘上。每个 task 包含 R 个缓冲区，R = reducer 个数（也就是下一个 stage 中 task 的个数），缓冲区被称为 bucket，其大小为 spark.shuffle.file.buffer.kb，默认是 32KB。

其实 bucket 表示缓冲区，即 ShuffleMapTask 调用分区器的后数据要存放的地方。

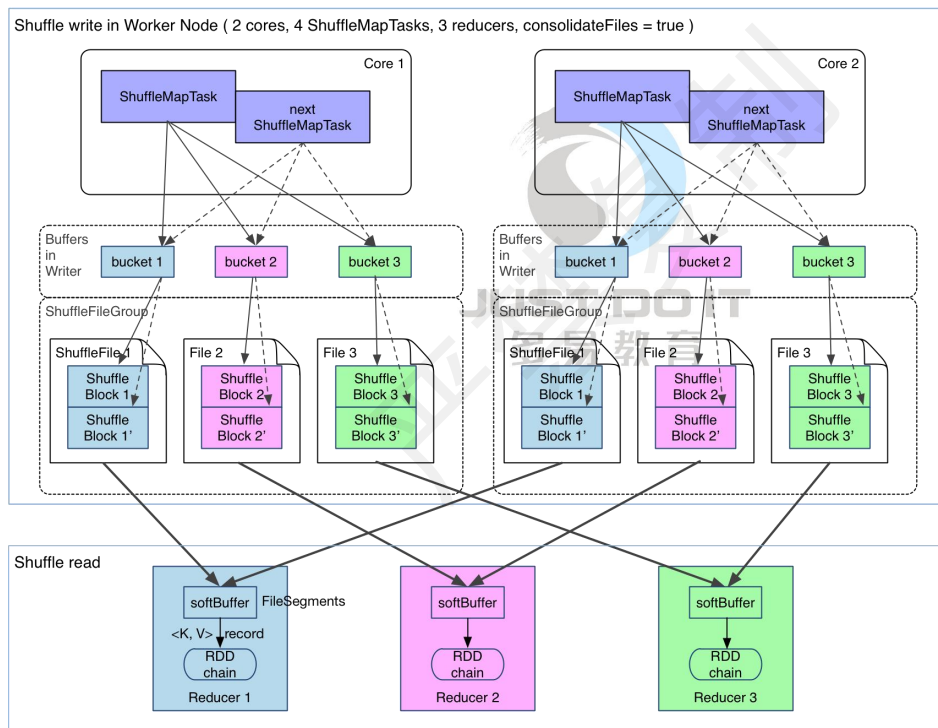
ShuffleMapTask 的执行过程：先根据 pipeline 的计算逻辑对数据进行运算，然后根据分区器计

算出每一个 record 的分区编号。每得到一个 record 就将其送到对应的 bucket 里，具体是哪个 bucket 由 `partitioner.getPartition(record.getKey())` 决定。每个 bucket 里面的数据会满足溢写的条件会被溢写到本地磁盘上，形成一个 `ShuffleBlockFile`，或者简称 `FileSegment`。之后的下游的 task 会根据分区会去 fetch 属于自己的 `FileSegment`，进入 shuffle read 阶段。

老版本的 `HashShuffleManager` 存在的问题：

1. 产生的 `FileSegment` 过多。每个 `ShuffleMapTask` 产生 R (下游 Task 的数量) 个 `FileSegment`， M 个 `ShuffleMapTask` 就会产生 $M * R$ 个文件。一般 Spark job 的 M 和 R 都很大，因此磁盘上会存在大量的数据文件。
2. 缓冲区占用内存空间大。每个 `ShuffleMapTask` 需要开 R 个 bucket， M 个 `ShuffleMapTask` 就会产生 $M * R$ 个 bucket。虽然一个 `ShuffleMapTask` 结束后，对应的缓冲区可以被回收，但一个 worker node 上同时存在的 bucket 个数可以达到 $cores * R$ 个，占用的内存空间也就达到了 $cores * R * 32 \text{ KB}$ 。对于 8 核 1000 个 reducer 来说，占用内存就是 256MB。

● 优化后的：



可以明显看出，在一个 core 上连续执行的 `ShuffleMapTasks` 可以共用一个输出文件 `ShuffleFile`。先执行完的 `ShuffleMapTask` 形成 `ShuffleBlock i`，后执行的 `ShuffleMapTask` 可以将输出数据直接追加到 `ShuffleBlock i` 后面，形成 `ShuffleBlock j`，每个 `ShuffleBlock` 被称为 `FileSegment`。下一个 stage 的 reducer 只需要 fetch 整个 `ShuffleFile` 就行了。这样每个 Executor 持有的文件数降为 $cores * R$ 。

`FileConsolidation` 功能可以通过 `spark.shuffle.consolidateFiles=true` 来开启。

11.3 SortShuffleManager

11.3.1 三种 shuffle 方式

- BypassMergeSortShuffleHandle
- SerializedShuffleHandle
- BaseShuffleHandle

三种 shuffleHandle 方式对应了 3 中 shuffleWriter

```
override def getWriter[K, V](
  handle: ShuffleHandle,
  mapId: Long,
  context: TaskContext,
  metrics: ShuffleWriteMetricsReporter): ShuffleWriter[K, V] = {
  val mapTaskIds = taskIdMapsForShuffle.computeIfAbsent(
    handle.shuffleId, _ => new OpenHashSet[Long](initialCapacity = 16))
  mapTaskIds.synchronized { mapTaskIds.add(context.taskAttemptId()) }
  val env = SparkEnv.get
  handle match {
    case unsafeShuffleHandle: SerializedShuffleHandle[K @unchecked, V @unchecked] =>
      new UnsafeShuffleWriter(
        env.blockManager,
        context.taskMemoryManager(),
        unsafeShuffleHandle,
        mapId,
        context,
        env.conf,
        metrics,
        shuffleExecutorComponents)
    case bypassMergeSortHandle: BypassMergeSortShuffleHandle[K @unchecked, V @unchecked] =>
      new BypassMergeSortShuffleWriter(
        env.blockManager,
        bypassMergeSortHandle,
        mapId,
        env.conf,
        metrics,
        shuffleExecutorComponents)
    case other: BaseShuffleHandle[K @unchecked, V @unchecked, _] =>
      new SortShuffleWriter(
        shuffleBlockResolver, other, mapId, context, shuffleExecutorComponents)
  }
}
```

11.3.2 Shuffle 方式的决定机制

Shuffle 方式，是在什么时候决定的

```
val rdd5: RDD[(String, Int)] = rdd1
  .flatMap(s => s.split( regex = "\\s+"))
  .map(s => (s, 1))
  .reduceByKey(_ + _)
```

```
def combineByKeyWithClassTag[C](
  createCombiner: V => C,
  mergeValue: (C, V) => C,
  mergeCombiners: (C, C) => C,
  partitioner: Partitioner,
  mapSideCombine: Boolean = true,
  serializer: Serializer = null)(implicit ct: ClassTag[C]): RDD[(K, C)] = self.withScope {
  require(mergeCombiners != null, "mergeCombiners must be defined") // required as of Spark 0.9.0
  if (keyClass.isArray) {...}
  val aggregator = new Aggregator[K, V, C](...)
  if (self.partitioner == Some(partitioner)) {...} else {
    new ShuffledRDD[K, V, C](self, partitioner)
      .setSerializer(serializer)
      .setAggregator(aggregator)
      .setMapSideCombine(mapSideCombine)
  }
}
```

```
override def getDependencies: Seq[Dependency[_]] = {
  val serializer = userSpecifiedSerializer.getOrElse {
    val serializerManager = SparkEnv.get.serializerManager
    if (mapSideCombine) {
      serializerManager.getSerializer(implicitly[ClassTag[K]], implicitly[ClassTag[C]])
    } else {
      serializerManager.getSerializer(implicitly[ClassTag[K]], implicitly[ClassTag[V]])
    }
  }
  List(new ShuffleDependency(prev, part, serializer, keyOrdering, aggregator, mapSideCombine))
}
```

new ShuffleDependency 的过程中，就会注册 shuffleHandle，如下：

```
class ShuffleDependency[K: ClassTag, V: ClassTag, C: ClassTag](
  @transient private val _rdd: RDD[_ <: Product2[K, V]],
  val partitioner: Partitioner,
  val serializer: Serializer = SparkEnv.get.serializer,
  val keyOrdering: Option[Ordering[K]] = None,
  val aggregator: Option[Aggregator[K, V, C]] = None,
  val mapSideCombine: Boolean = false)
  extends Dependency[Product2[K, V]] {

  if (mapSideCombine) {...}
  override def rdd: RDD[Product2[K, V]] = _rdd.asInstanceOf[RDD[Product2[K, V]]]

  private[spark] val keyClassName: String = reflect.classTag[K].runtimeClass.getName
  private[spark] val valueClassName: String = reflect.classTag[V].runtimeClass.getName
  /.../
  private[spark] val combinerClassName: Option[String] = ...

  val shuffleId: Int = _rdd.context.newShuffleId()

  val shuffleHandle: ShuffleHandle = _rdd.context.env.shuffleManager.registerShuffle(
    shuffleId, _rdd.partitions.length, dependency = this)

  _rdd.sparkContext.cleaner.foreach(_.registerShuffleForCleanup(shuffleDependency = this))
}
```

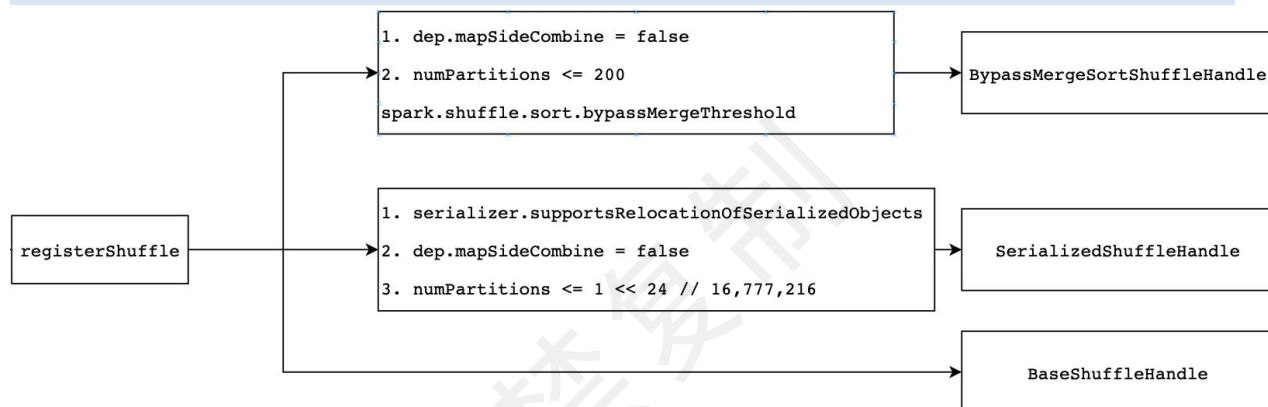
registerShuffle 的源码实现如下：

```

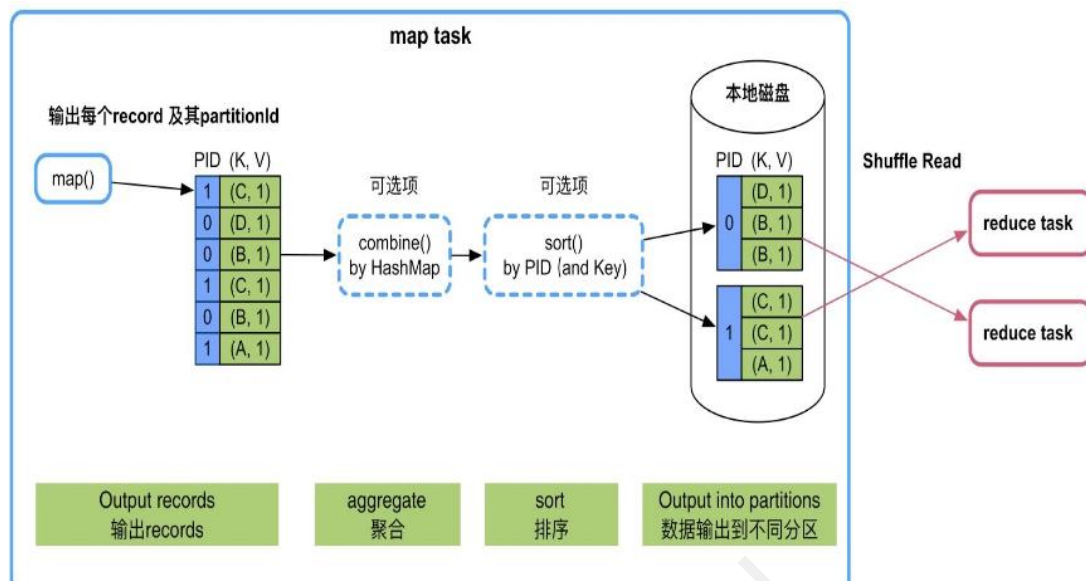
override def registerShuffle[K, V, C](
  shuffleId: Int,
  numMaps: Int,
  dependency: ShuffleDependency[K, V, C]): ShuffleHandle = {
  if (SortShuffleWriter.shouldBypassMergeSort(conf, dependency)) {
    // If there are fewer than spark.shuffle.sort.bypassMergeThreshold partitions and we don't
    // need map-side aggregation, then write numPartitions files directly and just concatenate
    // them at the end. This avoids doing serialization and deserialization twice to merge
    // together the spilled files, which would happen with the normal code path. The downside is
    // having multiple files open at a time and thus more memory allocated to buffers.
    1 new BypassMergeSortShuffleHandle[K, V](
      shuffleId, numMaps, dependency.asInstanceOf[ShuffleDependency[K, V, V]])
  } else if (SortShuffleManager.canUseSerializedShuffle(dependency)) {
    // Otherwise, try to buffer map outputs in a serialized form, since this is more efficient:
    2 new SerializedShuffleHandle[K, V](
      shuffleId, numMaps, dependency.asInstanceOf[ShuffleDependency[K, V, V]])
  } else {
    // Otherwise, buffer map outputs in a deserialized form:
    3 new BaseShuffleHandle(shuffleId, numMaps, dependency)
  }
}

```

ShuffleHandle 方式选择的条件判断直观图示:



11.4 Shuffle Write 整体框架



11.5 BaseShuffleHandle 运行机制

在该模式下，数据会先写入一个内存数据结构中，此时根据不同的 shuffle 算子，可能选用不同的数据结构。如果是 `reduceByKey` 这种聚合类的 shuffle 算子，那么会选用 Map 数据结构，一边通过 Map 进行聚合，一边写入内存；如果是 `join` 这种普通的 shuffle 算子，那么会选用 Array 数据结构，直接写入内存。接着，每写一条数据进入内存数据结构之后，就会判断一下，是否达到了某个临界阈值。如果达到临界阈值的话，那么就会尝试将内存数据结构中的数据溢写到磁盘，然后清空内存数据结构。

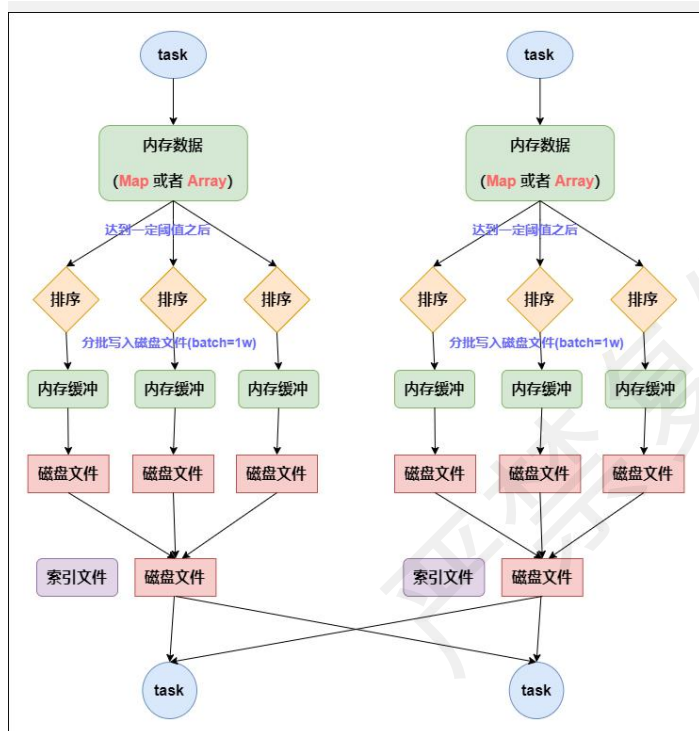
在溢写到磁盘文件之前，会先根据 key 对内存数据结构中已有的数据进行排序。排序过后，会分批将数据写入磁盘文件。默认的 batch 数量是 10000 条，也就是说，排序好的数据，会以每批 1 万条数据的形式分批写入磁盘文件。写入磁盘文件是通过 Java 的 `BufferedOutputStream` 实现的。`BufferedOutputStream` 是 Java 的缓冲输出流，首先会将数据缓冲在内存中，当内存缓冲满溢之后再一次写入磁盘文件中，这样可以减少磁盘 IO 次数，提升性能。

一个 task 将所有数据写入内存数据结构的过程中，会发生多次磁盘溢写操作，也就产生多个临时文件。最后会将之前所有的临时磁盘文件都进行合并，这就是 merge 过程，此时会将之前所有临时

磁盘文件中的数据读取出来，然后依次写入最终的磁盘文件之中。此外，由于一个 task 就只对应一个磁盘文件，也就意味着该 task 为下游 stage 的 task 准备的数据都在这一个文件中，因此还会单独写一份索引文件，其中标识了下游各个 task 的数据在文件中的 start offset 与 end offset。

SortShuffleManager 由于有一个磁盘文件 merge 的过程，因此大大减少了文件数量。比如第一个 stage 有 50 个 task，总共有 10 个 Executor，每个 Executor 执行 5 个 task，而第二个 stage 有 100 个 task。由于每个 task 最终只有一个磁盘文件，因此此时每个 Executor 上只有 5 个磁盘文件，所有 Executor 只有 50 个磁盘文件。

普通运行机制的 SortShuffleManager 工作原理如下图所示：



```
// Data structures to store in-memory objects before we spill. Depending on whether we have an
// Aggregator set, we either put objects into an AppendOnlyMap where we combine them, or we
// store them in an array buffer.
```

```
@volatile private var map = new PartitionedAppendOnlyMap[K, C]
@volatile private var buffer = new PartitionedPairBuffer[K, C]
```

map 或者 buffer 填满了，是否要 spill，跟 MR 完全不同；它会先尝试申请更多的内存空间，尽量不 spill；只有在申请不到更多内存，且已经填充的数据超过阈值比例，才 spill；

```
protected def maybeSpill(collection: C, currentMemory: Long): Boolean = {
  var shouldSpill = false
  if (elementsRead % 32 == 0 && currentMemory >= myMemoryThreshold) {
    // Claim up to double our current memory from the shuffle memory pool
    val amountToRequest = 2 * currentMemory - myMemoryThreshold
    val granted = acquireMemory(amountToRequest) // 申请更多内存
    myMemoryThreshold += granted // 判断阈值更新为：原来阈值+申请到的内存数
    // If we were granted too little memory to grow further (either tryToAcquire returned 0,
    // or we already had more memory than myMemoryThreshold), spill the current collection
    shouldSpill = currentMemory >= myMemoryThreshold // 用掉的内存是否大于新的上限阈值
  }
}
```

11.6 ByPassMergeShuffle 运行机制

Reducer 端任务数比较少的情况下, 基于 Hash Shuffle 实现机制明显比基于 Sort Shuffle 实现机制要快, 因此基于 Sort Shuffle 实现机制提供了一个带 Hash 风格的回退方案, 就是 bypass 运行机制。

对于 Reducer 端任务数少于配置属性 `spark.shuffle.sort.bypassMergeThreshold` 设置的个数时, 使用带 Hash 风格的回退计划。

bypass 运行机制的触发条件如下:

- shuffle map task 数量小于 `spark.shuffle.sort.bypassMergeThreshold=200` 参数的值
- 不是聚合类的 shuffle 算子, 没有对 key 排序的需求

此时, 每个 task 会为每个下游 task 都创建一个临时磁盘文件, 并将数据按 key 进行 hash 然后根据 key 的 hash 值, 将 key 写入对应的磁盘文件之中。当然, 写入磁盘文件时也是先写入内存缓冲, 缓冲写满之后再溢写到磁盘文件的。最后, 同样会将所有临时磁盘文件都合并成一个磁盘文件, 并创建一个单独的索引文件。

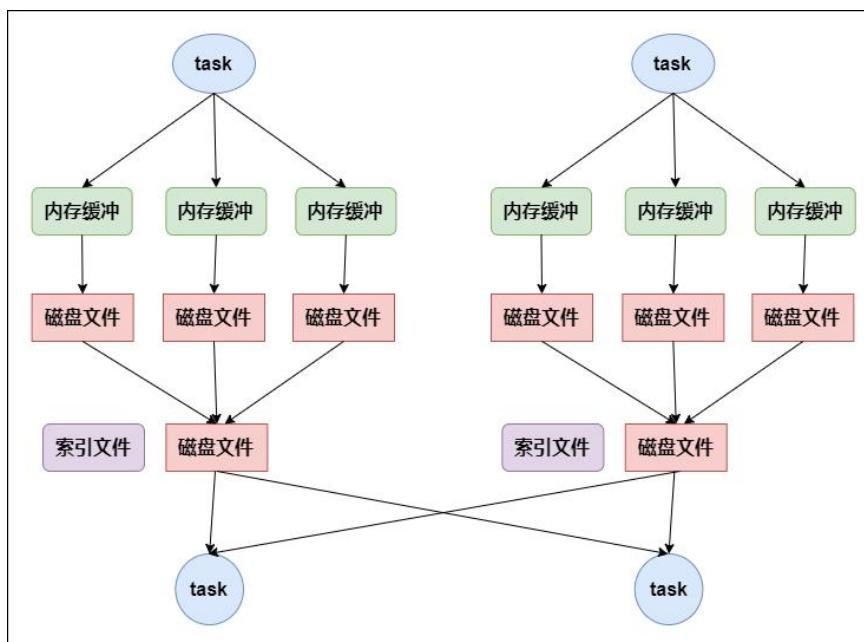
该过程的磁盘写机制其实跟未经优化的 HashShuffleManager 是一模一样的, 因为都要创建数量惊人的磁盘文件, 只是在最后会做一个磁盘文件的合并而已。因此少量的最终磁盘文件, 也让该机制相对未经优化的 HashShuffleManager 来说, shuffle read 的性能会更好。

而该机制与普通 BaseShuffleHandle 运行机制的不同在于:

第一, 磁盘写机制不同;

第二, 不会进行排序。

也就是说, 启用该机制的最大好处在于, shuffle write 过程中, 不需要进行数据的排序操作, 也就省掉了这部分的性能开销。



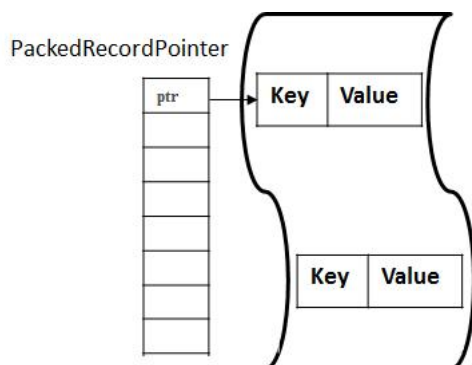
byPassMergeSortShuffle 是没有专门的 shuffle 缓存数据结构的；

上图中的“内存缓冲”其实是写文件时的 api（BufferedWriter）的底层小缓冲；

11.7 Tungsten Sort Shuffle 运行机制

UnsafeShuffleWriter（SerializedShuffleHandle）是对普通 sort 的一种优化，排序的不是内容本身，而是内容序列化后字节数组的指针(元数据)，把数据的排序转变为了指针数组的排序，实现了直接对序列化后的二进制数据进行排序。由于直接基于二进制数据进行操作，所以在这里面没有序列化和反序列化的过程。内存的消耗降低，相应的也会减少 gc 的开销。

这些优化的实现导致引入了一个新的内存管理模型，即 Page，Page 其实也是一个 MemoryBlock，不过因为它是引入了指针 Pointer 的特性，所以又支持 off-heap 以及 on-heap 两种模式，可以直接访问系统的内存。处理模型如下图所示：



采用了用 8 位 byte 的数组指针，这个数组指针的前 24 位存储着数据记录的 PartitionId，中间 13 位存储着内存页的编号，其实也是内存块的编号，最后 24 位存储着数据记录在这个内存页中的地址偏移量。

这种方式是利用 java 中的 unsafe API 完成的；这个 unsafe 类中就封装了一些像 C/C++ 中的指针操作，可以直接访问系统的内存。既然能够像指针一样直接访问内存，那就会存在一定的风险，比如给指针赋值或者指针指向的地址加上偏移量后可能会指向系统程序的地址，从而覆盖原来地址中的内容，有可能会直接导致系统的崩溃，所以说指针操作是不安全的，因此给它取名为 UnsafeShuffleWriter。

要实现 Tungsten Sort Shuffle 机制需要满足以下条件：

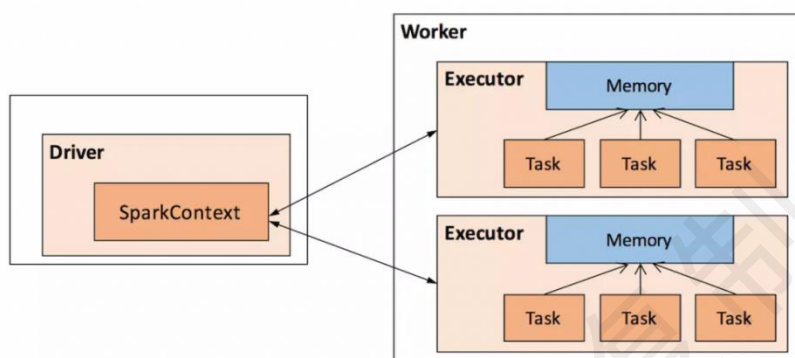
- Shuffle 的序列化器支持序列化后的对象的重定位（当前仅支持 KryoSerializer、Spark SQL 框架自定义的序列化器）。
- Shuffle 依赖中不带聚合操作或没有对输出进行排序的要求。
- Shuffle 过程中的输出分区个数少于 2^{24} （16777216）个。

JUST DO IT
多易教育

12 Spark 内存管理机制

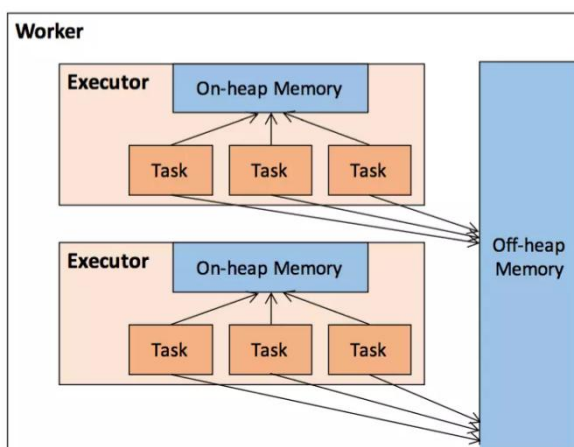
12.1 概述

在执行 Spark 的应用程序时，Spark 集群会启动 Driver 和 Executor 两种 JVM 进程，前者为主控进程，负责创建 Spark 上下文，提交 Spark 作业（Job），并将作业转化为计算任务（Task），在各个 Executor 进程间协调任务的调度，后者负责在工作节点上执行具体的计算任务，并将结果返回给 Driver，同时为需要持久化的 RDD 提供存储功能。由于 Driver 的内存管理相对来说较为简单，本章节要对 Executor 的内存管理进行分析，下文中的 Spark 内存均特指 Executor 的内存。



12.2 堆内内存和堆外内存

作为一个 JVM 进程，Executor 的内存管理建立在 JVM 的内存管理之上，Spark 对 JVM 的堆内（On-heap）空间进行了更为详细的分配，以充分利用内存。同时，Spark 引入了堆外（Off-heap）内存，使之可以直接在工作节点的系统内存中开辟空间，进一步优化了内存的使用。



12.3 堆内内存

堆内内存的大小，由 Spark 应用程序启动时的 `-executor-memory` 或 `spark.executor.memory` 参数配置。Executor 内运行的并发任务共享 JVM 堆内内存，这些任务在缓存 RDD 和广播（Broadcast）数据时占用的内存被规划为存储（Storage）内存，而这些任务在执行 Shuffle 时占用的内存被规划为执行（Execution）内存，剩余的部分不做特殊规划，那些 Spark 内部的对象实例，或者用户定义的 Spark 应用程序中的对象实例，均占用剩余的空间。不同的管理模式，这三部分占用的空间大小各不相同（下面第 2 小节介绍）。

12.3.1 堆内内存的申请与释放

Spark 对堆内内存的管理是一种逻辑上的“规划式”的管理，因为对象实例占用内存的申请和释放都由 JVM 完成，Spark 只能在申请后和释放前记录这些内存：

- 申请内存

Spark 在代码中 new 一个对象实例

JVM 从堆内内存分配空间，创建对象并返回对象引用

Spark 保存该对象的引用，记录该对象占用的内存

- 释放内存

Spark 记录该对象释放的内存，删除该对象的引用

等待 JVM 的垃圾回收机制释放该对象占用的堆内内存

- 堆内内存优缺点分析

堆内内存采用 JVM 来进行管理。而 JVM 的对象可以以序列化的方式存储，序列化的过程是将对象转换为二进制字节流，本质上可以理解为将非连续空间的链式存储转化为连续空间或块存储，在访问时则需要进行序列化的逆过程——反序列化，将字节流转化为对象，序列化的方式可以节省存储空间，但增加了存储和读取时候的计算开销。

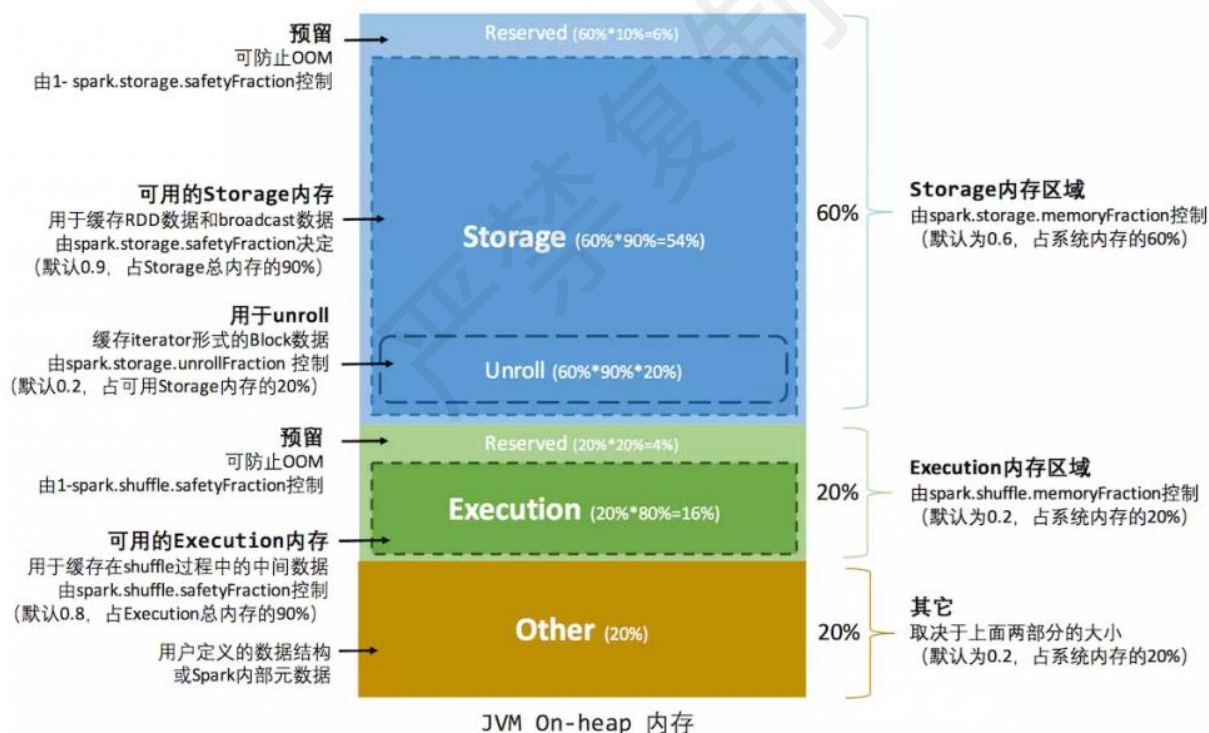
对于 Spark 中序列化的对象，由于是字节流的形式，其占用的内存大小可直接计算。

对于 Spark 中非序列化的对象，其占用的内存是通过周期性地采样近似估算而得，即并不是每次新增的数据项都会计算一次占用的内存大小。这种方法：

- 降低了时间开销但是有可能误差较大，导致某一时刻的实际内存有可能远远超出预期；此外，在被 Spark 标记为释放的对象实例，很有可能在实际上并没有被 JVM 回收，导致实际可用的内存小于 Spark 记录的可用内存。所以 Spark 并不能准确记录实际可用的堆内内存，从而也就无法完全避免内存溢出（OOM, Out of Memory）的异常。
- 虽然不能精准控制堆内内存的申请和释放，但 Spark 通过对存储内存和执行内存各自独立的规划管理，可以决定是否要在存储内存里缓存新的 RDD，以及是否为新的任务分配执行内存，在一定程度上可以提升内存的利用率，减少异常的出现。

12.3.2 堆内内存分区(静态方式,弃)

在静态内存管理机制下，存储内存、执行内存和其他内存三部分的大小在 Spark 应用程序运行期间是固定的，但用户可以在应用程序启动前进行配置，堆内内存的分配如图所示：



可以看到，可用的堆内内存的大小需要按照下面的方式计算：

```
可用的存储内存 = systemMaxMemory * spark.storage.memoryFraction * spark.storage.safetyFraction  
可用的执行内存 = systemMaxMemory * spark.shuffle.memoryFraction * spark.shuffle.safetyFraction
```

其中 `systemMaxMemory` 取决于当前 JVM 堆内内存的大小，最后可用的执行内存或者存储内存要在此

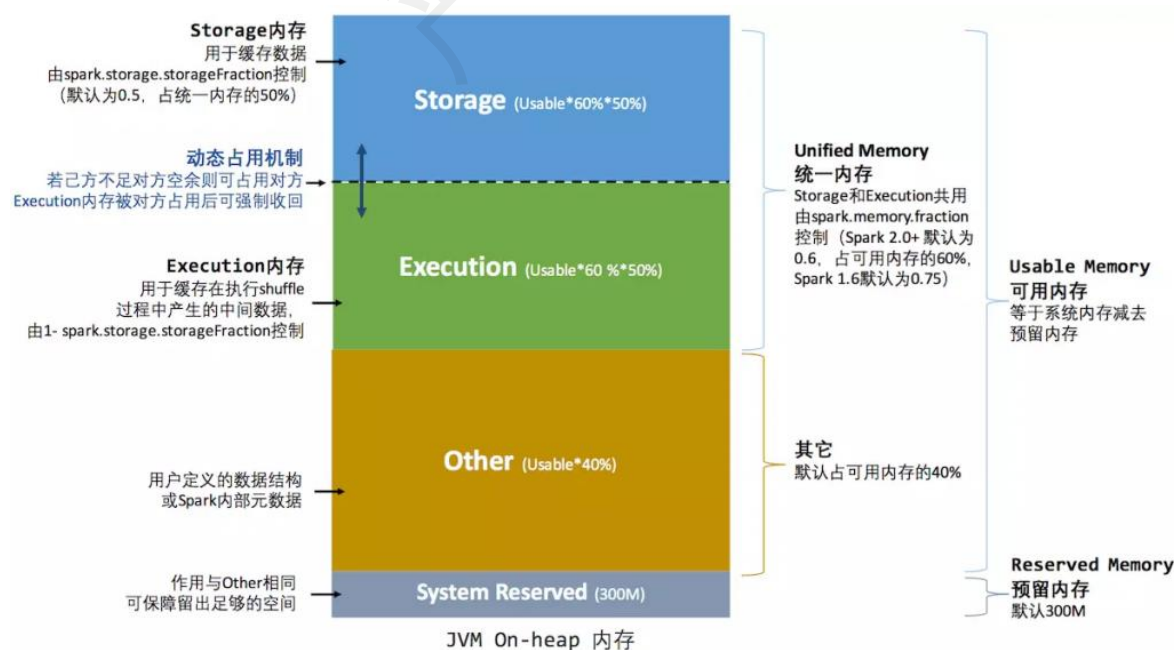
基础上与各自的 `memoryFraction` 参数和 `safetyFraction` 参数相乘得出。上述计算公式中的两个 `safetyFraction` 参数，其意义在于在逻辑上预留出 $1 - \text{safetyFraction}$ 这么一块保险区域，降低因实际内存超出当前预设范围而导致 OOM 的风险（上文提到，对于非序列化对象的内存采样估算会产生误差）。值得注意的是，这个预留的保险区域仅仅是一种逻辑上的规划，在具体使用时 Spark 并没有区别对待，和“其它内存”一样交给了 JVM 去管理。

12.3.3 堆内内存分区(统一方式,现)

默认情况下，Spark 仅仅使用了堆内内存。Executor 端的堆内内存区域大致可以分为以下四大块：

分区	说明
Execution 内存	主要用于存放 Shuffle、Join、Sort、Aggregation 等计算过程中的临时数据
Storage 内存	主要用于存储 spark 的 cache 数据,例如 RDD 的缓存、unroll 数据
用户内存 (User Memory)	主要用于存储 RDD 转换操作所需要的数据,例如 RDD 依赖等信息
预留内存 (Reserved Memory)	系统预留内存, 会用来存储 Spark 内部对象

整个 Executor 端堆内内存如果用图来表示的话，可以概括如下：



对上图进行以下说明：

- `systemMemory = Runtime.getRuntime.maxMemory`，其实就是通过参数 `spark.executor.memory` 或 `--executor-memory` 配置的。
- `reservedMemory` 在 Spark 2.2.1 中是写死的，其值等于 300MB，这个值是不能修改的（如果在测试环境下，我们可以通过 `spark.testing.reservedMemory` 参数进行修改）；
- `usableMemory = systemMemory - reservedMemory`，这个就是 Spark 可用内存；

12.4 堆外内存(Off-heap Memory)

为了进一步优化内存的使用以及提高 Shuffle 时排序的效率，Spark 引入了堆外（Off-heap）内存，使之可以直接在工作节点的系统内存中开辟空间，存储经过序列化的二进制数据。除了没有 `other` 空间，堆外内存与堆内内存的划分方式相同，所有运行中的并发任务共享存储内存和执行内存。

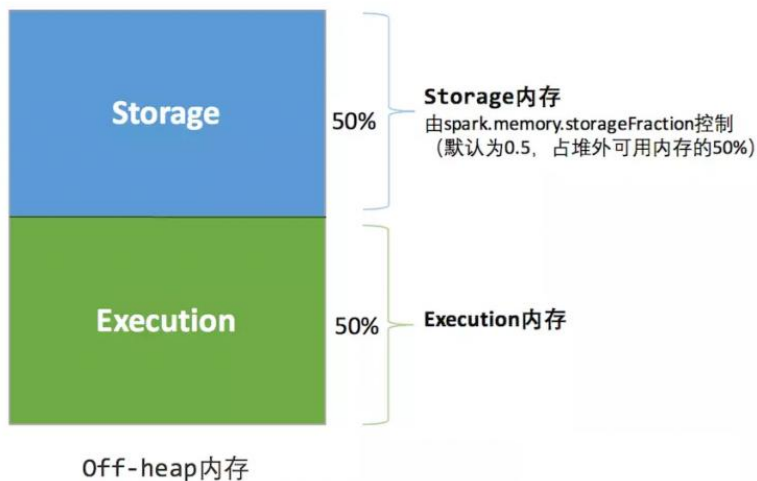
Spark 1.6 开始引入了 Off-heap memory(详见 SPARK-11389)。这种模式不在 JVM 内申请内存，而是调用 Java 的 `unsafe` 相关 API 进行诸如 C 语言里面的 `malloc()` 直接向操作系统申请内存。由于这种方式不经过 JVM 内存管理，所以可以避免频繁的 GC，这种内存申请的缺点是必须自己编写内存申请和释放的逻辑。

12.4.1 堆外内存的优缺点

利用 JDK Unsafe API（从 Spark 2.0 开始，在管理堆外的存储内存时不再基于 Tachyon，而是与堆外的执行内存一样，基于 JDK Unsafe API 实现[3]），Spark 可以直接操作系统堆外内存，减少了不必要的内存开销，以及频繁的 GC 扫描和回收，提升了处理性能。堆外内存可以被精确地申请和释放，而且序列化的数据占用的空间可以被精确计算，所以相比堆内内存来说降低了管理的难度，也降低了误差。

12.4.2 堆外内存分区(静态方式,弃)

堆外的空间分配较为简单，存储内存、执行内存的大小同样是固定的

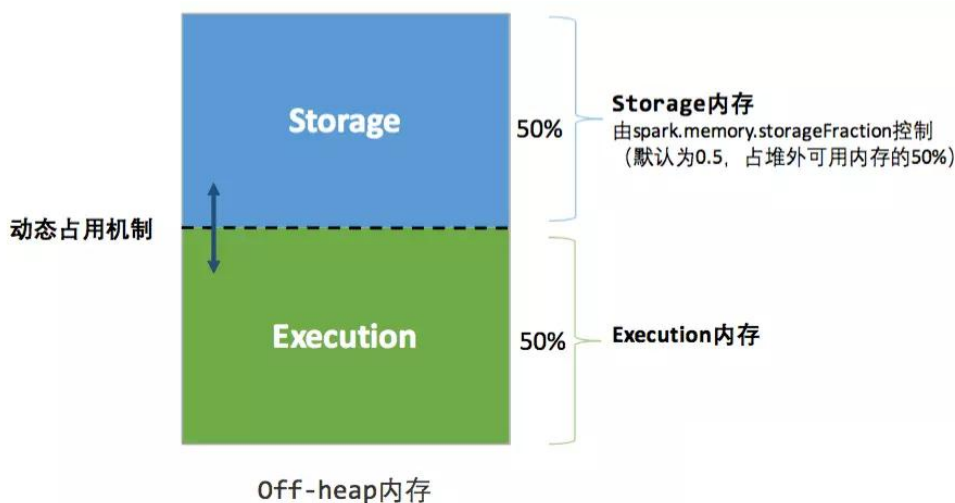


可用的执行内存和存储内存占用的空间大小直接由参数 `spark.memory.storageFraction` 决定，由于堆外内存占用的空间可以被精确计算，所以无需再设定保险区域。

静态内存管理机制实现起来较为简单，但如果用户不熟悉 Spark 的存储机制，或没有根据具体的数据规模和计算任务或做相应的配置，很容易造成“一半海水，一半火焰”的局面，即存储内存和执行内存中的一方剩余大量的空间，而另一方却早早被占满，不得不淘汰或移出旧的内容以存储新的内容。由于新的内存管理机制的出现，这种方式目前已经很少有开发者使用，出于兼容旧版本的应用程序的目的，Spark 仍然保留了它的实现。

12.4.3 堆外内存分区(统一方式,现)

相比堆内内存，堆外内存只区分 Execution 内存和 Storage 内存，其内存分布如下图所示：



关于动态占用机制，由于统一内存管理方式中堆内堆外内存的管理均基于此机制，所以单独提出来讲解。参见文本第三节。

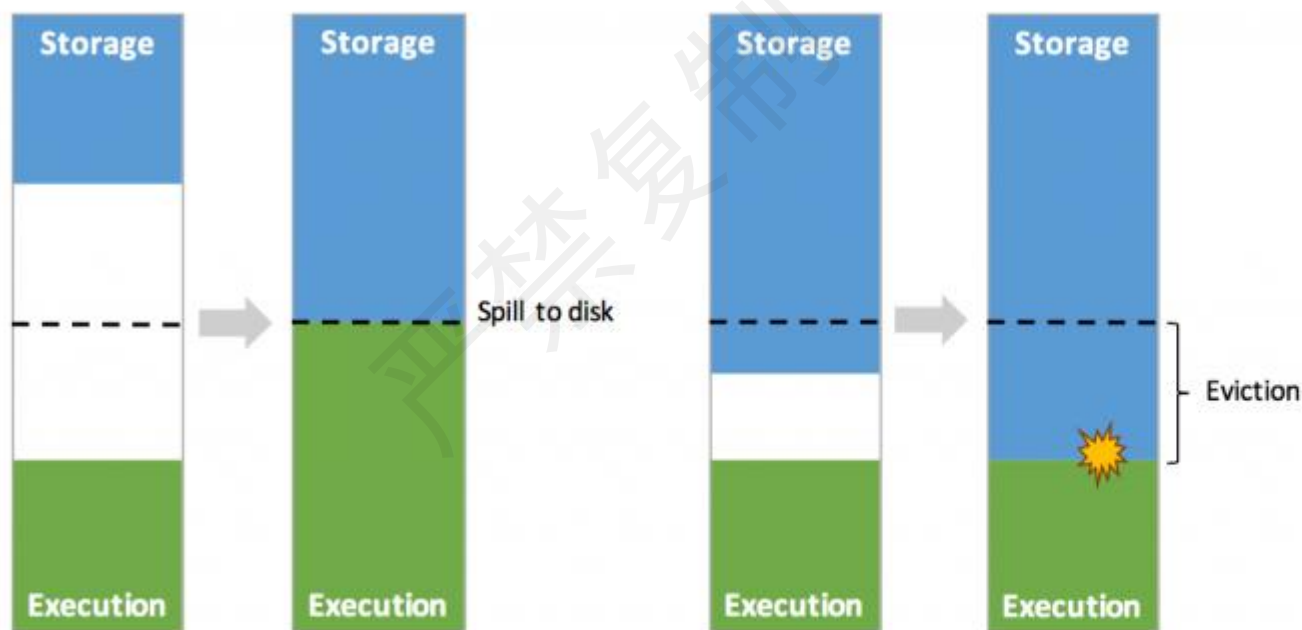
12.5 动态占用机制-Execution 与 Storage

上面两张图中的 Execution 内存和 Storage 内存之间存在一条虚线，这是为什么呢？

在 Spark 1.5 之前，Execution 内存和 Storage 内存分配是静态的，换句话说就是如果 Execution 内存不足，即使 Storage 内存有很大空闲程序也是无法利用到的；反之亦然。这就导致我们很难进行内存的调优工作，我们必须非常清楚地了解 Execution 和 Storage 两块区域的内存分布。

而目前 Execution 内存和 Storage 内存可以互相共享的。也就是说，如果 Execution 内存不足，而 Storage 内存有空闲，那么 Execution 可以从 Storage 中申请空间；反之亦然。所以上图中的虚线代表 Execution 内存和 Storage 内存是可以随着运作动态调整的，这样可以有效地利用内存资源。

Execution 内存和 Storage 内存之间的动态调整可以概括如下：



与静态内存管理一样，当双方空间都被占满后，若有新增内容双方都需要将其存储到硬盘

若己方不足对方空余则可占用对方。Storage占用对方的内存可被淘汰

12.6 动态调整策略

具体的实现逻辑如下：

- 程序提交的时候我们都会设定基本的 Execution 内存和 Storage 内存区域（通过 `spark.memory.storageFraction` 参数设置）；
- 在程序运行时，双方的空间都不足时，则存储到硬盘；将内存中的块存储到磁盘的策略是按照 LRU 规则(Least Recently Used)进行的。若己方空间不足而对方空余时，可借用对方的空间；（存储空间不足是指不足以放下一个完整的 Block）
- Execution 内存的空间被对方占用后，可让对方将占用的部分转存到硬盘，然后”归还”借用的空间
- Storage 内存的空间被对方占用后，目前的实现是无法让对方”归还”，因为需要考虑 Shuffle 过程中的很多因素，实现起来较为复杂；而且 Shuffle 过程产生的文件在后面一定会被使用到，而 Cache 在内存的数据不一定在后面使用。

注意，上面说的借用对方的内存需要借用方和被借用方的内存类型都一样，都是堆内内存或者都是堆外内存，不存在堆内内存不够去借用堆外内存的空间。



12.7 task 内存使用细节—(了解)—

12.7.1 Task 内存申请流程

为了更好地使用内存，Executor 内运行的 Task 之间共享着 Execution 内存。具体的，Spark 内部维护了一个 HashMap 用于记录每个 Task 占用的内存：

当 Task 需要在 Execution 内存区域申请 numBytes 内存，其先判断 HashMap 里面是否维护着这个 Task 的内存使用情况，如果没有，则将这个 Task 内存使用置为 0，并且以 TaskId 为 key，内存使用为 value 加入到 HashMap 里面。

之后为这个 Task 申请 numBytes 内存，如果 Execution 内存区域正好有大于 numBytes 的空闲内存，则在 HashMap 里面将当前 Task 使用的内存加上 numBytes，然后返回；如果当前 Execution 内存区域无法申请到每个 Task 最小可申请的内存，则当前 Task 被阻塞，直到有其他任务释放了足够的执行内存，该任务才可以被唤醒。

每个 Task 可以使用 Execution 内存大小范围为 $1/2N \sim 1/N$ ，其中 N 为当前 Executor 内正在运行的 Task 个数。

一个 Task 能够运行必须申请到最小内存为 $(1/2N * \text{Execution 内存})$ ；当 $N=1$ 的时候，Task 可以使用全部的 Execution 内存。比如如果 Execution 内存大小为 10GB，当前 Executor 内正在运行的 Task 个数为 5，则该 Task 可申请的内存范围为 $10 / (2 * 5) \sim 10 / 5$ ，也就是 1GB ~ 2GB 的范围。

12.7.2 内存分配示例

为了更好的理解上面堆内内存和堆外内存的使用情况，这里给出一个简单的例子

12.7.2.1 只用了堆内内存

现在我们提交的 Spark 作业关于内存的配置如下：

```
--executor-memory 18g
```

由于没有设置 `spark.memory.fraction` 和 `spark.memory.storageFraction` 参数，我们可以看到 Spark UI 关于 Storage

Memory 的显示如下：

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
driver	123.206.77.132:33618	Active	0	0.0 B / 429.1 MB	0.0 B	0	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B		Thread Dump
1	www.it-ebooks.com:50956	Active	0	0.0 B / 10.1 GB	0.0 B	1	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump

Showing 1 to 2 of 2 entries

上图很清楚地看到 **Storage Memory** 的可用内存是 10.1GB，这个数是咋来的呢？根据前面的规则，我们可以得出以下的计算：

```
systemMemory = spark.executor.memory
reservedMemory = 300MB
usableMemory = systemMemory - reservedMemory

StorageMemory= usableMemory * spark.memory.fraction *
spark.memory.storageFraction
```

如果我们把数据代进去，得出以下的结果：

```
systemMemory = 18Gb = 19327352832 字节
reservedMemory = 300MB = 300 * 1024 * 1024 = 314572800
usableMemory = systemMemory - reservedMemory = 19327352832 - 314572800 =
19012780032

StorageMemory= usableMemory * spark.memory.fraction *
spark.memory.storageFraction
= 19012780032 * 0.6 * 0.5 = 5703834009.6 = 5.312109375GB
```

不对啊，和上面的 10.1GB 对不上啊。为什么呢？这是因为 Spark UI 上面显示的 **Storage Memory** 可用内存其实等于 **Execution** 内存和 **Storage** 内存之和，也就是 `usableMemory * spark.memory.fraction`：

```
StorageMemory= usableMemory * spark.memory.fraction
= 19012780032 * 0.6 = 11407668019.2 = 10.62421GB
```

还是不对，这是因为我们虽然设置了 `-executor-memory 18g`，但是 Spark 的 **Executor** 端通过 `Runtime.getRuntime.maxMemory` 拿到的内存其实没这么大，只有 17179869184 字节，

所以 $\text{systemMemory} = 17179869184$ ，然后计算的数据如下：

```
systemMemory = 17179869184 字节
reservedMemory = 300MB = 300 * 1024 * 1024 = 314572800
usableMemory = systemMemory - reservedMemory = 17179869184 - 314572800 =
16865296384

StorageMemory= usableMemory * spark.memory.fraction
= 16865296384 * 0.6 = 9.42421875 GB
```

我们通过将上面的 $16865296384 * 0.6$ 字节除以 $1024 * 1024 * 1024$ 转换成 9.42421875 GB，和 UI 上显示的还是对不上，这是因为 Spark UI 是通过除以 $1000 * 1000 * 1000$ 将字节转换成 GB，如下：

```
systemMemory = 17179869184 字节
reservedMemory = 300MB = 300 * 1024 * 1024 = 314572800
usableMemory = systemMemory - reservedMemory = 17179869184 - 314572800 =
16865296384

StorageMemory= usableMemory * spark.memory.fraction
= 16865296384 * 0.6 字节 = 16865296384 * 0.6 / (1000 * 1000 * 1000) = 10.1GB
```

现在终于对上了。

具体将字节转换成 GB 的计算逻辑如下 (core 模块下面的 `/core/src/main/resources/org/apache/spark/ui/static/utils.js`)：

```
function formatBytes(bytes, type) {
    if (type !== 'display') return bytes;
    if (bytes == 0) return '0.0 B';
    var k = 1000;
    var dm = 1;
    var sizes = ['B', 'KB', 'MB', 'GB', 'TB', 'PB', 'EB', 'ZB', 'YB'];
    var i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(dm)) + ' ' + sizes[i];
}
```

```
}

```

我们设置了 `-executor-memory 18g`，但是 Spark 的 Executor 端通过 `Runtime.getRuntime.maxMemory` 拿到的内存其实没这么大，只有 17179869184 字节，这个数据是怎么计算的？

`Runtime.getRuntime.maxMemory` 是程序能够使用的最大内存，其值会比实际配置的执行器内存的值小。这是因为内存分配池的堆部分分为 Eden, Survivor 和 Tenured 三部分空间，而这里面一共包含了两个 Survivor 区域，而这两个 Survivor 区域在任何时候我们只能用到其中一个，所以我们可以使用下面的公式进行描述：

```
ExecutorMemory = Eden + 2 * Survivor + Tenured
Runtime.getRuntime.maxMemory = Eden + Survivor + Tenured
```

上面的 17179869184 字节可能因为你的 GC 配置不一样得到的数据不一样，但是上面的计算公式是一样的。

12.7.2.2 堆内内存+堆外内存

现在如果我们启用了堆外内存，情况咋样呢？我们的内存相关配置如下：

```
spark.executor.memory          18g
spark.memory.offHeap.enabled   true
spark.memory.offHeap.size      10737418240
```

从上面可以看出，堆外内存为 10GB，现在 Spark UI 上面显示的 Storage Memory 可用内存为 20.9GB，如下：

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	(GC Time)	Input	Shuffle Read	Shuffle Write	Logs	Thread Dump
driver	123.206.77.132:48167	Active	0	0.0 B / 11.2 GB	0.0 B	0	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B		Thread Dump
1	www.it-eblog.com:51908	Active	0	0.0 B / 20.9 GB	0.0 B	1	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump
2	www.it-eblog.com:56932	Active	0	0.0 B / 20.9 GB	0.0 B	1	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump
3	www.it-eblog.com:43380	Active	0	0.0 B / 20.9 GB	0.0 B	1	0	0	0	0	0 ms (0 ms)	0.0 B	0.0 B	0.0 B	stdout stderr	Thread Dump

其实 Spark UI 上面显示的 Storage Memory 可用内存等于堆内内存和堆外内存之和，计算公式如下：

堆内：

```
systemMemory = 17179869184 字节
```

```
reservedMemory = 300MB = 300 * 1024 * 1024 = 314572800  
  
usableMemory = systemMemory - reservedMemory = 17179869184 - 314572800 =  
16865296384  
  
totalOnHeapStorageMemory = usableMemory * spark.memory.fraction  
= 16865296384 * 0.6 = 10119177830
```

堆外

```
totalOffHeapStorageMemory = spark.memory.offHeap.size = 10737418240  
  
StorageMemory = totalOnHeapStorageMemory + totalOffHeapStorageMemory  
= (10119177830 + 10737418240) 字节  
= (20856596070 / (1000 * 1000 * 1000)) GB  
= 20.9 GB
```

12.7.3 几个问题

12.7.3.1 不再细分 unroll, 统一为 storage

MemoryManager 在 storage 内存中细分了 unroll, 静态内存管理的实现划分了 unroll 这部分内存, 并设置了比例。统一内存管理不再细分 unroll, 统一为 storage。

12.7.3.2 为什么设置 300M 预留内存

统一内存管理最初版本 other 这部分内存没有固定值 300M 设置, 而是和静态内存管理相似, 设置的百分比, 最初版本占 25%。百分比设置在实际使用中出现了问题, 若给定的内存较低时, 例如 1G, 会导致 OOM, 具体讨论参考这里 [Make unified memory management work with small heaps](#), 因此, other 这部分内存做了修改, 先划出 300M 内存。

12.7.3.3 spark.memory.fraction 由 0.75 降至 0.6

spark.memory.fraction 最初版本的值是 0.75，很多分析统一内存管理这块的文章也是这么介绍的，同样的，在使用中发现这个值设置的偏高，导致了 gc 时间过长，spark 2.0 版本将其调整为 0.6，详细谈论参见 Reduce spark.memory.fraction default to avoid overrunning old gen in JVM default config。

12.8 总结

如下表：

内存类别	区域划分	管理方式	优缺点
on-heap	- Execution Memory - Storage Memory - User Memory - Reserved Memory	使用 JVM 管理	
off-heap	- Execution Memory - Storage Memory	手动管理，不经过 JVM	可以避免频繁的 GC 但是必须自己编写内存申请和释放的逻辑

13 灵魂 N 问

1. 为什么 scala 的迭代器上的 `map/flatMap...` 算子，具有 `lazy` 执行的特性？

scala 迭代器上的 `map` 等算子，底层并没有调用迭代器的 `hasNext` 和 `next` 方法，而是利用传入的函数 `f` 创建了一个新的迭代器实例；

2. 迭代器算子中传入的计算函数，要怎样才能真正触发执行？

需要调用迭代器上 `foreach` 等“行动”算子（本质上就是要调用迭代器的 `hasNext` 和 `next` 方法）才会真正触发执行；

3. 为什么 rdd 的 `transformation` 算子，具有 `lazy` 执行的特性？

因为 rdd 的 `transformation` 算子，底层并没有触发 `dag` 划分，`stage` 提交，`task` 提交等 `job` 提交的动作，而只是利用算子中所传入的函数 `f`，来创建一个新的 RDD 实例返回；所以并不会真正触发计算任务的执行；

4. 为什么调用 rdd 的 `action` 算子，就能触发计算逻辑的执行？

`action` 算子的底层，都会触发 `sc.runJob` 方法，而该方法，底层会调用 `DAGScheduler` 进行 `dag` 的 `stage` 划分及生成 `taskset`，并继而调用 `TaskScheduler` 执行 `submitTasks` 将 `task` 任务调度给各个 `executor` 去执行；

5. 为什么说 rdd 中并没有真正存储数据？而触发 rdd 上的行动算子时又能拿到计算结果是什么？

rdd 虽然没有真正存储数据，但是 rdd 的 `compute()` 方法将返回该 rdd 所代表的数据的计算逻辑；

触发 rdd 行动算子时，底层会调用 `DAGScheduler` 进行 dag 的 stage 划分及生成 taskset，并继而调用 `TaskScheduler` 执行 `submitTasks` 将 task 任务调度给各个 `executor` 去执行，从而将 rdd 的数据给计算出来；

6. rdd 上的行动算子，是如何获取到 rdd 上的迭代器的？

通过 rdd 的 `compute()` 方法获取

7. task 具体是个什么？它从哪里来？又到哪里去？

task 是 `DAGScheduler` 根据划分好的 stage 生成的，代表着这个 stage 所要执行的计算任务，内含丰富信息：从哪个源头读数据，中间需要做什么函数映射，后面输出到哪里；

task 在 spark 中一共有两种具体实现：`ShuffleMapTask` 和 `ResultTask`；这两者的区别主要是计算结果的输出不同；`ShuffleMapTask` 计算出数据后通过 `ShuffleWriter` 输出 shuffle 文件；而 `ResultTask` 计算出数据后一般是输出到“外部”（如 hdfs，或者收集到 driver 端单机集合，或者打印屏幕等）

8. spark 中的 Task 有哪几种具体实现类？

task 在 spark 中一共有两种具体实现：`ShuffleMapTask` 和 `ResultTask`；这两者的区别主要是计算结果的输出不同；`ShuffleMapTask` 计算出数据后通过 `ShuffleWriter` 输出 shuffle 文件；而 `ResultTask` 计算出数据后一般是输出到“外部”（如 hdfs，或者收集到 driver 端单机集合，或者打印屏幕等）

9. Executor 执行 task，具体来说是怎么个执行法？

executor 收到 Driver 端发送过来的序列化后的 task 字节信息，然后反序列化得到 task 对象，然后将其封装到 TaskRunner 对象中，并放入线程池提交执行；

10. 为什么一个 Executor 中可以同时执行多个 task？

executor 内部是用一个线程池来进行 task 执行的；

11. rdd.flatMap(f1).map(f2).filter(f3).groupByKey(f4).mapValues(f5).map(f6).reduceByKey(f7) 形成了一个怎样的计算逻辑链条？需要划分成几个 stage？

逻辑链条：f1->f2->f3->f4->f5->f6->f7

划分 3 个 stage：分界点在 groupByKey 和 reduceByKey

11. spark 中的 stage 有哪几种？

有 ShuffleMapStage 和 ResultStage 两种；在一个 DAG 中，最后一个 stage 是 ResultStage，其他都是 ShuffleMapStage；

12. 通俗来说，shuffle 究竟干了什么？

shuffle 在当前的 spark 版本中，有 3 中具体方式：BaseShuffleHandle / BypassMergeSortShuffleHandle / SerializedShuffleHandle

无论哪个具体方式，都要做的事情是：

- 在 shuffleWriter 中，调用 rdd 迭代器进行数据计算
- 将计算得到的数据写入 shuffle 缓存（BypassMergeSortShuffle 除外）
- 将 shuffle 缓存数据溢出成本地磁盘文件以便下游任务读取

13. 闭包引用变量，和广播变量有什么区别？

闭包引用变量，会跟随 **task** 对象一起被序列化，因而在每一个 **task** 对象中都会拥有这个闭包引用对象的“拷贝”；

而广播变量，是通过 **spark** 的广播机制序列化后，发给每个 **executor**，从而每个 **executor** 只持有一份“拷贝”，而 **executor** 中 **task** 可以共享这个变量

14. map 端 join 的实现机制是怎样的？reduce 端 join 的实现机制又是怎样的？

map 端 join 的实现机制是，把参与 join 的小表，设法发给处理“流表”（大表）数据的任务节点（**maptask**）；以便在这些任务节点中可以直接完成两表数据的关联；

具体实现方式上，不同运算框架略有不同；

比如 **mapreduce** 中，可以通过给 **job** 添加缓存文件来实现

而在 **spark** 中，可以通过广播变量，闭包引用变量和添加缓存文件的方式来实现

15. 在 spark 编程中，在 rdd 算子的函数中使用了自定义的类型，是否一定需要实现序列化？

通常是需要的；

不过也不一定，如果类的对象是在算子函数中构建出来临时使用，且不需要作为本 **stage** 的输出，

则可以不用实现序列化；

16. 什么是网络服务进程？自己实现一个网络服务进程，关键代码是什么？

就是一个常驻后台运行并具备网络通信功能的服务程序；

自己实现一个网络服务进程，关键代码就是开启 **socket** 服务，绑定地址监听端口，然后等待客户端

请求，并为各类请求编写自己的处理逻辑即可；

17. 什么是客户端？自己实现一个客户端程序，关键代码是什么？

在网络通信中，发出请求来获取服务的一端，通常称之为客户端；

自己实现一个客户端程序，关键代码是 `new Socket(服务端地址, 端口)` 来创建连接，然后通过 `socket` 连接获取输入输出流，通过流来向服务器发送信息，并接受服务端的返回响应，并根据响应编写自己的业务处理逻辑即可；

18. 如何实现用客户端向服务端发送一个对象？

将对象序列化成字节数组，然后发送即可；

19. spark 中的 driver 究竟是个怎样的角色？

`driver` 是 `spark` 程序运行时的实质“主管”角色。其核心是 `dagscheduler` 和 `taskscheduler`；这两个 `scheduler` 负责将用户编写的 `RDD` 计算链条（`DAG`），以 `shuffle` 为分界点划分 `stage`，并将 `stage` 转化成 `taskSet`，并调度给 `executor` 去执行；

在底层源码中，`driver` 是一个线程；根据运行部署模式，该线程可以在 `job` 提交客户端进程（`SparkSubmit`）中被启动执行，也可以在 `ApplicationMaster` 中被启动执行；而 `Driver` 这个线程中，主要就是执行用户主类的 `main` 方法（从而创建 `SparkContext`，从而初始化了 `dagscheduler` 和 `taskScheduler`）

20. spark 中的 executor 又是一个怎样的角色？

`executor` 是 `spark` 运行时的真正执行计算任务的进程;它收到 `driver` 调度过来的 `task`,并将 `task` 封装成 `taskRunner` 并放入线程池中执行;

21. `Ordering` 是个什么东东? `Ordered` 又是什么东东?

`Ordering` 类似于 `Java` 中的 `Comparator`, 提供一个“第三方”比较器, 用来比较两个对象的大小;

`Ordered` 类似于 `Java` 中的 `Comparable` 接口, 一个类如果实现了 `Comparable` 接口, 则这个类的对象自身就具备调用 `compareTo` 方法来跟另一个对象进行比较的功能;

不过在 `scala` 中, 因为有内置定义好的隐式转换, 从而让 `Ordering` 和 `Ordered` 可以非常方便地自动隐式转换;

22. `Comparator` 是个什么东东? `Comparable` 又是什么东东?

`Comparator` 是 `JDK` 中的一个接口, 通过实现该接口来定义一个“第三方”比较器, 用来比较两个对象的大小;

`Comparable` 也是 `JDK` 中的一个接口, 一个类如果实现了 `Comparable` 接口, 则这个类的对象自身就具备调用 `compareTo` 方法来跟另一个对象进行比较的功能;

23. `Iterator` 是个什么东东? 里面有什么内容?

`Iterator` 是 `JDK` 中对迭代获取数据过程的抽象接口; 里面定义了 `hasNext` 和 `next` 这两个核心的方法; 通过实现 `Iterator` 接口, 可以让我们封装屏蔽掉底层数据读取的具体逻辑, 为上层开发提供一个统一的数据访问方式;

24. 你觉得集合和 **Iterator** 有什么区别？有什么关联？

集合本质上是一种特定性的数据存储，集合总是以某种特定的数据结构将数据存储在内存中；而 **Iterator** 则是数据访问过程的封装，是为了屏蔽底层数据存储的数据访问细节、为上层应用提供统一简便的访问方式而生；

25. 分布式并行计算中，为什么会引入 "分区 **partition**"， "任务分片 **split**" 等概念？

在分布式并行计算中，一个源数据集往往要交给很多个并行 **task** 实例来计算处理。而如果不进行数据分片（对应任务分片），则这些并行的 **task** 不能很好地划分数据处理职责，产生重复处理，从而失去了并行的意义；

26. 分布式并行计算中，为什么需要 **shuffle** 这么个玩意儿？

分布式并行计算中，一个源数据集往往划分成多个任务片，交给多个并行 **task** 实例来计算处理；而这样一来，这些 **task** 各自都只堵到了源数据集的一部分数据，从而没法得到需要全局汇总的结果；从而在分布式计算中，往往需要把计算流程划分成上下游阶段，上游各自处理一部分源数据，然后把数据进行混洗（**shuffle**）后，交给下游 **task** 处理，以便能获得所需要的全局汇总结果；

27. 在泛型定义中， $A >: B$ ， $A <: B$ ， $A : B$ ， $A <\% B$ 分别表示什么？

$A >: B$ ，表示参数类型的下界是 **B** 类型

$A <: B$ ，表示参数类型的上界是 **B** 类型

$A : B$ ，上下文界定，表示参数类型 **A**，要存在一个隐式转换可以换成 **B** 类型

$A <\% B$ ，视图界定，表示可以传入所有的 **B** 子类型，及可隐式转换到 **B** 的类型

29. 在泛型定义中, **+A** , **-B** 分别表示什么含义? 它们有什么意义?

C[+T]: 如果 **A** 是 **B** 的子类, 那么 **C[A]** 是 **C[B]** 的子类, 称为协变

C[-T]: 如果 **A** 是 **B** 的子类, 那么 **C[B]** 是 **C[A]** 的子类, 称为逆变

一句话总结: 协变类型参数可用作接口的方法的返回类型; 逆变类型参数可用作接口的方法的参数类型。从而让接口更具扩展性和确定性;

比如, 假设 **Person** 继承自 **Animal**;

协变: 一个接口方法如果能返回 **List[Animal]**, 那么可以返回 **List[Person]** 也必将是接口调用者所能接受的;

逆变: 一个接口方法如果能接受 **List[Person]**, 如能让其也能接受 **List[Animal]**, 那将更具扩展性;

30. 函数定义 和 函数调用, 有什么区别?

函数定义是描述函数的处理逻辑; 定义的时候并不会触发函数内逻辑的执行;

函数调用, 是用定义好的函数去执行逻辑处理;

31. 什么是 DAG/stage/taskset/task

DAG 就是 spark 开发中, 用户所编写的 RDD 之间的整个转化关系;

stage 是 DAG 中一段可以放在同一个 task 中执行的逻辑, 包含若干个 RDD; stage 的划分依据是两个 RDD 之间的计算逻辑是否需要 shuffle;

task, 是一段计算逻辑的封装对象;

taskset, 是一个 stage 根据 rdd 分区并行度转化成的多个 task

32. 什么时候需要 shuffle ?

在 spark 源码中, 如果上下游两个 RDD 之间是 ShuffleDependency (宽依赖), 则需要 shuffle;
广义上来说, 在分布式的场景中, 一段计算逻辑必须要划分成上下游任务并在上下游任务之间进行数据的混洗, 则需要 shuffle;

33. spark 中的 RDD 依赖关系有哪几种, 分别有什么特点?

RDD 依赖关系有 NarrowDependency (窄依赖) 和 ShuffleDependency (宽依赖)

而 NarrowDependency 又有两种子类 (OneToOneDependency 和 RangeDependency)

ShuffleDependency: 父 RDD 的任一个分区, 都要被子 RDD 的所有分区所依赖;

OneToOneDependency: 子 RDD 的一个分区, 固定映射 (依赖) 父 RDD 的一个分区;

RangeDependency: 子 RDD 的一个分区, 固定映射父 RDD 的一段号码连续的分区;

其实在 spark 中还有一个 NarrowDependency 的匿名实现, 就是在 coalesce 算子中形成的一种依赖, 这种窄依赖中, 子 RDD 的一个分区会固定映射父 RDD 中的若干个分区, 映射规则由数据量平衡算法决定

34. spark 中划分 stage 的流程是怎样的?

DAGScheduler 根据触发行动算子的 rdd (finalRdd), 生成 finalStage

然后在 submitStage 中, 用递归算法, 逐级回溯得到各级前序 stage, 并先提交最前序的 stage, 然后逐级提交后序的 stage;

35. reduceByKey 算子是不是一定带来 shuffle? 什么情况下不会产生 shuffle?

不一定，reduceByKey, join, groupByKey 等算子中，都会判断父 RDD 的 partitioner 是否与算子中传入的 partition（即算子所产生的的子 RDD 的 partitioner）一致，如果一致，则不需要 shuffle;

36. 负责划分 stage 的模块叫什么？负责调度 task 的模块叫什么？

划分 stage 的模块叫 dagscheduler;

负责调度 task 的模块叫 taskScheduler;

37. RDD 的 partitioner 与分区数，是如何决定的？

详见教案;

